



Security Assessment

Final Report



Aftermath Finance

August 2026

Prepared for Aftermath Finance

Table of Contents

Project Summary.....	6
Project Overview.....	6
Protocol Overview.....	8
Assessment Methodology.....	10
Threat and Security Overview.....	11
Actors.....	11
Privileged (assumed honest, but treated as compromise targets):.....	11
Semi-trusted.....	11
External.....	11
Assets.....	12
Trust Boundaries and Assumptions.....	12
Attack Vectors considered.....	13
Oracle price integrity.....	13
Perpetuals accounting and funds.....	14
Vault fairness and LP funds.....	15
Liveness and denial of service.....	16
Governance, upgrades, and authority.....	17
Findings Summary.....	19
Severity Matrix.....	19
Detailed Findings.....	20
Critical Severity Issues.....	29
C-01: Unbounded atomic expired-order cleanup lets attackers permanently gas-lock an order book....	29
C-02: Trading PnL collateral-price drift can make position claims exceed the real vault balance.....	34
C-03: Routine funding can break collateral-token conservation in inverse markets and make the ClearingHouse vault insolvent.....	41
High Severity Issues.....	47
H-01: A semi-trusted vendor's per-market collateral-oracle update locks funds across an entire market-making vault.....	47
H-02: Signed Pyth Lazer payloads can make a carried-forward price look fresh.....	53
Medium Severity Issues.....	57
M-01: Downcast from u256 to u64 could make positions unliquidatable.....	57
M-02: Settlement price can be changed after settlement closes have begun, breaking value conservation.....	59
M-03: Default bad-debt thresholds make any insurance-fund-exceeding bad debt revert public liquidation, leaving recovery only to permissioned ADL.....	62
M-04: Bad debt socialization can cause vault shortfall in certain cases.....	65
M-05: ADL can leave a flat position with negative collateral that settlement cannot resolve.....	67
M-06: Missing hard cap on branch_max/leaf_max enables permanent orderbook DOS.....	69
M-07: A semi-trusted vendor's first market registration initializes the package-global collateral oracle	

used by every vault for that collateral.....	71
M-08: Vault controllers can rewrite existing LPs' fee and withdrawal terms.....	74
M-09: u64 cancellation totals can make a large losing position impossible to liquidate.....	76
M-10: Small force-withdraw requests can cancel all vault orders across markets.....	83
M-11: Force withdraw applies the collateral haircut before combining collateral and funding.....	87
M-12: Freezing a lagging-version registry or clearing house can make it unfreezable.....	92
M-13: Liquidation bad-debt socialization can under-recover or over-charge collateral in inverse markets.	94
M-14: Maker Solvency Rejections Can Gas-Lock the Order Book.....	101
M-15: Reduce-Only Order Churn Can Repeatedly Block Liquidation.....	104
M-16: Rolling blend masks a stale leg by returning the maximum leg timestamp, letting stale prices bypass the aggregator's staleness guard.....	107
M-17: Pyth integration completely ignores the price confidence interval, exposing downstream protocols to highly uncertain prices.....	109
M-18: Rolling-feed maintenance operations lack protocol-level parameter and state-machine validation..	111
M-19: Immediate collateral-haircut increases can manufacture liquidation eligibility for solvent positions.	114
M-20: Resting Perpetuals orders remain executable during a force-withdraw pause and can censor exits.....	116
M-21: Permissionless CoinRegistry::migrate_legacy_metadata can be front-run to permanently block vault creation.....	118
M-22: Liquidation sizing subtracts an insurance-fund fee that settlement waives, over-liquidating solvent (negative-raw-collateral / positive-equity) positions.....	120
M-23: Tightening the book-spread limit leaves the liquidation mark at the obsolete wider bound, enabling permissionless liquidation of healthy positions.....	126
M-24: Dust-sized base oracle prices let attackers realize inflated short PnL and drain vault collateral	129
M-25: Vault reads the collateral-oracle price with no strictly-positive guard, so a zero collateral price freezes every LP entry and exit.....	132
M-26: Funding/TWAP parameter changes are applied retroactively to un-accrued funding and to the liquidation mark.....	134
M-27: Completing any force-withdraw clears the vault's force-withdraw pause unconditionally, letting the owner repeatedly censor the trustless exit.....	137
M-28: A fill resets the consecutive zero-fill cap, leaving total limit-order traversal dependent on taker size.....	140
M-29: Changing the funding frequency before the old boundary can charge intervals that exist only on the new grid.....	142
M-30: Passive maker fills bypass post-fill margin and direction controls.....	146
M-31: Full-risk trading during closed equity sessions can externalize reopening-gap losses.....	149
M-32: Deposits price a closed-market position with a live mark and can dilute existing LPs.....	153
M-33: A vault owner can capture part of a deposit through LP share rounding.....	156
Low Severity Issues.....	159
L-01: Insurance-fund surplus reserve uses live long-only open interest, drainable during settlement..	159



L-02: Collateral oracle price has no spot-vs-TWAP divergence guard, unlike the base oracle.....	163
L-03: Public settlement lets early closers escape bad-debt socialization applied later, stranding the last closers.....	166
L-04: Stale dynamic-field state after AuthorityCap destruction.....	168
L-05: Vault deposits abort on any single negative-margin market position, halting all new LP capital..	171
L-06: A matured margin-ratio proposal can become simultaneously un-committable, un-deletable, and block creating new proposal.....	173
L-07: receive_from_account lets a non-ADMIN assistant claim received objects and skips the package-version gate.....	176
L-08: Final funding after market close can be diluted by a zero-premium settlement update.....	178
L-09: Cross-Vendor Rolling Feed Object Can Be Bound During Provisioning.....	181
L-10: twap_period_ms accepts unbounded values, letting a trusted user neutralize or freeze the TWAP safeguard.....	183
L-11: Transferable loss basis can be joined into profitable LP positions to eliminate performance fees.....	185
L-12: Missing upgrade_version for Config prevents deprecating old package bytecode.....	187
L-13: Limit orders revert on partial fills if the remaining posted size is below min_order_usd_value....	189
L-14: VendorMetadata setters omit version-gating, allowing users to bypass package upgrades.....	191
L-15: Emergency pause and freeze states prevent revocation of compromised vault capabilities.....	192
L-16: Curator metadata fields use Option::fill and become permanently write-once.....	194
L-17: OwnerWithdraw events report the pre-fee amount instead of the user's actual proceeds.....	195
L-18: Vault metadata setters are not gated on the vault's version or pause state, allowing mutation of a frozen vault.....	197
L-19: 10u64.pow arithmetic panics and permanently DoSes the price feed for tokens with exponents >= 20 or <= -20.....	199
L-20: Integration upgrades cannot disable vulnerable entry points from prior package versions.....	201
L-21: FeedInfoObject constructors return an owned object while permanently claiming a deterministic identity, letting a revocable assistant brick a feed.....	203
L-22: update_price_feed unconditionally emits UpdatedPriceFeed events for stale prices, poisoning off-chain indexers.....	206
L-23: A revocable package assistant can permanently capture or exhaust source registrations.....	207
L-24: A paused market still blocks permissionless vault force-withdrawals.....	208
L-25: Dust-path force-withdraw overcharges the fractional withdrawer by making them absorb the full whole-position close cost.....	211
L-26: Unbounded max_one_execution_amount_bps causes a u64 multiplication overflow that permanently bricks execution of a committed TWAP order.....	214
L-27: TWAP per-slice size is not aligned to the market lot size, so execution reverts and silently bricks the order.....	216
L-28: Accepted fee settings can make rounded maker rebates larger than taker fees and abort a trading session.....	219
Informational Severity Issues.....	222
I-01: Price update from oracle with timestamp far in the future means this value can't be updated.....	222



I-02: Validating against empty RESTRICTED_KEYS.....	224
I-03: Dead public(package) functions across multiple packages.....	225
I-04: Redundant positivity checks in set_config parameter validation.....	228
I-05: Mark price can be manipulated by up to `max_book_index_spread` by trader.....	229
I-06: Oracle freeze documentation does not match downstream behavior.....	231
Disclaimer.....	233
About Certora.....	233

Project Summary

Project Overview

This document describes the security review of **Aftermath Finance**. The work was undertaken from **June 8th, 2026** to **August 17th, 2026**.

Project Name: Aftermath Finance Perpetuals

Platform: Sui Network

Repository Link	Files in Scope	Branch	Initial Commit Hash	Final Commit Hash
move-iperps	packages/perpetuals/sources/* */*.move packages/position/sources/**/* .move	audit/certora	547a404	0a090ee
move-market-making-vault	packages/market-making-vault /sources/**/**.move packages/coins/**/sources /**/**.move	audit/certora	d699a61	b80f07f
move-oracle-aggregator	packages/oracle-aggregator/sources/**/**.move packages/integrations/*/ sources/**/**.move packages/integrations/*/ sources/**.move	audit/certora	97aeca8	5a59882

move-ifixed	packages/ifixed/sources/ **/*.move	audit/certora	198c4cd	198c4cd
move-ordered-map	sources/ordered_map.move	audit/certora	44d404c	44d404c
move-vendor	packages/vendor/sources/**/. move packages/templates/*/sources/ **/*.move	audit/certora	f018c18	3c85413
move-control-flow	packages/control-flow/sources /**/*.move	audit/certora	22f0f91	22f0f91
move-enum-option	sources/**/*.move	audit/certora	58fbad4	58fbad4
move-authority-cap	packages/authority-cap/source s/**/*.move	audit/certora	aaadde5	579ad1f

There are some dependencies repositories but out of scope:

- aftermath-pyth-crosschain
- aftermath-test-coins
- aftermath-move-test-utilities

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following pages.

Protocol Overview

Aftermath Perpetuals is an on-chain perpetual-futures exchange built on Sui in Move 2024. Traders take long or short exposure to an oracle-priced base asset against USD-denominated collateral without ever holding the underlying, and orders are matched by a central-limit order book held entirely on chain rather than by an AMM curve.

Each market is a shared `ClearingHouse<T>` object generic over its collateral coin `T`. It holds the market's configuration and state, the order book (a custom B+ tree from the `ordered-map` package), a commingled vault carrying every trader's collateral together with the market's insurance fund, and each trader's position as a dynamic field. All trading flows through a hot-potato session: a caller opens a session, places or cancels orders or performs a liquidation, and must close the session in the same transaction, at which point margin, funding, fees, and open interest are reconciled. Risk is managed by initial and maintenance margin ratios, periodic funding, permissionless liquidation, and an auto-deleveraging path plus loss socialization when bad debt exceeds what the insurance fund can absorb.

A separate market-making vault lets liquidity providers pool collateral so that a keeper can act as counterparty across several markets on their behalf. LP shares are minted and redeemed against the vault's net asset value, which is recomputed per market inside each deposit or withdrawal session. Alongside the ordinary keeper-driven withdrawal the vault offers a trustless force-withdraw, which lets an LP unwind their share and exit after a configured delay without keeper cooperation.

Prices come from an oracle aggregator that normalizes Pyth, Pyth Lazer, Stork, and Switchboard feeds into a single signed fixed-point representation. All protocol arithmetic uses `ifixed`, an 18-decimal signed fixed-point type packed into `u256`. Authorization is capability-based rather than address-based: `AuthorityCap<Context, Role>` values, scoped to a package, a vendor, an account, or a vault, are what grant every privileged action, so custody of a capability is equivalent to holding the authority it represents.

The features that distinguish the protocol, and that shaped where this review spent its effort, are:

- **Fully on-chain central limit order book.** Matching runs inside Move smart contracts over a custom B+ tree, with no off-chain sequencer, so order placement, matching, and settlement are all verifiable on chain.

- **User-owned collateral accounts with isolated margin.** Collateral sits in per-user accounts rather than a shared pool, and each market position is margined in isolation.
- **A full order-type set.** Standard limit, post-only, immediate-or-cancel, and fill-or-kill orders, plus market orders, all reduce-only capable.
- **Committed automated orders.** TWAP orders and stop-loss / take-profit tickets are escrowed objects whose parameters are hash-committed and driven forward by a permissioned executor list chosen by the order's owner.
- **Median mark price.** The mark used for margin and liquidation is the median of the order-book price, the funding price, and a TWAP-derived price, so no single input decides it alone.
- **Permissionless liquidation with a four-layer loss cascade.** Anyone can liquidate an under-margined position for a fee, and losses beyond that are absorbed in order by partial liquidation, the insurance fund, socialization, and finally auto-deleveraging.
- **The afLP market-making vault.** LPs pool collateral for a keeper to market-make across many markets, earning a share of trading fees and vault PnL, with shares priced off NAV.
- **Trustless force-withdraw.** After a configured delay an LP can unwind their share and exit without keeper cooperation, which is the vault's guarantee against operator inaction.
- **A multi-vendor market model.** Independent vendors create and operate their own markets under scoped, revocable capabilities, sharing one package and one registry.
- **An integrator fee program.** Front ends register for an integrator id and earn a share of taker fees, capped per account by a ceiling the account holder sets.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

This chapter enumerates the actors, assets, trust boundaries, and threats considered during the review. Protocol mechanics are described only to the extent needed to frame each risk; see the preceding design chapter for full behaviour.

Actors

Privileged (assumed honest, but treated as compromise targets):

- Package admins (per package) — highest authority: upgrades, configuration, and kill switches.
- Package assistants — delegated, revocable operational roles.
- Vendors — independent protocol operators that create and configure their own markets and oracle-feed objects under vendor-scoped capabilities. Their privileged roles include admin, assistant, and maintenance.
- Account authorities — admin/assistant capabilities controlling a perpetual trading account.
- Pause and freeze guardians — emergency responders able to halt activity.
- ADL operators — holders of the package-scoped ADL capability who execute the protocol's permissioned bad-debt recovery process.

Semi-trusted

- Oracle publishers / signers — off-chain price producers whose signatures the chain verifies.
- Vault operator / owner — semi-trusted, permissionless vault creators with vault-scoped administrative powers, runs the market-making strategy, sets vault parameters, and services withdrawals.

External

- Liquidity providers (LPs) — deposit collateral, hold transferable shares, and may force-withdraw.
- Traders — makers and takers.

- Relayers / keepers — permissionless actors who push signed oracle updates and trigger funding, liquidations, and ADL.
- Liquidators — permissionless, fee-incentivized.
- Integrators — route orders and earn fees.

Adversaries considered: malicious traders, malicious or opportunistic LPs, malicious relayers and sequencing/MEV actors, a malicious or compromised vendor, a compromised privileged key, and an honest operator supplying erroneous inputs.

Assets

- LP capital and vault funds, and the fair per-share value of vault positions.
- Trader collateral held in perpetual accounts.
- Insurance fund — the backstop for bad debt and settlement shortfalls.
- Protocol, vendor, and operator fees.
- Oracle price integrity — accuracy, freshness, and correct asset binding of every feed.
- Accounting integrity — correctness of margin, funding, PnL, and open-interest state.
- Liveness — the ability to trade, deposit, withdraw, liquidate, settle, and update prices.
- Authority / capabilities — the governance and operational rights gating all of the above.

Trust Boundaries and Assumptions

- Oracle signers are trusted for price truth. The chain verifies signatures and signer validity, not whether a signed price is correct or timely; all downstream safety inherits this.
- Privileged roles are trusted but keys are fallible. Admin, assistant, maintenance, and operator keys are expected to behave honestly, but the model assumes any may be compromised or supply incorrect inputs.
- Vendors are intended to be isolated trust domains — one provider's misconfiguration or compromise should not affect another's feeds.
- Current deployment assumes no external market vendors: Several accepted risks become more important if third-party vendors are enabled
- Market vendors and maintenance roles are trusted configuration roles: Immediate changes to haircuts, spread limits, funding settings, or feed mappings may affect existing positions.
- Package upgrades are additive. Publishing a new version does not remove old code; superseded logic remains callable unless explicitly retired by on-chain version state.

- Collateral price changes can cause USD-denominated accounting and token-denominated settlement to diverge. To avoid solvency issues caused by collateral price changes, the team has announced they will limit collateral to USD stablecoins.
- Transactions are atomic, but multi-step flows can be interleaved. A single transaction may consume many oracle and market objects; sessions that span multiple transactions can be raced by others between steps.
- Liquidators and keepers must act in time: Fast price moves, poor book liquidity, congestion, or transactions that repeatedly abort can leave losses for the insurance fund or other users.
- Closed equity markets intentionally keep the last oracle price: Orders may still be placed while the market is closed. The book and mark price may move only within the configured spread, but that spread does not limit the price gap when the oracle becomes fresh again.
- Vault users trust the vault owner: Existing fee rates, lock periods, and forced-withdrawal delays can be changed for existing LP positions (See [M-08](#))
- Users trust the executors they list for automated orders: In particular, a market stop order does not commit to a worst execution price.
- An oracle package freeze is not a global price stop: Some direct readers can continue using the last stored quote until their own freshness or safety check fails. Emergency procedures must pause downstream markets separately.

Attack Vectors considered

Oracle price integrity

- Stale or halted inputs treated as fresh — a blended synthetic price can inherit the freshness of its most recent leg rather than its most stale one, and a message-level timestamp can advance while the underlying price has not, masking staleness. Consequence: markets price against outdated values, causing wrong or missed liquidations and mispriced entries.
- Feed substitution / redirection — a maintenance action or a mis-bound feed object can point a live symbol at an unrelated but validly signed feed without changing its on-chain identity, so downstream markets do not detect the swap.
- Cross-vendor coupling — a feed can end up governed by another vendor's maintenance authority, breaking vendor isolation.

- Replay / sequencing of archived signed updates — outdated but validly signed updates can poison time-weighted averages or pollute the off-chain event stream, diverging off-chain consumers from on-chain state.
- Ignored uncertainty — discarding a feed's confidence signal lets highly uncertain prices be treated as precise.
- Superseded oracle code remaining reachable — because old package versions persist, a fix shipped only as an integration upgrade can be bypassed by invoking the old version on the same live objects.
- A signed timestamp far in the future can prevent later normal updates until the feed is reset ([I-01](#))
- A very small but positive base price can reduce margin requirements and let cooperating accounts turn an inflated short profit into real collateral ([M-24](#)).
- A trader can move the book-based part of the mark price within `max_book_index_spread`; this affects liquidation prices and vault valuation ([I-05](#)).
- Extreme or steered prices — a tiny but positive base price can collapse margin requirements and inflate short-side profit, while attacker-controlled quotes can move the book component of the mark price or trigger stop orders from a non-traded midpoint.

Perpetuals accounting and funds

- Margin enforcement — every path that establishes or grows exposure (order admission, maker fills, taker fills, position flips, collateral deallocation) must leave the position backed by its required margin. Asymmetries between admission-time and fill-time checks, or between maker and taker paths, are prime targets for establishing exposure the collateral does not back.
- Solvency and bad-debt containment — positions must not become insolvent faster than they can be liquidated, and liquidation must remain economically incentivized. Threats: adverse moves outrunning liquidation, incentives that can be driven to zero, and any path that commits a position with no cushion, so unbacked losses fall on the insurance fund or remaining users.
- Collateral and PnL conservation — trader PnL must net to zero and pooled collateral must cover all obligations. This invariant is threatened by rounding and fixed-point precision, unit-scaling mismatches, asymmetric fee rounding, funding redistribution errors, and divergence between USD-denominated accounting and collateral-token settlement (including a collateral depeg)

- Collateral valuation — collateral must be valued conservatively across its full range (haircut application, collateral-oracle pricing, and negative- or zero-crossing balances). Mispricing here lets an actor over-borrow against, or over-withdraw from, a position.
- Funding integrity — funding accrual, timing, and settlement must be resistant to being skipped, mis-timed, or steered, since these transfer value between longs and shorts and between traders and the vault. Changing the funding frequency or price-averaging window without first checkpointing the old state can create, erase, or misprice several funding intervals.
- Liquidation, settlement, and ADL fairness — forced position changes (liquidation, market settlement, auto-deleveraging) must fire only when warranted, close at fair prices, and neither seize value from solvent positions nor leave unresolved residue; each is also a grieving surface that can be blocked or delayed.
- Value extraction via ordering and cooperating accounts — self-trading, wash trading, coordinated maker/taker fills, and ADL gaming must not be usable to move value between controlled accounts, evade risk checks, or harvest fees and rebates.
- Delayed maker fills and inconsistent risk snapshots — a resting order may be safe when posted but fill later at a worse price without being rechecked against the intended initial or maintenance margin. A matching session may also reuse stale exposure and pending-order snapshots, allowing concentration or relative-margin limits to be bypassed.
- Closed-market gap exposure — when an equity feed remains usable at its last closing price, an attacker can prepare opposite positions in isolated accounts before a scheduled closure. A large reopening gap can make one account profitable while the other leaves bad debt for the insurance fund or loss-sharing process.

Vault fairness and LP funds

- Fair share valuation — shares must be minted on deposit and redeemed on withdrawal at a net asset value that reflects the vault's true holdings and is resistant to stale or manipulated pricing. Threats: valuation on stale/manipulable oracle state, share inflation or dilution, and first-depositor or rounding edge cases that let one party mint or redeem at an unfair rate.
- Value transfer between LPs — no share holder should be able to gain at another's expense through deposit/withdraw timing, share splitting or joining, cost-basis manipulation, or MEV around vault operations. Deposits and redemptions must not be sandwichable around NAV-moving events.

- Inconsistent valuation snapshots — a deposit or withdrawal can record the collateral price at one point but value individual markets later using newer marks, or mix settlement-time and live collateral prices. An attacker who times price updates between those steps can mint or redeem shares against an internally inconsistent NAV.
- Redemption liveness and exit guarantee — every LP must be able to realize their fair share, ultimately without operator cooperation. Threats: the operator withholding service, censorship or blocking of the trustless exit path, exits degraded or stalled by market illiquidity, and LPs being forced to accept unfair slippage to leave.
- Risk conservation on exit — a withdrawing participant must take a proportional share of both value and position risk; partial exits must not leave the residual shared position at worse leverage or a thinner margin profile for the LPs who remain.
- Fee integrity — performance and management fees must be assessed correctly and be neither avoidable nor over-charged. Cost-basis and high-water-mark accounting must remain sound across transfers, joins, and splits of share tokens so that fee obligations cannot be shed or double-counted.
- Operator authority bounds — the operator drives strategy but must not be able to move vault funds outside the intended deposit/allocate/withdraw flows, extract value from LPs, or grief them beyond ordinary strategy risk; the vault's trading-account capability must never serve as a drain path.
- Lifecycle and state-control integrity — vault creation, share-token issuance, configuration changes, and the operational pause/freeze must not be grievable, front-runnable, hijackable, or strippable in ways that block LP entry or exit or destabilize the state a redemption depends on.
- Forced-withdrawal isolation — one LP's exit must not cancel every pending vault order, force that LP to bear the full cost of closing a shared dust position, or become blocked by one paused market. An owner must also not be able to reset the withdrawal pause repeatedly to prevent completion of the trustless exit.

Liveness and denial of service

- Availability of critical operations — trading, collateral movement, price updates, funding, liquidation, and settlement must remain executable under adversarial conditions. The core threat is any actor driving a required operation to abort or become uncallable, degrading the exchange or blocking risk reduction.
- State and resource exhaustion — unbounded growth of on-chain structures (order books, dynamic-field collections, per-account or per-market lists) can raise gas cost until

operations fail. Griefers may deliberately bloat these structures to gas-lock a market or a code path.

- Hostile order-book state — expired, non-filling, or rejected maker orders can force matching to spend its gas budget scanning and cleaning before it reaches usable liquidity. If a market order requires a full fill before cleanup is committed, each failed transaction restores the same hostile entries and can repeatedly block market orders, liquidations, and forced withdrawals.
- Poison-state griefing — placing the system in a state where a required operation always reverts: an entry that can never be cleared, a counter that can be exhausted, or an input that triggers an arithmetic fault. The hazard is amplified where no party can undo the condition.
- Contention and serialization — shared mutable objects that funnel unrelated operations through a single point let one actor's activity stall or slow others; cross-user and cross-vault contention should be minimized so one participant cannot throttle the whole system.
- Permanent lockout — states with no recovery path — squatted deterministic identities, consumed one-time slots, overflowed limits — can permanently disable a capability. The threat is any irreversible denial with no administrative remedy.
- Reliance on external liveness — safety depends on relayers, keepers, liquidators, and oracle publishers acting in time. If these are absent, censored, or under-incentivized, price updates, funding, liquidations, and exits stall regardless of on-chain correctness.

Governance, upgrades, and authority

- Least privilege and authority scope — each role and capability should grant only its intended powers. Over-broad authority — unbounded parameters, cross-domain reach, or the ability to touch an unrelated state — widens the blast radius of any misuse or error.
- Key-compromise blast radius — the model must bound what a single compromised privileged key can achieve. Even trusted roles warrant protocol-level guardrails (bounded parameters, staged changes) so that one key cannot instantly disable a safety invariant, redirect critical inputs, or move funds.
- Upgrade correctness and code retirement — upgrades must be able to actually retire superseded logic. If old code cannot be disabled, published fixes and newly added constraints can be bypassed by invoking prior versions on the same live state.



- Irreversibility and recovery — privileged and revocable actions that cannot be undone are a hazard. The protocol should avoid permanent, unrecoverable effects, and revoking a role must meaningfully limit the harm that role can still cause.
- Separation of duties and domain isolation — provisioning, maintenance, and administration should be distinct responsibilities, and trust domains (vendors, packages, vaults) should remain isolated, so a lower-privilege or temporary role cannot perform — or pre-stage — higher-privilege effects, and one domain cannot influence another.
- Configuration safety — high-impact parameter and identity changes should be bounded and, where warranted, staged or delayed so they are observable before taking effect and cannot instantaneously neutralize a safeguard. Concrete threats include an immediate collateral-haircut increase liquidating previously healthy positions, a spread reduction leaving an unsafe old mark in memory, and the first vendor selecting a package-wide collateral feed used by unrelated vaults.
- Emergency-control soundness — pause, freeze, and kill-switch mechanisms must exist, be correctly scoped to the state they protect, and be neither abusable for griefing nor bypassable when genuinely needed.
- Off-chain signal integrity — events and other outward-facing signals must faithfully reflect committed on-chain state, so that monitoring, automation, and governance decisions cannot be spoofed by activity that did not actually change state.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	3	3	1
High	2	2	2
Medium	33	33	20
Low	28	28	19
Informational	6	6	4
Total	72	72	46

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
C-01	Unbounded atomic expired-order cleanup lets attackers permanently gas-lock an order book	Critical	Fixed
C-02	Trading PnL collateral-price drift can make position claims exceed the real vault balance	Critical	Acknowledged
C-03	Routine funding can break collateral-token conservation in inverse markets and make the ClearingHouse vault insolvent	Critical	Acknowledged
H-01	A semi-trusted vendor's per-market collateral-oracle update locks funds across an entire market-making vault	High	Fixed
H-02	Signed Pyth Lazer payloads can make a carried-forward price look fresh	High	Fixed
M-01	Downcast from u256 to u64 could make positions unliquidatable	Medium	Fixed
M-02	Settlement price can be changed after settlement closes have begun, breaking value conservation	Medium	Fixed
M-03	Default bad-debt thresholds make any insurance-fund-exceeding bad debt revert	Medium	Acknowledged



	public liquidation, leaving recovery only to permissioned ADL		
M-04	Bad debt socialization can cause vault shortfall in certain cases	Medium	Fixed
M-05	ADL can leave a flat position with negative collateral that settlement cannot resolve	Medium	Acknowledged
M-06	Missing hard cap on branch_max/leaf_max enables permanent orderbook DOS	Medium	Fixed
M-07	A semi-trusted vendor's first market registration initializes the package-global collateral oracle used by every vault for that collateral	Medium	Acknowledged
M-08	Vault controllers can rewrite existing LPs' fee and withdrawal terms	Medium	Acknowledged
M-09	u64 cancellation totals can make a large losing position impossible to liquidate	Medium	Fixed
M-10	Small force-withdraw requests can cancel all vault orders across markets	Medium	Acknowledged
M-11	Force withdraw applies the collateral haircut before combining collateral and funding	Medium	Fixed
M-12	Freezing a lagging-version registry or clearing house can make it unfreezable	Medium	Fixed

M-13	Liquidation bad-debt socialization can under-recover or over-charge collateral in inverse markets	Medium	Acknowledged
M-14	Maker Solvency Rejections Can Gas-Lock the Order Book	Medium	Fixed
M-15	Reduce-Only Order Churn Can Repeatedly Block Liquidation	Medium	Fixed
M-16	Rolling blend masks a stale leg by returning the maximum leg timestamp, letting stale prices bypass the aggregator's staleness guard	Medium	Fixed
M-17	Pyth integration completely ignores the price confidence interval, exposing downstream protocols to highly uncertain prices	Medium	Acknowledged
M-18	Rolling-feed maintenance operations lack protocol-level parameter and state-machine validation	Medium	Fixed
M-19	Immediate collateral-haircut increases can manufacture liquidation eligibility for solvent positions	Medium	Acknowledged
M-20	Resting Perpetuals orders remain executable during a force-withdraw pause and can censor exits	Medium	Fixed
M-21	Permissionless CoinRegistry::migrate_legacy_metadata can be front-run to permanently block vault creation	Medium	Fixed

M-22	Liquidation sizing subtracts an insurance-fund fee that settlement waives, over-liquidating solvent (negative-raw-collateral / positive-equity) positions	Medium	Fixed
M-23	Tightening the book-spread limit leaves the liquidation mark at the obsolete wider bound, enabling permissionless liquidation of healthy positions	Medium	Acknowledged
M-24	Dust-sized base oracle prices let attackers realize inflated short PnL and drain vault collateral	Medium	Acknowledged
M-25	Vault reads the collateral-oracle price with no strictly-positive guard, so a zero collateral price freezes every LP entry and exit	Medium	Fixed
M-26	Funding/TWAP parameter changes are applied retroactively to un-accrued funding and to the liquidation mark	Medium	Fixed
M-27	Completing any force-withdraw clears the vault's force-withdraw pause unconditionally, letting the owner repeatedly censor the trustless exit	Medium	Fixed
M-28	A fill resets the consecutive zero-fill cap, leaving total limit-order traversal dependent on taker size	Medium	Fixed
M-29	Changing the funding frequency before the old boundary can charge intervals that exist only on the new grid	Medium	Acknowledged

M-30	Passive maker fills bypass post-fill margin and direction controls	Medium	Acknowledged
M-31	Full-risk trading during closed equity sessions can externalize reopening-gap losses	Medium	Acknowledged
M-32	Deposits price a closed-market position with a live mark and can dilute existing LPs	Medium	Fixed
M-33	A vault owner can capture part of a deposit through LP share rounding	Medium	Fixed
L-01	Insurance-fund surplus reserve uses live long-only open interest, drainable during settlement	Low	Acknowledged
L-02	Collateral oracle price has no spot-vs-TWAP divergence guard, unlike the base oracle	Low	Fixed
L-03	Public settlement lets early closers escape bad-debt socialization applied later, stranding the last closers	Low	Acknowledged
L-04	Stale dynamic-field state after AuthorityCap destruction	Low	Acknowledged
L-05	Vault deposits abort on any single negative-margin market position, halting all new LP capital	Low	Acknowledged



L-06	A matured margin-ratio proposal can become simultaneously un-committable, un-deletable, and block creating new proposal	Low	Fixed
L-07	receive_from_account lets a non-ADMIN assistant claim received objects and skips the package-version gate	Low	Fixed
L-08	Final funding after market close can be diluted by a zero-premium settlement update	Low	Fixed
L-09	Cross-Vendor Rolling Feed Object Can Be Bound During Provisioning	Low	Acknowledged
L-10	twap_period_ms accepts unbounded values, letting a trusted user neutralize or freeze the TWAP safeguard	Low	Acknowledged
L-11	Transferable loss basis can be joined into profitable LP positions to eliminate performance fees	Low	Acknowledged
L-12	Missing upgrade_version for Config prevents deprecating old package bytecode	Low	Fixed
L-13	Limit orders revert on partial fills if the remaining posted size is below min_order_usd_value	Low	Fixed
L-14	VendorMetadata setters omit version-gating, allowing users to bypass package upgrades	Low	Fixed

L-15	Emergency pause and freeze states prevent revocation of compromised vault capabilities	Low	Fixed
L-16	Curator metadata fields use Option::fill and become permanently write-once	Low	Fixed
L-17	OwnerWithdraw events report the pre-fee amount instead of the user's actual proceeds	Low	Fixed
L-18	Vault metadata setters are not gated on the vault's version or pause state, allowing mutation of a frozen vault	Low	Fixed
L-19	10u64.pow arithmetic panics and permanently DOS the price feed for tokens with exponents ≥ 20 or ≤ -20	Low	Fixed
L-20	Integration upgrades cannot disable vulnerable entry points from prior package versions	Low	Fixed
L-21	FeedInfoObject constructors return an owned object while permanently claiming a deterministic identity, letting a revocable assistant brick a feed	Low	Fixed
L-22	update_price_feed unconditionally emits UpdatedPriceFeed events for stale prices, poisoning off-chain indexers	Low	Fixed
L-23	A revocable package assistant can permanently capture or exhaust source registrations	Low	Fixed

L-24	A paused market still blocks permissionless vault force-withdrawals	Low	Acknowledged
L-25	Dust-path force-withdraw overcharges the fractional withdrawer by making them absorb the full whole-position close cost	Low	Acknowledged
L-26	Unbounded max_one_execution_amount_bps causes a u64 multiplication overflow that permanently bricks execution of a committed TWAP order	Low	Fixed
L-27	TWAP per-slice size is not aligned to the market lot size, so execution reverts and silently bricks the order	Low	Fixed
L-28	Accepted fee settings can make rounded maker rebates larger than taker fees and abort a trading session	Low	Fixed
I-01	Price update from oracle with timestamp far in the future means this value can't be updated	Informational	Acknowledged
I-02	Validating against empty RESTRICTED_KEYS	Informational	Fixed
I-03	Dead public(package) functions across multiple packages	Informational	Fixed
I-04	Redundant positivity checks in set_config parameter validation	Informational	Fixed



I-05	Mark price can be manipulated by up to max_book_index_spread by trader	Informational	Acknowledged
I-06	Oracle freeze documentation does not match downstream behavior	Informational	Fixed

Critical Severity Issues

C-01: Unbounded atomic expired-order cleanup lets attackers permanently gas-lock an order book

Severity: Critical	Impact: High	Likelihood: High
Files: packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/orderbook.move	Status: Fixed	

Description:

Order matching walks every eligible maker leaf on the crossed side and only frees the consumed prefix with a single `map::batch_drop` at the very end of the traversal, both for market `execute_market_order<T>` and limit `execute_limit_order<T>` fills:

```
// File: perpetuals/sources/clearing_house.move
// fun execute_limit_order<T> and function fun execute_market_order<T>
// Drop all the orders that have been filled
if (last_order_id_matched > 0) {
    map::batch_drop(map, last_order_id_matched, include_last_order_id);
}
```

The loop bound is the taker's remaining size. The problem is that an expired maker consumes none of it. In `process_fill_maker`, an expired maker has its allowed size forced to zero, the order is settled/cancelled, the pending counter is decremented, an event is emitted, but the taker match is zero:

```
// File: perpetuals/sources/clearing_house.move
if (timestamp_ms >= option::destroy_with_default(maker_expiration_timestamp_ms,
0xFFFFFFFFFFFFFFFF)) {
    allowed_maker_size = 0;
    pending_cancellation_reason =
option::some(events::cancellation_reason_expired!());
}
```

With `allowed_maker_size == 0` the matched branch returns `(0, 0, 0, ...)` and `taker_size_matched` is `allowed_maker_size = 0` ([clearing_house.move](#)). So each expired leaf advances the cursor by one order but leaves the taker's size unchanged: neither taker size nor expiry bounds the scan — only the number of resting orders does.

Two facts make this weaponizable. First, the per-order margin/value gate uses the cached mark price, not the resting price, so a poison order priced at the minimum tick stays tiny and cheap to post.

Second, account creation is permissionless ([create_account](#)) while cancellation is ownership-gated (`cancel_orders` asserts the caller's authority cap; the book layer asserts `account_id == order.account_id`). Honest keepers therefore cannot purge another account's stale orders.

Third, no minimum expiration is enforced, so poison orders can be born already-expired with zero waiting. Posting only requires `expiration_timestamp_ms >= now`, and matching treats `now >= expiration` as expired.

Setting the expiration equal to the current clock timestamp therefore passes the post-time check yet leaves the order expired for any fill at the same-or-later timestamp; since the Sui Clock is monotonic, the order is poisoned the instant it rests. There is no accumulation window and no block/checkpoint delay — the attacker posts N born-expired orders and the book is poisoned in the same instant.

Attack: empty one side of a market, then from many Sybil accounts post a large number of min-value asks all at the minimum tick price with the expiration set to the current timestamp so each order is born already-expired. Because there is no positive tick-aligned price below the minimum tick, every honest bid that wants to trade must cross the entire poison prefix; the orders all share one price (no pagination by limit price) and each consumes zero taker size, so the taker's size never reaches zero and the loop must visit all of them plus a final `batch_drop` that re-traverses the removed prefix. Once the order count exceeds the per-transaction compute budget, every crossing transaction runs out of gas mid-scan; Sui's atomic abort rolls back the partial cleanup and restores the identical poison prefix, so the next attempt fails the same way. The book is frozen market-wide for any taker: limit and market orders, stop orders, TWAP slices, and vault force-withdrawals all route through the same matching path and all abort.

Here the defect is that expired makers are cleaned up atomically inside an unbounded, taker-triggered scan that can be pushed past the gas limit and can never make forward progress.

The attack transfers no funds and seizes nothing permanently, but beyond the liveness DoS it produces a temporary lock of committed capital and removes risk-management for as long as the poison persists (which, absent a permissionless sweep and given cheap re-poisoning, is effectively until a fix). The two matching loops are reached only from `execute_limit_order` and `execute_market_order`, so the impact follows exactly what crosses the book:

- Position exit is blocked. Closing or reducing a position is a taker order that crosses the book; on a poisoned side it aborts, so margin committed to open positions cannot be freed. Poisoning both sides blocks opening and closing entirely.
- Stop-loss and TWAP orders do not execute (they route through `place_market/limit_order`), so users lose automated risk management and can take market losses they intended to cap.
- Vault (MMV) LP exit is locked. `process_clearing_house_for_force_withdraw` unwinds vault positions via `place_market_order`, which crosses the Clearhouse book; the LPs' emergency exit is precisely what the poison DoSes, so LP capital tied to open vault positions is stranded.

Two facts bound the impact and keep it from being a total or permanent fund freeze. Liquidations are unaffected: `liquidate` does not cross the book — it settles at mark via `execute_liquidation` → `settle_liquidated_position` — so undercollateralized positions are still closed out and no bad debt accrues from stalled liquidations. Free/idle collateral remains withdrawable: `deallocate_free_collateral`, `deallocate_collateral`, and account `withdraw_collateral` do not touch the order book. So the lock is confined to margin backing open positions and to vault-LP capital in open positions, not to idle balances, and a stuck trader whose position goes underwater is liquidated out rather than trapped.

Consider following more detailed descrip:

1. Attacker calls `create_account` N times (permissionless) and funds each minimally.
2. From each account, post min-value asks at the minimum tick price with `expiration_timestamp_ms` set to the current timestamp so they are born already-expired; value passes because `assert_order_value` uses `mark_price`, not the tick price.
3. No waiting is needed (the orders expire the instant they rest); only keep the count above the compute budget.

4. Any user submits a buy that crosses: matching visits every expired leaf (each returns `taker_size_matched == 0`), never fills, runs out of gas, and aborts — restoring the prefix. The market is now permanently unmatched.

An executable PoC `poc_expired_order_scan_dos.move` confirms the mechanism: the poison asks are posted with `expiration = now` (no clock advance in the test), and a single crossing taker processes every born-expired maker for zero fill. Instrumenting the matching loop prints exactly N iterations per taker transaction ($N=5 \rightarrow 5$, $N=40 \rightarrow 40$).

Measuring with `sui move test -s` across $N = 5/20/40/80$ with identical setup shows the per-transaction gas growing linearly:

Expired makers N	marginal gas (Move VM units)	per added maker
5 $\rightarrow$ 20	+27,918	~1,860
20 $\rightarrow$ 40	+57,202	~2,860
40 $\rightarrow$ 80	+114,576	~2,860

Each resting expired maker adds ~2,860 gas units to every taker that must cross it (settle funding + emit event + borrow the maker position), independent of fill. The linear growth is the measured fact; the absolute out-of-gas count depends on the taker's gas budget. Using Sui's maximum per-transaction budget (50 SUI) as the ceiling, ~2,860 units/maker puts the out-of-gas point on the order of $N \approx 10^4$ (roughly 15,000–20,000) expired makers for a taker willing to pay the maximum; ordinary takers set far smaller budgets and are bricked at N in the low hundreds.

Capital is affordable for the attackers. Each poison order only has to clear the market's `min_order_usd_value` (default \$0.50 — the floor of the registry's [\$0.50, \$1000] range), so at the 10% initial-margin ratio each order locks ~\$0.05 of recoverable margin, and `max_pending_orders` is 100 per account. Freezing normal trading ($N \approx 200$) costs ~\$10 of recoverable margin across 2 accounts; sizing the attack to defeat even a max-budget clearer ($N \approx 17,000$) costs ~\$850 of recoverable margin across ~170 Sybil accounts, plus a modest amount of one-time placement gas. Because the margin is recoverable and the per-order floor is at the registry minimum, the

economic barrier is minimal unless the market sets `min_order_usd_value` toward the high end of its range.

Recommendations:

Add a permissionless, explicitly-bounded expired-order purge that commits independently of any taker order — a cursor/limit sweep with monotonic forward progress that does not abort when it hits its limit, so keepers (or the crossing taker itself) can retire stale orders in committed batches. Additionally enforce a hard cap on the number of maker entries scanned per matching transaction such that reaching the cap does not roll back the cleanup already performed.

Customer Response:

Fixed in commit [7007b4c](#) reject born-expired posts orders and add a MAINTENANCE-gated stale-order `try_cancel_stale_limit_order` sweeper and in commit [67c846f](#) add `max_zero_fill_matches = 200` in `execute_limit_order`

Fix Review:

Fix confirmed.

C-02: Trading PnL collateral-price drift can make position claims exceed the real vault balance

Severity: Critical	Impact: High	Likelihood: High
Files: packages/position/sources/position.move packages/perpetuals/sources/clearing_house.move	Status: Acknowledged	

Description:

Trading PnL is economically zero-sum in USD, but the protocol realizes each position's PnL by converting the signed USD delta into collateral-token units using the `collateral_price` of the current session.

This breaks token conservation when equal and opposite USD PnL is realized at different collateral prices. A profitable trader can receive $+X / p_1$ collateral tokens, while the losing counterparty later pays only $-X / p_2$ collateral tokens. If $p_1 \neq p_2$, the token deltas do not cancel.

When the drift is positive, the sum of position collateral claims can become larger than the actual collateral tokens held by the clearing-house vault. Early users can withdraw real tokens, while later users may revert with `insufficient_vault_collateral`. This is a protocol insolvency condition.

Root Cause

`position::add_to_collateral_usd` mutates `position.collateral` by converting a signed USD amount into collateral-token units at the caller-provided `collateral_price`.

[File: packages/position/sources/position.move:157-201](#)

```
public fun add_to_collateral_usd(
  position: &mut Position,
  fixed_usd: u256,
  collateral_price: u256
): u256 {
  if (macros::is_neg!(fixed_usd)) {
    let delta_mag = macros::div_ceil_positive!(
      macros::negate_nonzero!(fixed_usd),
      collateral_price,
```

```
    );  
    // subtract collateral-token units from position.collateral  
  } else {  
    let delta_mag = macros::div_floor_positive!(fixed_usd, collateral_price);  
    // add collateral-token units to position.collateral  
  };  
  
  position.collateral = collateral_after;  
}
```

The helper only changes the position's accounting claim. It does not transfer real collateral tokens between the winning and losing traders, and it does not enforce that all opposite PnL deltas from the same economic lifecycle use one fixed collateral price.

In one matching session, maker and taker use the same `collateral_price` stored in the `SessionHotPotato`. That case alone does not create material drift.

The problem appears because PnL is realized only when a fill reduces or reverses an existing position. If a trader is opening or increasing a position, realized PnL is zero.

Examples from `add_base_to_position`:

[File: packages/position/sources/position.move:276-299](#)

- Selling while long reduces or reverses the long and realizes PnL.
- Buying while short reduces or reverses the short and realizes PnL.
- Selling while already short only increases/opens a short and returns `pn1 = 0`.
- Buying while already long only increases/opens a long and returns `pn1 = 0`.

Therefore, a normal trade can realize PnL for one side while the other side merely opens a new position and realizes no opposite PnL until a later session.

Realistic Insolvency Scenario

Assume collateral is volatile and its oracle price changes between sessions.

```
Session 1:  
collateral_price = p1 = $1
```

Alice is long from \$100.

Bob is flat.

Alice sells to close at \$200.

Bob buys to open at \$200.

Alice realizes +\$100 PnL.

Bob realizes \$0 PnL because he only opens a new long.

Alice position.collateral increases by:

$+\$100 / \$1 = +100$ collateral tokens

Later:

Session 2:

collateral_price = p2 = \$2

Bob sells to close at \$100.

Bob realizes -\$100 PnL.

Bob position.collateral decreases by:

$-\$100 / \$2 = -50$ collateral tokens

The USD PnL over the lifecycle is zero-sum:

Alice: +\$100

Bob: -\$100

Net: \$0

But the collateral-token accounting is not zero-sum:

Alice credited: +100 tokens

Bob debited: -50 tokens

Net drift: +50 tokens

The protocol has created an extra 50-token claim without receiving 50 real tokens. Repeating this pattern can make total position claims exceed the market vault's actual collateral balance.

General formula:

```
Positive trader credit = +X / p1
Negative trader debit  = -X / p2
Net token drift        = X * (1 / p1 - 1 / p2)
```

If $p1 < p2$, aggregate collateral claims increase. If $p1 > p2$, collateral can be stranded in the vault instead. Both directions prove that token accounting is not conserved.

Code Path

Taker settlement realizes PnL and converts it once using the session collateral price:

[File: packages/position/sources/position.move:444-448](#)

```
let collateral_change = ifixed::sub(
  pnl_taker,
  ifixed::add(taker_fees, integrator_fees),
);
position.add_to_collateral_usd(collateral_change, collateral_price);
```

Maker settlement uses the same conversion pattern:

[File: packages/position/sources/position.move:743-751](#)

```
let (pnl, base_after) = position.add_base_to_position(
  order_is_ask,
  base_asset_delta,
  quote_asset_delta
);
position.add_to_collateral_usd(
  ifixed::sub(pnl, fees),
  collateral_price
);
```

The session reads `collateral_price` from the oracle at session start and stores it in the hot potato:

[File: packages/perpetuals/sources/clearing_house.move:4712-4783](#)

```
let collateral_price = market_params.collateral_oracle_price(
  collateral_oracle,
  clock
);
```

```
let hot_potato = SessionHotPotato<T> {  
    collateral_price,  
    ...  
};
```

Withdraw/deallocation eventually tries to pay the accounting claim from the real vault balance:

[File: packages/perpetuals/sources/clearing_house.move:3746-3755](#)

```
position.sub_from_collateral(  
    ifixed::from_balance(amount, scaling_factor)  
);  
  
withdraw_vault_collateral(  
    borrow_mut_market_vault(clearing_house),  
    account_balance,  
    amount  
);
```

The transfer aborts if the vault does not actually hold enough collateral tokens:

[File: packages/perpetuals/sources/clearing_house.move:3761-3767](#)

```
assert!(  
    balance::value(&vault.collateral_balance) >= amount,  
    insufficient_vault_collateral!()  
);
```

This is the insolvency sink: accounting says a user has withdrawable collateral, but the real vault balance may be insufficient.

Impact

This can cause:

- Protocol insolvency in collateral-token units.
- Later withdrawals/deallocations reverting with `insufficient_vault_collateral`.
- Loss socialization to users who withdraw after positive drift has accumulated.
- Incorrect collateral accounting across normal trading, not only liquidation or bad-debt flows.

- Potential stranded collateral in the opposite price direction.

The issue is most severe when the market supports volatile collateral whose oracle price can materially change between trading sessions. If only stable collateral is supported and the collateral price remains close to \$1, the practical magnitude is smaller, but the accounting invariant is still broken.

Why Existing Matching Does Not Prevent This

The protocol does use one `collateral_price` for all maker/taker settlement inside a single session. That prevents same-session price mismatch.

However, it does not solve the lifecycle problem:

1. One side may close and realize PnL now.
2. The opposite side may only open/increase a position now, realizing zero PnL.
3. That opposite side realizes the offsetting PnL later, in a different session, at a different `collateral_price`.

So the bug does not require the maker and taker of the same match to use different prices. It only requires opposite USD PnL to be realized at different times, which is normal in a perpetual market.

Recommendations:

The core invariant should be:

For any zero-sum trading PnL lifecycle, collateral-token claims must also be conserved.

Possible fixes:

1. Do not settle trading PnL by independently converting each realized USD delta at the current collateral oracle price.
2. Track realized trading PnL in a USD-denominated liability/accounting bucket and convert only when collateral tokens are actually moved in a conservation-preserving way.
3. Alternatively, store the collateral-token basis for open positions so the later opposite realization uses a matched conversion basis instead of the later oracle price.

4. Add an invariant test that simulates:

- trader A closes for $+X$ USD at $p1$;
- trader B opens in that fill;
- trader B later closes for $-X$ USD at $p2$;
- final total position collateral claims plus vault balance remain conserved.

Any fix should explicitly test $p1 \neq p2$ with volatile collateral. Same-session maker/taker tests are not sufficient because the bug appears across sessions.

Customer Response:

Acknowledged. The project team acknowledged and decided not to use any different collateral other than USDC (stablecoins).

C-03: Routine funding can break collateral-token conservation in inverse markets and make the ClearingHouse vault insolvent

Severity: Critical	Impact: High	Likelihood: High
Files: packages/perpetuals/sources/market.move packages/position/sources/position.move packages/perpetuals/sources/clearing_house.move	Status: Acknowledged	

Description:

Aftermath supports inverse markets. In an inverse market, the contract is denominated in USD, but settlement happens in a volatile collateral asset such as BTC or ETH.

Official docs describe this model here: [Inverse Markets](#).

This makes funding settlement dangerous. Funding is economically zero-sum in USD, but the code settles each position's funding into collateral tokens using the collateral price at that position's own settlement time. If the receiving side settles when collateral is cheap and the paying side settles when collateral is expensive, the receiver is credited more tokens than the payer loses.

The difference is not backed by real tokens in the ClearingHouse market vault. Over time, the sum of position collateral claims can become larger than the vault's real collateral balance. Later withdrawals or deallocations can then revert with `insufficient_vault_collateral`.

This is a protocol insolvency issue for inverse markets.

Root Cause

Funding is accrued as a USD value, not as a collateral-token value.

[File: packages/perpetuals/sources/market.move:1029-1059](#)

```
let funding_rate = ifixed::mul(state.premium_twap, period_adjustment);

state.cum_funding_rate_long = ifixed::add(
    state.cum_funding_rate_long, funding_rate
```

```
);  
state.cum_funding_rate_short = ifixed::add(  
    state.cum_funding_rate_short, funding_rate  
);
```

Later, each position settles that USD funding into collateral tokens using the current `collateral_price`.

[File: packages/position/sources/position.move:930-1036](#)

```
let numerator = abs_rate_delta * abs_base;  
  
// User receives funding.  
let collateral_magnitude = numerator / collateral_price;  
position.collateral = collateral_after;  
  
// User pays funding.  
let collateral_magnitude = (numerator + collateral_price - 1) / collateral_price;  
position.collateral = collateral_after;
```

`position.collateral` is denominated in collateral tokens. Funding settlement only changes this accounting field. It does not move real collateral tokens between the payer and receiver.

Therefore, for the vault to remain solvent, the token amount credited to receivers must match the token amount debited from payers. This is not guaranteed when they settle at different collateral prices.

Why Inverse Markets Make This Critical

For linear USDC markets, `collateral_price` is close to \$1, so the drift is usually small.

For inverse markets, collateral is volatile. The docs explicitly support markets such as:

- `BTC/USD` collateralized by `BTC`
- `SUI/USD` collateralized by `BTC`
- `SUI/USD` collateralized by `ETH`

In these markets, `collateral_price` can move materially between two funding settlements. Because the contract is USD-denominated but settled in the collateral asset, the same USD funding amount can become different token amounts at different times.

Example

Assume one funding period transfers \$1,000 from longs to shorts.

The short is the receiver and settles when BTC is \$40,000:

```
Receiver credit = $1,000 / $40,000 = 0.025 BTC
```

The long is the payer and settles later when BTC is \$50,000:

```
Payer debit = $1,000 / $50,000 = 0.020 BTC
```

The funding is zero-sum in USD:

```
+$1,000 - $1,000 = $0
```

But it is not zero-sum in collateral tokens:

```
+0.025 BTC - 0.020 BTC = +0.005 BTC
```

The system has created an extra 0.005 BTC accounting claim without receiving 0.005 BTC of real collateral.

General formula:

```
token drift = X * (1 / p_receiver - 1 / p_payer)
```

where X is the USD funding amount.

If $p_{\text{receiver}} < p_{\text{payer}}$, total position claims increase. Repeating this over many funding periods can make the vault insolvent.

Live Market Reachability

This does not require the market to be closed.

`settle_position_funding` is a public live-market function. It accepts an arbitrary `account_id`, reads the current collateral oracle price, and settles funding for that position.

[File: packages/perpetuals/sources/clearing_house.move:1736-1768](#)

```
public fun settle_position_funding<T>(  
    clearing_house: &mut ClearingHouse<T>,  
    oracle: &PriceFeedStorage,
```

```
    account_id: u64,  
    clock: &Clock,  
  ) {  
    let collateral_price = clearing_house  
      .market_params  
      .collateral_oracle_price(oracle, clock);  
  
    clearing_house  
      .borrow_mut_position(account_id)  
      .settle_position_funding_and_emit(...);  
  }
```

Funding is also settled during normal user actions, including session start and deallocation. For example, deallocation settles funding first, then computes withdrawable free collateral.

[File: packages/perpetuals/sources/clearing_house.move:4657-4685](#)

This means the bug is reachable during normal live-market operation. It is not only a close-market issue.

Insolvency Sink

When a user deallocates or withdraws free collateral, the protocol computes the user's claim from `position.collateral`, then tries to pay real tokens from the ClearingHouse market vault.

[File: packages/perpetuals/sources/clearing_house.move:3680-3756](#)

```
let free_collateral_fixed = position.compute_free_collateral(...);  
  
position.sub_from_collateral(  
  ifixed::from_balance(amount, scaling_factor)  
);  
  
withdraw_vault_collateral(  
  borrow_mut_market_vault(clearing_house),  
  account_balance,  
  amount  
);
```

The real token transfer fails if the vault balance is lower than the accounting claim.

[File: packages/perpetuals/sources/clearing_house.move:3761-3769](#)

```
assert!(  
    balance::value(&vault.collateral_balance) >= amount,  
    insufficient_vault_collateral!  
);
```

This is where insolvency appears: accounting says the user can withdraw, but the vault does not have enough real collateral tokens.

Why Closed-Market Settlement Does Not Save Live Markets

The closed-market settlement path uses a frozen `settlement_prices.collateral_price`, so all positions close against the same collateral price.

[File: packages/perpetuals/sources/clearing_house.move:2220-2298](#)

That path is safer because it avoids per-position live price differences.

The bug is in the live funding path, where positions can settle funding at different times and therefore at different collateral prices.

Impact

This can cause:

- ClearingHouse market vault insolvency in collateral-token units.
- Valid deallocation or withdrawal attempts reverting with `insufficient_vault_collateral`.
- Early withdrawers receiving real tokens while later users are left with unpaid accounting claims.
- Socialized loss across remaining traders or LP/vault strategies using the affected market.
- Repeated accumulation over routine funding periods, because this is not limited to liquidation or bad debt.

For inverse markets, this is a Critical issue because the protocol explicitly supports volatile collateral settlement.

Recommendations:

Funding settlement must preserve collateral-token conservation.

Recommended options:

Convert funding into collateral-token units once per funding epoch using one market-wide collateral price, then settle all positions against token-denominated cumulative funding indices.

Add an invariant test for inverse markets:

1. Create equal long/short positions.
2. Accrue a nonzero funding amount X .
3. Settle the receiver at collateral price p_1 .
4. Settle the payer at collateral price p_2 where $p_1 \neq p_2$.
5. Assert that total position collateral claims do not exceed the real vault collateral balance.

The test should use volatile collateral prices, not USDC-only prices.

Customer Response:

Acknowledged. The project acknowledged and decided not to use any different collateral other than USDC (stablecoins).

High Severity Issues

H-01: A semi-trusted vendor's per-market collateral-oracle update locks funds across an entire market-making vault

Severity: High	Impact: High	Likelihood: Medium
Files: packages/market-making-vault/sources/vault.move packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/market.move packages/perpetuals/sources/registry.move	Status: Fixed	

Description:

Collateral type **T** has two places that store its oracle: (1) per-market — each clearing house's `core_params / MarketInfo<T>`; (2) global — registry `CollateralInfo<T>`, which vaults cache and trust. A vendor can change only the per-market copy via `set_collateral_oracle_params`. The global record is unchanged by design (only the first `register_market` or package Admin/Assistant `set_collateral_info` writes it). That split is intentional — vendors must not rewrite a shared collateral record. The bug is on the vault side: Deposit/withdraw use the global oracle cache, but force-withdraw also requires the clearing house's oracle. After a vendor updates only the market's copy, those two no longer match — so force-withdraw cannot pick one oracle object that satisfies both checks, and LP funds get stuck.

The vault validates and prices collateral against a single global-derived cache, not against each clearing house's own oracle. It reads `(collateral_storage_id, collateral_source_id)` once from the global `CollateralInfo<T>` and caches them in `vault_params` (refreshed by `set_collateral_pfs_info`, — which itself only ever re-reads the global record). At the start of every deposit / withdraw / force-withdraw session the vault asserts the caller's collateral oracle against that single cached value and snapshots the collateral price from it. The difference is what happens during per-market processing:

- Deposit (`process_clearing_house_for_deposit`) and normal withdrawal (`process_clearing_house_for_withdraw`) pass the clearing house **only the base oracle**,

call only `update_funding / mark_price` on it, and value the collateral leg entirely off the session-cached collateral price. They never hand a collateral oracle to the clearing house and never reach `market::collateral_oracle_price`.

- Force-withdraw (`process_clearing_house_for_force_withdraw`) is the only path that passes the collateral oracle on to the clearing house — via `ch::deallocate_collateral(..., base_oracle, collateral_oracle, ...)` — which re-asserts it against the CH's own `core_params.collateral_storage_id` (`market::collateral_oracle_price`).

Impact — fund lock on the trustless force-withdraw exit (storage-id change). When a vendor changes their market's collateral oracle `storage_id` from `S_old` to `S_new`, force-withdraw must satisfy two checks with one `PriceFeedStorage` object. No single object has both storage ids.

Step 0 — vendor update moves CH to `S_new`, vault cache stays `S_old`:

```
// clearing_house.move
// Vendor setter updates CH market params + per-market MarketInfo only (not
// global CollateralInfo).
clearing_house.borrow_mut_market_params().set_collateral_oracle_params(...,
storage_id, ...);
registry.set_collateral_oracle_params<T>(ch_id, storage_id, source_id);
// After this: core_params.collateral_storage_id == S_new
// But vault.vault_params.collateral_storage_id still == S_old (from global
// CollateralInfo cache)
```

Step 1 — session start freezes price from vault cache (`S_old`):

```
// vault.move (start_force_withdraw_session)
// Vault requires the caller to pass the oracle object whose storage_id matches
// the CACHED value.
assert_collateral_price_feed_storage_is_correct(collateral_oracle,
vault.vault_params.collateral_storage_id);
let collateral_price = get_price(
    collateral_oracle, clock,
    vault.vault_params.collateral_source_id, // price read from S_old feed
    vault.vault_params.collateral_pfs_tolerance,
```

```
);  
// collateral_price is stored in WithdrawSession and reused later
```

Step 2 — per-market force-withdraw processing re-checks vault cache (S_{old}) and price equality:

```
// vault.move (process_clearing_house_for_force_withdraw)  
// Same collateral_oracle object must still match the vault cache ( $S_{old}$ ).  
assert_collateral_price_feed_storage_is_correct(  
    collateral_oracle,  
    withdraw_session.vault.vault_params.collateral_storage_id, // expects  $S_{old}$   
);  
let current_collateral_price = get_price(  
    collateral_oracle, clock,  
    withdraw_session.vault.vault_params.collateral_source_id,  
    withdraw_session.vault.vault_params.collateral_pfs_tolerance,  
);  
// Price must match the snapshot taken at session start (from  $S_{old}$  feed).  
assert!(  
    current_collateral_price == withdraw_session.collateral_price,  
    errors::bad_oracle_price(),  
);
```

Step 3 — same transaction passes that oracle into CH deallocation, which asserts market params (S_{new}):

```
// vault.move  
// Vault forwards the SAME collateral_oracle object into clearing house  
deallocation.  
ch::deallocate_collateral(  
    &mut clearing_house,  
    withdraw_session.vault.account_cap(),  
    &mut withdraw_session.account,  
    base_oracle,  
    collateral_oracle, // still the object caller passed (must be  $S_{old}$  to pass  
Step 2)  
    amount_to_deallocate_balance,  
    clock
```



```
);  
  
// clearing_house.move -> market.move (inside deallocate_collateral_internal)  
let collateral_price = market_params.collateral_oracle_price(collateral_oracle,  
clock);  
// collateral_oracle_price asserts:  
assert!(  
    oracle.storage_id() == market_params.core_params.collateral_storage_id, //  
expects S_new  
    invalid_collateral_price_feed_storage!()  
);
```

why both branches abort (one `PriceFeedStorage` object, two incompatible storage-id requirements):

Caller passes oracle with <code>storage_id</code>	Vault check (Steps 1–2)	CH <code>collateral_oracle_price</code> (Step 3)
<code>S_old</code>	PASS (matches cache + session snapshot)	FAIL (<code>S_old != S_new</code> in <code>core_params</code>)
<code>S_new</code>	FAIL (<code>S_new != vault cache S_old</code> , or price <code>!=</code> session snapshot from <code>S_old</code>)	PASS

- passing `S_old` satisfies the vault snapshot but fails the CH `S_new` assertion in `deallocate_collateral`;
- passing `S_new` fails the vault's storage-id check and/or `current == session` equality (price came from a different feed than the `S_old` snapshot).

Because `end_withdraw_session` asserts the session's `ch_ids` work-list is fully drained, the one aborting market bricks the entire force-withdraw session. Force-withdraw is the keeper/admin-independent exit and the only path that can pull collateral out of the CH positions; a single semi-trusted vendor's routine oracle update therefore freezes the LP collateral allocated as margin across all of the vault's markets from the trustless exit — recoverable only by the project Admin calling `registry::set_collateral_info` (global) then `vault::set_collateral_pfs_info` (vault cache), re-coupling the keeper-independent exit to admin liveness. Deposits and normal withdrawals do not abort (they never invoke the CH collateral oracle) — see the mispricing impact below.

Impact — silent NAV mispricing on deposit and normal withdrawal (storage-id or source-id change). Because deposit and normal withdraw never consult the clearing house's collateral oracle, a vendor's change of either the `storage_id` or the `source_id` does not abort them; instead the vault keeps pricing the collateral leg off its cached (now stale) feed while the clearing house values margin and deallocation off the changed feed. The vault mints/redeems LP shares at a

collateral price that diverges from the CH's risk engine — over-redeeming (LP dilution) or under-paying LPs. This window is exploitable by any LP with no further privilege.

Under the project's semi-trusted vendor model, the trigger is a single vendor's ordinary configuration action; the impact is a separate LP vault whose trustless force-withdraw exit is frozen across all its markets, plus an exploitable NAV-mispricing window on deposit/withdrawal.

Recommendations:

Do not give the vendor the authority to change the collateral oracle of the clearing house market. Instead, just Admin or Assistant under the project team control can set the collateral oracle.

Customer Response:

Fixed in commit [c262260](#). Now only the project admin and assistant can call `set_collateral_oracle_params`.

Fix Review:

Fix confirmed.

H-02: Signed Pyth Lazer payloads can make a carried-forward price look fresh

Severity: High	Impact: High	Likelihood: Medium
Files: packages/integrations/pyth-lazer/sources/price_feed_storage.move packages/integrations/pyth-lazer-rolling/sources/price_feed_storage.move packages/oracle-aggregator/sources/price_feed.move	Status: Fixed	

Description:

Pyth Lazer has two relevant timestamps. The message timestamp `timestampUs` says when the signed message was created. `feedUpdateTimestamp` says when the price itself was generated. These timestamps are not interchangeable when Pyth carries an older price into a new message.

Pyth lets a subscriber choose the properties in the signed payload. Its payload reference states that only requested fields are included and recommends `feedUpdateTimestamp` so consumers can detect a carried-forward price. The official Sui example requests `price` and `exponent`, omits `feedUpdateTimestamp`, asks for `leEcdsa`, and then sends the returned payload to the on-chain verifier ([fetch-and-verify.ts:50](#)). This confirms that Pyth can sign a payload without the price-observation timestamp. See also the Pyth [subscription guide](#) and [payload reference](#).

[The Pyth document](#) recommends but not forbidding omitting `feedUpdateTimestamp`: "For production applications, include `"feedUpdateTimestamp"` in your properties array. This field lets you determine whether a price was freshly generated or carried forward from an earlier update. Compare it against `timestampUs` to assess price freshness. See Payload Reference — Price Availability Semantics for details.

The Aftermath integration accepts this payload. When `feed_update_timestamp` is missing or has no value, `timestamp_ms` uses the newer message timestamp: [Code location](#)

```
fun timestamp_ms(feed: &Feed, update: &Update): u64 {
    let market_session = feed.market_session();

    if (market_session.is_some()) {
        if (market_session.borrow().is_closed()) return
```

```
update.timestamp().to_ms();
};

let mut feed_update_timestamp = feed.feed_update_timestamp();
if (feed_update_timestamp.is_none()) return update.timestamp().to_ms();

let mut feed_update_timestamp = feed_update_timestamp.extract();
if (feed_update_timestamp.is_none()) return update.timestamp().to_ms();

feed_update_timestamp.extract().to_ms()
}
```

The two missing-timestamp branches are outside the `closed` session block. They therefore also run while the Pyth market session is open, or when `market_session` was not requested as this `market_session` is also optional. The rolling integration has the same fallback for each price-bearing leg ([price_feed_storage.move:407](#)).

The parser supports this input. It starts optional properties as `none`, sets `marketSession` only when property 9 exists ([feed.move:228](#), [feed.move:326](#)), and sets `feedUpdateTimestamp` only when property 12 exists ([feed.move:348](#)). A signed payload can therefore omit either field. In contrast, the integration aborts if `price` or `exponent` is missing, but does not abort when these session and freshness fields are missing ([price_feed_storage.move:426](#)).

Anyone can submit the signed bytes through `update_price_feed`. The function verifies the Pyth signature but requires no authority capability ([price_feed_storage.move:202](#)). The attacker cannot choose an arbitrary price or forge a signature. They only choose a valid subscription schema that omits `feedUpdateTimestamp` and relay the message that Pyth signs.

The dangerous case needs Pyth to carry forward an old price, for example because too few publishers supplied a new price. Pyth documents this behavior in [Understanding Price Data](#). The attacker cannot cause this condition. However, this is the exact condition that an oracle freshness check must detect. Without `feedUpdateTimestamp`, a newly signed message contains an old price but receives a new timestamp in Aftermath.

The core oracle accepts every increasing timestamp and updates the stored price, timestamp, and TWAP ([price_feed.move:267](#)). Repeated messages with the same old price also move the TWAP toward that price. The perpetuals freshness check trusts the stored timestamp

([market.move:1524](#)), so both the freshness check and the spot-versus-TWAP check can pass while the real observation is old.

This stale base-index price affects position PnL, margin, liquidation, and collateral withdrawal. For example, the withdrawal path reads the index spot and TWAP, checks their divergence, and then uses them to calculate free collateral ([clearing_house.move:4852](#)). A stale-high index can hide the real loss of a long position. The account may withdraw collateral that should remain locked. When a fresh price returns, the position can become undercollateralized and leave bad debt for the insurance fund or other traders.

The severity is High. The financial impact can be large because the stale value is used by primary risk checks. Likelihood is lower than the impact because exploitation needs a Pyth carry-forward period, a meaningful real-price move, a market that uses the affected Lazer source, and enough time and liquidity to act before recovery or pause. The attacker cannot create the oracle disruption, so this is not rated Critical.

Recommendations:

For every market configured to use a 24/5 Pyth Lazer feed, require both `marketSession` and `feedUpdateTimestamp` in every signed payload. Abort if `marketSession` is absent, or if `feedUpdateTimestamp` is absent or has no value. The subscriber chooses the payload properties, so the adapter must not infer a session or freshness from an omitted field.

For every explicit non-closed session, use only `feedUpdateTimestamp` as the price timestamp. Reject observations that are too old or too far in the future. Never fall back to the message `timestampUs` for these sessions.

When `marketSession` is explicitly `closed`, route the update through a separate 24/5 closed-session policy. Since Aftermath intentionally permits activity while the market is closed, preserve the real `feedUpdateTimestamp` and do not present the carried-forward price as fresh merely because Pyth signed a new message. The safest policy is to permit cancellation and reduce-only actions until a fresh, explicitly non-closed observation arrives. If full off-hours trading is required, apply explicit risk limits for trading against a carried-forward price.

Apply the same rules to Pyth Lazer Rolling. Require both fields for every price-bearing leg, reject missing or inconsistent session values, and enter the closed-session policy only when the relevant legs explicitly report `closed`.

Add regression tests for an omitted property 9, an omitted property 12, property 12 with no value, a stale timestamp in every supported non-closed session, and inconsistent Rolling-leg sessions. These cases should abort. Also test that an explicit `closed` value reaches only the dedicated 24/5 policy and that a carried-forward price cannot silently advance the stored freshness timestamp or TWAP.

Customer Response:

Fixed in commit [5a59882](#). The change now enforces that `feedUpdateTimestampMs` must be provided outside of when the market session is closed.

When the market session is closed, the code comments acknowledges that the code defaults to `update.timestamp()` , and `feed_update_timestamp` is not required as it is stale during that session.

Fix Review:

Fix confirmed.

Medium Severity Issues

M-01: Downcast from u256 to u64 could make positions unliquidatable

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/position/sources/position.move	Status: Fixed	

Description:

Each market can configure a `max_open_interest` which represents the maximum number of units of the underlying that a position can have open. However, if set too high it would be possible for an attacker to create a delta neutral pair of positions that are unliquidatable. Depending on the leverage allowed by the market this could be done reasonably cheaply. Realistically this will only happen for a token that had a low price in USD. Existing Aftermath v1 markets have a maximum value for `max_open_interest` of 20,000,000 of its tokens. There's nothing preventing it being set higher than this value. If the `max_open_interest` ever exceeds `u64::MAX / 1e9` or ~18.45 billion tokens then an attack is possible where by an attacker:

- creates an order for less than ~18.45B units (and self-fills it from a sybil account)
- creates a second order which will sum to an open interest greater than 18.45B but less than `max_open_interest` (and self fills it)

The attacker now has a delta neutral position and they have become unliquidatable.

The reason they are able to open a position greater than `u64::MAX / 1e9` is because [position.move#L293](#) / [position.move#L279](#) adds together two `ifixed` values. These will not overflow when added. The root cause of the unliquidatability is that, when `liquidate` is called, this line ends up executed: [clearing_house.move#L4415](#)

This will revert if the value being downcast to `u64` is bigger than `u64::MAX`. This effectively makes the position unliquidatable. Even though it is unlikely that a token will be worth so little in USD that a `max_open_interest` exceeding `u64::MAX / 1e9` is necessary, if this scenario were to occur the impact of this bug would be Critical.

Recommendations:

Consider increasing the type used to represent units to `u128` or add a check to make `max_open_interest < u64::MAX / 1e9`.

Customer Response:

Fixed in commit [9a31852](#) by changing liquidation related variables to be of type `u128` and adding `ifixed::[from/to]_u128balance` functions.

Fix Review:

Fix confirmed.

M-02: Settlement price can be changed after settlement closes have begun, breaking value conservation

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move	Status:	

Description:

At settlement every position closes against `SettlementPrices.base_price` rather than against a counterparty — the market vault is the implicit counterparty for all closes. Value conservation requires every position to settle at the same price: with $\sum \text{base} = 0$ and $\sum \text{quote} = 0$ across all positions, settling each at one uniform price P gives $\sum \text{pnl} = P \cdot \sum \text{base} + \sum \text{quote} = 0$, so the vault pays out exactly the collateral it holds and nets to zero.

`set_settlement_prices` lets the vendor/admin overwrite `base_price` / `collateral_price` at any time while the market is closed. Its only state guard is `assert_market_is_closed`:

```
// clearing_house.move
public fun set_settlement_prices<VendorKey, Role, T>(...) {
  ...
  clearing_house.assert_market_is_closed(); // - only state guard
  let settlement_prices = clearing_house.borrow_mut_settlement_prices();
  ...
  settlement_prices.base_price = base_settlement_price;
  settlement_prices.collateral_price = collateral_settlement_price;
  ...
}
```

Nothing records that settlement closes have already begun. `SettlementPrices` ([clearing_house.move:222-233](#)) has no "settlement started" flag, `enabled` can be toggled freely, and the settlement close path re-reads the price live on every call ([clearing_house.move:4705-4707](#)). Because `close_position_at_settlement_prices_as_admin` settles positions while `enabled == false` ([:1136-1140](#)) and the public

`close_position_at_settlement_prices` runs after `enabled == true` (:2166-2170), the admin can settle one group of positions at `P1`, change the price to `P2`, and settle the rest at `P2`.

Settling at two prices breaks conservation. Splitting positions into a group settled at `P1` (net base `X`) and the rest at `P2` (net base `-X`) gives $\sum \text{pnl} = P1 \cdot X + P2 \cdot (-X) = X \cdot (P1 - P2)$, which is nonzero whenever $X \neq 0$ and $P1 \neq P2$. That is the amount by which payouts exceed the vault's collateral. When the deficit hits, the last settlers abort on the vault solvency check and can never withdraw their collateral:

```
// clearing_house.move - deallocate path, no insurance-fund buffer
assert!(
    vault.collateral_balance.value() >= collateral_to_deallocate_balance,
    insufficient_vault_collateral!()
);
```

With `deallocate_collateral = false`, the close does not pull from the vault but the position carries a collateral claim the vault cannot honor, surfacing as the same abort on a later withdrawal — silent insolvency.

This requires the privileged `AuthorityCap<VENDOR<VendorKey>, Role>` that passes `assert_admin_or_authorized_assistant_authority_cap` (:1156), the same trusted role that runs `close_market` and settlement — it is a centralization / missing-invariant issue, not a permissionless attack, hence Medium. Note this is a distinct capability from simply setting a bad price at `close_market`: a single uniform price (however wrong) keeps $\sum \text{pnl} = 0$ and the vault solvent; only re-pricing mid-settlement creates a shortfall. The safe way to enable settlement — `set_settlement_prices(enabled: some(true))` with the price options left `none`, which preserves the existing price (:1161-1166) — does not trigger this; the loss requires an affirmative price change after closing has started, which is why likelihood is low. A malicious or compromised vendor can also weaponize it: enable at a favorable price for a controlled account, let it close, then re-price for everyone else.

Recommendations:

Make the settlement price immutable once settlement has begun. In `set_settlement_prices`, once `enabled` has ever been set true (or any position has settled at the closed price), reject changes to `base_price` / `collateral_price` and allow only `deallocate_collateral` and `enabled` to change. This needs a `settlement_started` flag on `SettlementPrices` (or reuse "enabled has ever

been true"). Permit re-pricing only while `enabled == false` and no close has occurred, so an incorrect close price can still be corrected before anyone settles. Add a test asserting that a price change after the first settlement close aborts.

Customer Response:

Fix in commit [c262260](#) Price setting and enabling were split: `set_settlement_prices` may still correct prices while settlement is disabled, but `enable_settlement` freezes them and later edits abort with `settlement_already_enabled`. The old admin pre-enable close path was removed, so every settlement close uses one frozen price

Fix Review:

Fix confirmed.

M-03: Default bad-debt thresholds make any insurance-fund-exceeding bad debt revert public liquidation, leaving recovery only to permissioned ADL

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/market.move	Status: Acknowledged	

Description:

`create_market_params` ships every market with `max_bad_debt: 0` and `max_socialize_losses_mr_decrease: 0`, and the parameter setter only re-validates with a `u256` no-op, so nothing on-chain requires non-zero thresholds before a market is activated.

When liquidation records `bad_debt > 0` and the insurance fund cannot cover it in full (`insurance_fund_value < bad_debt`), `handle_bad_debt` takes the else branch: it first transfers the entire IF balance to the vault, then calls `try_socialize_bad_debt` with `amount_to_socialize = (bad_debt - insurance_fund_value) * collateral_price` — the uncovered deficit in USD collateral units, always > 0 in this branch. With default `max_bad_debt == 0`, `try_socialize_bad_debt` hard-aborts on the first threshold gate:

```
// clearing_house.move:4063-4075
// If the amount of bad debt is below the thresholds, proceed with
socializing losses, otherwise abort and handle with ADL.
// Checks: Notional bad debt amount below `max_bad_debt`
// - Bad debt amount relative to `open_interest * mark_price` below
`max_socialize_losses_mr_decrease`
let social_fundings = ifixed::div(amount_to_socialize, open_interest);
assert!(ifixed::less_than_eq(amount_to_socialize, max_bad_debt),
bad_debt_notional_above_threshold!());
assert!(ifixed::less_than_eq(
    ifixed::div(social_fundings, mark_price),
    max_socialize_losses_mr_decrease
),
```

```
        bad_debt_socialization_above_threshold!()  
    );
```

With `max_bad_debt == 0` this requires `amount_to_socialize <= 0`, which fails whenever the IF deficit is positive. `handle_bad_debt` is invoked from `execute_liquidation` immediately after `settle_liquidated_position`, still inside the same permissionless `liquidate<T>` transaction; The abort rolls back the entire liquidation — including the liqee size reduction — and the underwater position stays open. The zero-open-interest check is a second independent hard abort on the same path.

`try_socialize_bad_debt` also caps the relative socialize burden: `amount_to_socialize / (open_interest × mark_price)` must be $\leq$ `max_socialize_losses_mr_decrease`. It defaults to 0 alongside `max_bad_debt`, so any positive socialize amount would fail this gate too; under the shipped defaults the first gate (`max_bad_debt`) aborts first. Fixing only `max_bad_debt` while leaving `max_socialize_losses_mr_decrease` at zero (or too low for the deficit/OI ratio) leaves the same liquidation revert.

The only remaining recovery is `execute_adl`, which is gated by `AuthorityCap<PACKAGE, ADL>`. If the ADL operator is delayed or unavailable, public liquidators are blocked while the insolvent position stays open and losses grow. This is an availability / insolvency-growth defect on the otherwise-permissionless liquidation path, driven entirely by the shipped default configuration.

Recommendations:

Require non-zero, governance-approved `max_bad_debt` / `max_socialize_losses_mr_decrease` before a market can be activated, and validate them in the parameter setter. Alternatively, transition an insolvent position into an explicit ADL-required state instead of reverting the liquidation, and when open interest is zero socialize via the insurance fund or a protocol-controlled deficit account rather than aborting.

Customer Response:

The Aftermath team acknowledged this and intend to enforce via operation procedure: "The default values are usually changed during market creation by Admin, depending on the type of market."

M-04: Bad debt socialization can cause vault shortfall in certain cases

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move	Status: Fixed	

Description:

During bad debt socialization (before public settlement) the function `close_position_at_settlement_prices_as_admin` is called. If *only longs* or *only shorts* are closed then `try_socialize_bad_debt` uses the correct divisor. This is because if only longs/shorts are closing and shorts/longs are paying then the open interest of the shorts/longs isn't changing.

File: [packages/perpetuals/sources/clearing_house.move](#)

```
let social_fundings = ifixed::div(amount_to_socialize, open_interest);
```

But this isn't true if *in the act of paying the losing side the payer goes underwater*. Imagine there are two longs and two shorts L1, L2, S1, S2. Open interest is $L1 + L2 == S1 + S2$. Consider the case where L1 closes and some bad debt is socialized.

Thus S1 and S2 must pay the bad debt. But S1 may pay its share of the bad debt and *go underwater* itself!

This means that longs must now pay the bad debt for S1. However, L1 is gone now and so open interest is no longer $L1 + L2$, but just L2! The `open_interest` divisor is incorrect. This leads to a situation where at least the last call to `close_position_at_settlement_prices` (once public settlement is enabled) will fail since there aren't enough funds in the vault. The reason is that `social_fundings` is a per-unit funding charge, and it is recouped from the winning side. In the example above L1 has been removed from the longs. No funds are recouped from it, only from L2, when S1 goes underwater. When it comes time for L2 to be closed there just aren't enough funds for it in the vault.

Customer Response:

Fix in commit [c262260](#) redesigned closed-market settlement and removed the vulnerable path. `SettlementPrices` no longer stores a frozen `open_interest` `close_position_at_settlement_prices_as_admin` and its optional no-deallocation flow was removed; and public settlement no longer socializes bad debt through cumulative funding.

Fix Review:

Fix confirmed.

M-05: ADL can leave a flat position with negative collateral that settlement cannot resolve

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/adl.move packages/perpetuals/sources/clearing_house.move	Status: Acknowledged	

Description:

`close_position_at_settlement_prices_` only recognizes bad debt inside the `base_asset_amount != 0` branch (`clearing_house.move:4754`):

```
if (base_asset_amount != 0) {
  // ... realize PnL ...
  if (ifixed::is_neg(position.collateral()))
    bad_debt = position.reset_collateral();
};
```

A position with `base_asset_amount == 0` but `collateral < 0` is skipped entirely: `bad_debt` stays 0, the negative collateral is never socialized, and if `deallocate_collateral` is true the subsequent `ifixed::to_balance(position.collateral(), scaling_factor)` (`clearing_house.move:4781`, whose comment asserts "safe since non-negative") aborts, because `to_balance` requires `x < GREATEST_BIT` (`ifixed.move:148`) and a sign-magnitude negative fails that check. Settlement of that account is then permanently bricked; with `deallocate_collateral = false` the debt is instead silently dropped.

That `base == 0 && collateral < 0` state is reachable through ADL. `execute_adl` distributes a share of the bad-debt position's collateral to each counterparty with a bare signed add and no reset or solvency check on the counterparty (`adl.move:277`):

```
counterparty.add_to_collateral(absorbed_collateral); // absorbed_collateral < 0
for a short bad-debt position
```

`reset_collateral` is called only on the bad-debt position (`adl.move:220`), never on the

counterparties. A counterparty whose entire base is closed by the ADL reduction (`size_reduced == its base`) is left `base == 0`, and if the negative collateral it absorbs exceeds its own collateral it is left `collateral < 0`. That position then reaches settlement in the unhandled state above.

The state is created by the ADL keeper (`AuthorityCap<PACKAGE, ADL>`), a trusted role, so this is not an unprivileged attack — but it is a correctness break in the bad-debt waterfall that an honest keeper can trigger (for example when a small counterparty is fully closed while absorbing debt), producing a settlement-time denial of service or silently unrecognized bad debt. Hence Medium, gated on the ADL role.

Recommendations:

Update the bad-debt handling so the reported scenario is resolved regardless of whether the affected position still has open base exposure. Settlement should recognize and route any negative collateral through the normal bad-debt waterfall before attempting collateral deallocation, so a flat position with `collateral < 0` cannot be skipped or cause a latent `to_balance` abort. In addition, `execute_adl` should enforce a post-distribution solvency invariant for counterparties, or otherwise reset/route newly created debt, so ADL cannot leave a counterparty in an unresolved `base == 0 && collateral < 0` state.

Customer Response:

Acknowledged. ADL is the ultimate bad debt settlement mechanism. It's supposed to not be executed with errors and if so (hopefully no), it can be used again to fix the previous usage. For what regards the bug, we are planning some small improvements to the `close_market` procedure, but the main point is that bad debt (evaluated at the settlement prices) will need to be handled through ADL before enabling the permissionless settling of the position. There is no way to verify this fully happened at the moment of enabling the settlement, but this is the best we can do. All the bad debt will be handled through ADL, with a new entry function that will work on closed-markets only and will use the settlement prices to calculate bad debt.

M-06: Missing hard cap on branch_max/leaf_max enables permanent orderbook DOS

Severity: Medium	Impact: High	Likelihood: Low
Files: sources/ordered_map.move packages/perpetuals/sources/orderbook.move	Status: Fixed	

Description:

`check_map_params` enforces structural consistency between the B+ tree parameters (e.g. $\text{branch_min} \leq \text{branch_max}/2$) but places no upper bound on the absolute values of `branch_max` or `leaf_max`. `create_orderbook` can instantiate a map with arbitrarily large node size limits.

Attack / misconfiguration scenario

1. An orderbook is created with `leaf_max` set to a large value (e.g. 100,000+).
2. As orders accumulate, leaf nodes grow toward `leaf_max` without splitting.
3. Once a leaf approaches `leaf_max` entries, a single insert triggers `split_leaf`, which copies $\text{leaf_max} / 2$ elements into a new node. At `leaf_max = 250,000` this exhausts Sui's per-transaction computation budget (~50M gas units), causing the transaction to abort.
4. From this point, every insert into that price level aborts — the orderbook at that side/price is permanently stuck.

Why `change_params` does not rescue

`change_params` enforces $\text{new_branch_min} \leq \text{current_branch_min}$ and $\text{current_branch_max} \leq \text{new_branch_max}$ — it can only widen the range, not shrink nodes. Lowering `leaf_max` below the current value is rejected. Even if that constraint were lifted, rebalancing oversized existing nodes requires the same $O(\text{leaf_max})$ splitting operations that already hit the gas limit. There is no recovery path once the tree is in this state.

Recommendations:

Add a protocol-level hard cap in `check_map_params`:

```
const MAX_BRANCH_SIZE: u64 = 1_000;
const MAX_LEAF_SIZE:    u64 = 1_000;
assert!(branch_max <= MAX_BRANCH_SIZE, err_invalid_map_params!());
assert!(leaf_max    <= MAX_LEAF_SIZE,    err_invalid_map_params!());
```

The cap should be chosen conservatively enough that a full split at `leaf_max` elements fits well within a single transaction's gas budget, with headroom for the rest of the transaction's work (position updates, fee accounting, etc.). Values in the low hundreds are safe; values above a few thousand approach risk territory.

Customer Response:

Fixed in commit [67c846f](#). Added a hard cap of 200 on both `branch_max` and `leaf_max`, asserted in `orderbook::create_orderbook` before the maps are built, under new error code 3001.

The cap is enforced in the `Perpetuals` package, not in `ordered_map::check_map_params`. `ordered_map` is shared by several products and has no view of the consumer's per-transaction work. `orderbook::create_orderbook` is `public(package)` and its only caller is gated on `AuthorityCap<VENDOR, ADMIN|ASSISTANT>`. A loose ceiling will also be added upstream in `move-ordered-map`.

Fix Review:

Fix confirmed

M-07: A semi-trusted vendor's first market registration initializes the package-global collateral oracle used by every vault for that collateral

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/registry.move packages/perpetuals/sources/clearing_house.move packages/market-making-vault/sources/vault.move	Status: Acknowledged	

Description:

The package-global `CollateralInfo<T>` — the collateral oracle (`collateral_storage_id`, `collateral_source_id`) and `scaling_factor` that the market-making vault uses to price all collateral — is created by the first market that registers collateral type `T`, not by a package admin.

`register_market` adds the global `CollateralInfo<T>` only if it does not already exist, copying the storage/source straight from the registering clearing house's own params:

```
// registry.move:531-547 (register_market<T>)
if (!registry.is_collateral_registered<T>()) {
  df::add(
    &mut registry.id,
    keys::registry_collateral_info<T>(),
    CollateralInfo<T> { collateral_storage_id, collateral_source_id,
scaling_factor }
  );
  ...
};
```

This path is reachable by a semi-trusted vendor. `register_market` and `create_clearing_house` are both gated by `assert_admin_or_authorized_assistant_authority_cap` + `assert_vendor_has_ownership_over_clearing_house` (`clearing_house.move:401-402`, `4864`). The role check is tier-only (`assert_is_admin_or_assistant<Role>`, `authority.move:265`), so a vendor's own `AuthorityCap<VENDOR<VendorKey>, ADMIN>` (created by `create_vendor_admin_cap`, `authority.move:140`) satisfies it for markets that vendor owns. When creating the clearing house the vendor freely chooses `collateral_oracle` and

`collateral_source_id`; the only validation is that the source exists in that storage (clearing_house.move:4871-4874) — there is no constraint that the feed is the canonical one for coin type `T`.

The value the vendor picks then becomes a package-wide state that unrelated vaults consume. `create_vault` reads the global `CollateralInfo<C>` and forces the vault's collateral oracle to match it:

```
// vault.move:212-232 (create_vault)
let (collateral_storage_id, collateral_source_id, scaling_factor) =
perps_registry.collateral_info<C>();
assert!(collateral_storage_id == collateral_oracle.storage_id(),
errors::wrong_collateral_oracle());
... VaultParams { collateral_storage_id, collateral_source_id, scaling_factor,
... }
```

The cached `vault_params.collateral_source_id` is what every deposit, withdraw, force-withdraw, and NAV computation prices collateral with (vault.move:1440, 1789, 1900, 1963, 2122). A vault curator cannot opt out: to create the vault they must pass a `collateral_oracle` whose storage id equals the poisoned global, or `create_vault` reverts.

That the global is meant to be admin-controlled is evident in the code itself: `set_collateral_info<T>` (registry.move:763) lets a package `AuthorityCap<PACKAGE, ADMIN_OR_ASSISTANT>` change it. But that setter uses `df::borrow_mut` (registry.move:772), which aborts if the entry does not exist — so the admin can only *update* the global, never *create* it. The one and only creator is the first vendor's `register_market`. Package-wide collateral metadata that has a package-admin setter is therefore always bootstrapped by a vendor-scoped action with no admin gate and no feed validation — an authority inconsistency.

Impact. A first-registrant vendor who is malicious, compromised, or simply careless binds the package-global collateral feed for `T` to a wrong or manipulable source. Every vault for collateral `T` — including one operated by a different, honest vendor or an independent curator — caches and prices against it. If the feed is manipulable (permissionless oracle updates), the vendor can move the vault's NAV to mint cheap LP shares on deposit or over-redeem on withdraw, extracting value from the other LPs; if it is merely wrong, vault accounting is systematically mispriced and position NAV is valued against a collateral feed different from the clearing houses being valued.

The impact crosses the vendor's own scope, which is the same semi-trusted-vendor trust-boundary.

Likelihood is Low: it needs a bad-faith or negligent first registrant for a shared collateral type, an inattentive vault curator (who is forced to pass the poisoned oracle and could notice the mismatch), and, for value extraction, a genuinely manipulable feed. The harm is also recoverable going forward — a perps admin can correct the global with `set_collateral_info` and a vault admin can re-sync a vault with `interface::set_collateral_pfs_info` (vault.move:823) — but only reactively, and any mispriced deposits/withdrawals executed before the fix are already realized.

Recommendations:

Do not let vendor market registration initialize package-wide collateral metadata. Initialize `CollateralInfo<T>` only through a package-admin path (an allowlist of approved collateral types, each bound to a pre-approved storage/source/scaling by a `AuthorityCap<PACKAGE, ...>` holder), and make `register_market` require that the registering clearing house's collateral params match the already-approved `CollateralInfo<T>` rather than being able to create it.

Customer Response:

Acknowledged, but this is basically impossible to happen because we plan to create many markets before even considering the VENDOR feature to be enabled for external parties. For package size reasons, we prefer to not introduce additional code right now if not extremely necessary, but we'll pay attention to this for the cross margin protocol we are developing.

M-08: Vault controllers can rewrite existing LPs' fee and withdrawal terms

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: packages/market-making-vault/sources/vault.move	Status: Acknowledged	

Description:

The vault stores `owner_fee_rate`, `lock_period`, and `force_withdraw_delay` as mutable global `vault_params`. The per-LP objects (`UserLpCoin`, `WithdrawRequest`) snapshot only timestamps, not the terms — so every exit reads the CURRENT global values against the LP's original timestamp.

Withdrawal request eligibility uses the current `lock_period`:

[File: market-making-vault/packages/market-making-vault/sources/vault.move:1728-1731](#)

```
assert!(  
    user_lp_coin.start_timestamp_ms + vault.vault_params.lock_period <=  
    request_timestamp_ms,  
    errors::lock_period_not_passed()  
);
```

Force-withdraw eligibility uses the current `force_withdraw_delay` against the request's original timestamp:

[File: market-making-vault/packages/market-making-vault/sources/vault.move:1868-1871](#)

```
assert!(  
    request_timestamp_ms + vault.vault_params.force_withdraw_delay <= now_ms,  
    errors::force_withdraw_delay_not_passed()  
);
```

The owner fee applies the current rate to ALL gains accrued since deposit:

[File: market-making-vault/packages/market-making-vault/sources/vault.move:2822-2823](#)

```
let profit_usd = ifixed::sub(value_to_withdraw, provided_value_usd);  
let owner_fees_usd = ifixed::mul(profit_usd, vault.vault_params.owner_fee_rate);
```

The setters require an admin/assistant cap — [set_owner_fee_rate \(interface.move:934\)](#), [set_lock_period \(interface.move:957\)](#), [set_force_withdraw_delay \(interface.move:989\)](#) — and are bounded by config ceilings — 2 weeks / 1 day / 20% ([config.move:157-159](#)).

`min_expected_balance_out` protects the value received but not these configuration changes([vault.move:2840](#)): it only reverts, leaving the LP unable to exit under the original terms.

A vault admin or assistant can change the rules after an LP has already deposited. They can make the lock longer so the LP cannot withdraw yet, make a pending withdrawal wait longer, or raise the fee just before withdrawal and take a cut of all profits earned so far — not only profits after the fee change. These changes need a privileged cap and are capped (2 weeks / 1 day / 20%), so severity is Medium, but they still let the controller worsen terms the LP already accepted.

Recommendations:

Make new rule changes apply only going forward. When an LP deposits, store the unlock time and fee rate on their `UserLpCoin`. When they request a withdrawal, store when force-withdraw is allowed on their `WithdrawRequest`. Keep those stored values when shares are split or joined. Or, use timelock with capped minimum delay to change the configuration rules and give existing LPs a short window to exit under the old rules.

If code change is not possible, should document and inform users clearly that these terms can be changed at any time by the vault controllers.

Customer Response:

Acknowledged. This is expected. Users are supposed to deposit into vaults that they trust and this is explicitly communicated to them.

M-09: u64 cancellation totals can make a large losing position impossible to liquidate

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/adl.move packages/position/sources/position.move	Status: Fixed	

Description:

The protocol stores pending order quantity in fixed-point `u256`, but some cancellation paths sum canceled order sizes in `u64`.

This mismatch lets a trader create pending reduce-only orders whose total size is valid in the position accounting, but too large for the cancellation accumulator. When liquidation, ADL, or settlement tries to cancel these orders, the transaction aborts. Because Sui transactions are atomic, the orders are restored and the position remains unresolved.

The impact is serious if the state can be created: a bad position can block liquidation and bad-debt handling. However, the required position size is very large. This is only realistic if `max_open_interest` is configured far above normal market limits for a low-priced asset. This is why the overall likelihood is Low.

This issue is related to the known issue "Downcast from `u256` to `u64` could make positions unliquidatable", but it is not the same sink. The known issue is about a `u256` to `u64` downcast during liquidation size calculation. This report is about `u64` overflow while summing canceled order sizes in:

- `force_cancel_orders`
- `cancel_orders_`
- `try_cancel_orders_`

Also, capping `max_open_interest` at `u64::MAX / 1e9` is not enough for this exact bug, because two orders can overflow the aggregate at about half that amount.

Both issues share the same practical precondition: the market must allow a very large base-token position on a low-priced asset.

Pending quantities are stored in `u256`:

[File: packages/position/sources/position.move:453-461](#)

```
position.bids_quantity = ifixed::add(position.bids_quantity, fixed_value);
```

But forced cancellation uses `u64` totals:

[File: packages/perpetuals/sources/clearing_house.move:4101-4122](#)

```
let mut base_bid = 0;
...
base_bid = base_bid + size_cancelled;
```

The code limits the number of pending orders, but it does not limit the total same-side pending quantity. Also, `assert_reduce_only` checks each reduce-only order against the current position size, but it does not subtract reduce-only orders that are already resting.

Consider following attack Scenario :

Assume a low-priced market where a very large base position is allowed.

1. The attacker opens a short position of size `B`, where `B > u64::MAX / 2` in `B9` units.
2. The attacker places two non-expiring reduce-only bid orders.
3. Each bid has size `B`.
4. Each order is individually valid, because each one only checks against the short position size.
5. The total pending bid quantity becomes `2B`, which is greater than `u64::MAX`.

When the position later becomes liquidatable:

```
liquidate([bid1, bid2])
-> force_cancel_orders sums B + B in u64
```



```
-> u64 overflow  
-> transaction aborts
```

If the liquidator passes only one order ID, liquidation still fails:

```
liquidate([bid1])  
-> one order is canceled inside the transaction  
-> pending_bids is still not zero  
-> zero-pending assertion fails  
-> transaction aborts and the canceled order is restored
```

So both choices fail:

- passing all order IDs overflows the **u64** accumulator;
- passing fewer order IDs fails because pending orders remain.

The same pattern affects ADL and settlement because they also need to cancel all pending orders before closing the position.

A malicious trader can make a losing position impossible to resolve by protocol-controlled flows.

Possible impacts:

- liquidation reverts;
- ADL reverts;
- settlement reverts;
- bad debt cannot be realized or socialized;
- an attacker can keep the losing account blocked while realizing profit on a hedged account or external venue.

The owner can still voluntarily cancel orders one by one, but a malicious owner will not cooperate. The protocol should not rely on the account being liquidated to help the liquidation succeed.

The overflow threshold is based on base-token amount, not USD value.

With the two-order setup, the attacker needs more than about $9.22e9$ whole base tokens. This can be realistic only for a low-priced, high-supply asset if `max_open_interest` is configured too high.

For example:

- at $\$0.00001$ per token, the notional size is about $\$92k$;
- at $\$0.02$ per token, the notional size is about $\$184M$;
- for BTC, the required size is impossible in practice.

This issue is not reachable under that realistic cap. It becomes reachable only if the market cap is raised far above realistic values.

Recommendations:

Use the same fixed-point `u256` domain for cancellation accounting that the position already uses for pending quantities.

Illustrative diff:

```
public(package) fun force_cancel_orders(
    orderbook: &mut Orderbook,
    account_id: u64,
    order_ids: &vector<u128>,
    ch_id: ID,
    cancelation_reason: u8,
-): (u64, u64) {
+): (u256, u256) {
    let mut i = 0;
-    let mut base_ask = 0;
-    let mut base_bid = 0;
+    let mut base_ask = 0;
+    let mut base_bid = 0;
    let n_orders = vector::length(order_ids);

    while (i < n_orders) {
        let order_id = *vector::borrow(order_ids, i);
        let (size_cancelled, client_order_id) =
orderbook.cancel_limit_order(account_id, order_id);
```

```

+       let fixed_size_cancelled = ifixed::from_balance(size_cancelled,
constants::b9_scaling!());

    events::e22(
        ch_id,
        account_id,
        order_id,
        client_order_id,
        size_cancelled,
        cancelation_reason,
    );

    if (order_id::is_ask!(order_id))
-       base_ask = base_ask + size_cancelled
-       else base_bid = base_bid + size_cancelled;
+       base_ask = ifixed::add(base_ask, fixed_size_cancelled)
+       else base_bid = ifixed::add(base_bid, fixed_size_cancelled);
    i = i + 1;
};

(base_ask, base_bid)
}

fun settle_liquidated_position<T>(
    hot_potato: &mut SessionHotPotato<T>,
    liqee_account_id: u64,
-   liqee_base_ask_cancel: u64,
-   liqee_base_bid_cancel: u64,
+   liqee_base_ask_cancel: u256,
+   liqee_base_bid_cancel: u256,
    liqee_pending_orders_cancel: u64,
    ch_id: &ID,
) {
    ...
    position.sub_from_pending_amount(
        constants::ask!(),
-       ifixed::from_balance(liqee_base_ask_cancel, constants::b9_scaling!())
+       liqee_base_ask_cancel
    );

```

```
position.sub_from_pending_amount(  
    constants::bid!(),  
-    ifixed::from_balance(liqee_base_bid_cancel, constants::b9_scaling!())  
+    liqee_base_bid_cancel  
);  
}
```

Apply the same pattern in `cancel_orders_`, `try_cancel_orders_`, `execute_liquidation`, and the ADL caller. Per-order events can still emit the original `u64` order size.

As defense in depth, the protocol may also enforce a per-side aggregate pending-size limit during order placement. However, the main fix should be to make cancellation accounting use the same width as position pending accounting.

Customer Response:

Fixed in commit [dba11e4](#). In all three functions named `force_cancel_orders`, `cancel_orders_` and `try_cancel_orders_` each summed the cancelled sizes into a `u64` accumulator, while a position's pending quantity per side is tracked in `u256` and is not capped. Both preconditions also hold: `assert_reduce_only` clips each order to the position size computed from `position_base_before` plus fills and does not subtract orders already resting, and `try_cancel_stale_limit_order` judges reduce-only staleness by side only, so a keeper cannot pick such orders off one at a time.

The accumulators are now `u128`:

- `force_cancel_orders` → returns (`u128`, `u128`, `u64`)
- `cancel_orders_` → `cancel_size_ask` / `cancel_size_bid` are `u128`
- `try_cancel_orders_` → `cancel_size_ask` / `cancel_size_bid` are `u128`

`execute_liquidation` and `settle_liquidated_position` take `u128` for the two cancel totals, and the six conversion sites use `ifixed::from_u128balance` instead of `ifixed::from_balance`. The conversion still happens once per side rather than once per order, so the liquidation path gains only `u128` additions.

`u128` cannot overflow here. `max_pending_orders` is bounded by `RegistryConfig.up_max_pending_orders`, 100 by default, so the worst case is $100 * u64::MAX$, about $1.8e21$, against a `u128` range of about $3.4e38$.

We did not cap `max_open_interest`, since as the report notes that would not close this on its own, and we did not change `assert_reduce_only` to net out resting reduce-only orders, which is a behavioural change beyond the finding.

One correction to the likelihood assessment. The report treats the state as reachable only if `max_open_interest` is configured far above normal market limits. The default is `ifixed::from_u64(0xFFFFFFFFFFFFFFFF)`, which is `u64::MAX` whole base tokens, about `1.8e19`, so a market permits this at creation. The binding constraint is the `u64` order size in `B9` units, about `1.8e10` whole tokens, not the open-interest cap.

A second correction, in the other direction. Stacking more than two orders does not lower the bar. Margin is charged on the gross pending stack, since `abs_net_base` is `max(|base + bids|, |base - asks|)`, so N full-size reduce-only bids on a short of B require margin on $(N-1) * B$ while the overflow needs $N * B > u64::MAX$. Two orders is the cheapest case, and the threshold of roughly `9.22e9` whole base tokens stated in the report is correct. Splitting across both sides does not help either: the accumulators are per side, and `abs_net_base` is at least half the larger side.

Fix Review:

Fix confirmed.

M-10: Small force-withdraw requests can cancel all vault orders across markets

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: packages/market-making-vault/sources/vault.move	Status: Acknowledged	

Description:

A small LP can use force-withdraw to cancel all pending orders of the market-making vault.

The problem is that force-withdraw requires every pending order to be canceled, even when the withdrawal amount is very small. If the vault has orders in many markets, the force-withdraw must process every market and cancel all of those orders.

This does not let the attacker steal funds. The attacker only withdraws their own share. However, it can repeatedly stop the vault from market making. The keeper must repost all orders after each attack.

This is especially likely if `force_withdraw_delay` is set to `0`, because the attacker does not need to wait before force-processing each prepared request.

Root Cause

During force-withdraw, the vault cancels the order IDs provided by the caller:

[File: packages/market-making-vault/sources/vault.move:2310-2319](#)

```
ch::cancel_orders(  
    &mut clearing_house,  
    withdraw_session.vault.account_cap(),  
    &withdraw_session.account,  
    order_ids_copy,  
);
```

After that, it requires the position to have no pending orders:

[File: packages/market-making-vault/sources/vault.move:2322-2331](#)

```
assert!(  
    pending_asks == 0 &&  
    pending_bids == 0 &&  
    pending_orders == 0,  
    errors::wrong_order_ids_in_force_withdraw()  
);
```

This means force-withdraw only succeeds if the caller cancels all pending orders in that clearing house.

The check does not depend on the withdrawal amount. A dust withdrawal and a large withdrawal both require the same full order cancellation.

Consider the following attack scenario:

1. The attacker becomes a small LP in the vault.
2. The attacker waits until the LP coin is unlocked.
3. The attacker splits the LP coin into many small LP coins.
4. Each split coin keeps the original unlock timestamp, so all split coins are already mature.
5. The attacker creates many small withdraw requests, possibly from different addresses.
6. The attacker sets `min_expected_balance_out = 0`.
7. After `force_withdraw_delay` passes, the attacker force-processes each request.
8. For each force-withdraw, the attacker supplies all current vault order IDs.
9. The vault cancels all pending orders in every processed market.
10. The keeper must repost the vault's orders.

If `force_withdraw_delay` is configured to `0`, step 7 is immediate. This makes repeated griefing much easier.

`create_withdraw_request` only requires the LP amount to be nonzero:

```
assert!(lp_coin_amount != 0, errors::withdraw_amount_zero());
```

It also accepts a user-provided `min_expected_balance_out`, including `0`.

When an LP coin is split, the new coin keeps the original `start_timestamp_ms`:

```
start_timestamp_ms: user_lp_coin.start_timestamp_ms,
```

So one matured LP position can be split into many matured small LP coins.

The force-withdraw delay has no lower bound in the setter:

```
assert!(  
    force_withdraw_delay <= config.max_force_withdraw_delay(),  
    errors::invalid_force_withdraw_delay()  
);
```

So `force_withdraw_delay = 0` is valid.

Finally, ending a withdraw session requires all clearing houses to be processed:

```
assert!(ch_ids.is_empty(), errors::not_all_chs_processed());
```

Because each clearing house requires zero pending orders, one force-withdraw can wipe the vault's orders across all markets it uses.

The attacker does not steal vault funds. However, the attacker can repeatedly cause:

- all vault quotes to be canceled;
- loss of market-making revenue;
- wider spreads while the vault is not quoting;
- keeper reposting costs;

The cost is asymmetric:

- attacker cost: dust LP amount plus gas;
- keeper cost: repost orders across markets and manage risk while the book is empty.

This is a griefing / availability issue against the vault's core function.

Recommendations:

Enforce a minimum economically meaningful force-withdraw amount in `start_force_withdraw_session`.

This is important because if `force_withdraw_delay` is set to 0, an attacker can prepare many dust requests and execute them quickly. A minimum force-withdraw amount makes the attack much more expensive and less likely.

Recommended changes:

- Add the minimum-value check to `start_force_withdraw_session`, rather than `create_withdraw_request`, so small requests can still be processed through the normal owner-operated withdrawal path. The minimum should be protocol-bounded and should only gate the permissionless force-withdraw path. It must not allow the vault owner to arbitrarily raise the threshold and trap small LPs. Users below the threshold should still be able to cancel their withdraw request, recover the locked `UserLpCoin`, combine it with more LP balance, or wait for the owner-operated withdrawal path.
- Reject dust requests only when they enter `start_force_withdraw_session`. If the owner-operated withdrawal path does not process such a request, the user can still call `remove_withdraw_request` to recover the locked `UserLpCoin`, combine it with additional LP balance if necessary, and create a new request above the force-withdraw minimum.
- Clearly document the force-withdraw minimum and warn users that a request below it cannot use the permissionless force-withdraw fallback. Explain that these requests depend on the vault owner to process them normally and should be canceled and recreated with a sufficient amount if they remain pending.

The minimum force-withdraw amount, together with clear user documentation and the existing cancellation path, is a good immediate hardening step.

Customer Response:

Acknowledged. This is expected and we will evaluate potential countermeasures, but most likely we'll recommend to not set the force withdraw delay to 0 to highly limit the consequences of this issue

M-11: Force withdraw applies the collateral haircut before combining collateral and funding

Severity: Medium	Impact: Medium	Likelihood: Low
Files: packages/market-making-vault/sources/vault.move packages/position/sources/position.move	Status: Fixed	

Description:

The protocol applies a haircut only when the position's combined collateral and unrealized funding are positive:

```
effective_value(x) = x           if x < 0
effective_value(x) = x * (1 - haircut) if x >= 0
```

The normal position accounting first calculates **collateral + funding** and then applies this rule. However, force withdraw applies the haircut to positive funding separately when calculating **position_margin_ratio_target**:

[File: packages/market-making-vault/sources/vault.move:2258-2267](#)

```
let effective_ufundings = if (
  !ifixed::is_neg(ufundings) && collateral_haircut != 0
) {
  ifixed::mul(
    ufundings,
    ifixed::sub(ifixed::from_u64(1), collateral_haircut)
  )
} else {
  ufundings
};
```

This is incorrect when collateral and funding have opposite signs because the haircut is not a linear calculation.

The repository already tests the same zero-crossing case in `inperpetuals/tests/public_functions/test_maker_margin_check.move::margin_with_fundings_handles_haircut_zero_crossing`:

```
Collateral before funding:    -$5
Unrealized funding:          +$10
Combined value:               +$5
Haircut:                     50%
```

Before funding, negative collateral is valued in full, so the effective margin is $-\$5$. After adding funding, the combined value is $-\$5 + \$10 = +\$5$. The haircut is then applied to this positive result, producing the correct final margin of $+\$5 * 50\% = +\2.5 .

Therefore, the $\$10$ funding improves effective margin by $\$7.5$, from $-\$5$ to $+\$2.5$. Force withdraw instead haircuts the funding by itself and counts only $\$10 * 50\% = \5 of improvement. It consequently understates the achievable margin by $\$2.5$.

As a result, force withdraw can calculate a target margin ratio that is too low. A withdrawing LP can then close less exposure than required, avoiding part of the associated fees and slippage while leaving additional leverage and market risk with the remaining LPs.

The one-lot tolerance check has a related error. When the final margin ratio is above the target plus tolerance, the check asks: **would closing one lot less leave the position below the target?** If yes, the submitted close size is the closest valid size allowed by lot granularity.

[File: packages/market-making-vault/sources/vault.move:2538-2578](https://github.com/certora/aurora/blob/master/packages/market-making-vault/sources/vault.move:2538-2578)

Closing one lot less moves that lot's PnL from realized collateral back to unrealized PnL. The code estimates the resulting margin change as:

```
estimated_margin_change = one_lot_pnl * haircut
```

This is correct only when collateral remains positive before and after moving the PnL. For example, assume the collateral after closing is $-\$5$, one lot realized a $+\$2$ profit, and the haircut is 50%. Closing one lot less would change collateral to $-\$7$ and restore $+\$2$ as unrealized PnL. Because both collateral values are negative and are not haircutted, the exact margin change is zero:

Current margin component:	-\$5
One lot less:	$-\$7 + \$2 = -\$5$
Exact margin change:	\$0
Code estimate:	$\$2 * 50\% = \1

The estimate is also wrong when moving the PnL makes collateral cross zero. Consequently, the check may reject the closest valid lot-sized close or accept a non-minimal close, causing an unnecessary force-withdraw abort or excessive position reduction.

This issue requires `collateral_haircut > 0`.

Recommendations:

Use one shared helper for the protocol's piecewise haircut rule. The helper should ideally be exposed from the position package so the vault and position accounting cannot diverge:

```
+ fun effective_collateral_usd_value(  
+   value_usd: u256,  
+   collateral_haircut: u256,  
+ ): u256 {  
+   if (  
+     collateral_haircut != 0 &&  
+     !ifixed::is_neg(value_usd)  
+   ) {  
+     ifixed::mul(  
+       value_usd,  
+       ifixed::sub(ifixed::from_u64(1), collateral_haircut)  
+     )  
+   } else {  
+     value_usd  
+   }  
+ }
```

Calculate funding's effective contribution from the combined collateral state instead of haircutting funding independently:

```
- let effective_ufundings = if (  
-   !ifixed::is_neg(ufundings) && collateral_haircut != 0
```

```
- ) {  
-   ifixed::mul(  
-       ufundings,  
-       ifixed::sub(ifixed::from_u64(1), collateral_haircut)  
-   )  
- } else {  
-   ufundings  
- };  
+ let collateral_usd = ifixed::mul(  
+   position.collateral(),  
+   withdraw_session.collateral_price  
+ );  
+ let effective_collateral_before = effective_collateral_usd_value(  
+   collateral_usd,  
+   collateral_haircut  
+ );  
+ let effective_collateral_after_funding = effective_collateral_usd_value(  
+   ifixed::add(collateral_usd, ufundings),  
+   collateral_haircut  
+ );  
+ let effective_ufundings = ifixed::sub(  
+   effective_collateral_after_funding,  
+   effective_collateral_before  
+ );
```

For the one-lot tolerance check, calculate the exact counterfactual. Closing one lot less removes that lot's realized PnL from collateral and restores the same amount as unrealized PnL:

```
- let margin_delta_one_lot_less_closed = ifixed::mul(  
-   one_lot_pnl_at_mark,  
-   collateral_haircut  
- );  
+ let collateral_usd_after = ifixed::mul(  
+   position.collateral(),  
+   withdraw_session.collateral_price  
+ );  
+ let collateral_usd_one_lot_less = ifixed::sub(  
+   collateral_usd_after,
```

```
+   one_lot_pnl_at_mark
+ );
+ let effective_collateral_after = effective_collateral_usd_value(
+   collateral_usd_after,
+   collateral_haircut
+ );
+ let effective_collateral_one_lot_less = effective_collateral_usd_value(
+   collateral_usd_one_lot_less,
+   collateral_haircut
+ );
+ let margin_delta_one_lot_less_closed = ifixed::add(
+   // Change in effective collateral.
+   ifixed::sub(
+     effective_collateral_one_lot_less,
+     effective_collateral_after
+   ),
+   // The PnL is unrealized again, so add it back to margin.
+   one_lot_pnl_at_mark
+ );
```

Keep the existing `margin_after_with_one_lot_less_closed` and margin-ratio calculation, but feed them this exact delta. Alternatively, simulate the one-lot-less position through the normal position-accounting functions. Add tests where collateral stays positive, stays negative, and crosses zero for both positive and negative one-lot PnL.

Customer Response:

Fixed in commit [4704533](#). Force-withdraw now haircuts the combined collateral+funding value (and the one-lot tolerance uses the same clamp).

Fix Review:

Fix confirmed.

M-12: Freezing a lagging-version registry or clearing house can make it unfreezable

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/registry.move packages/perpetuals/sources/clearing_house.move	Status: Fixed	

Description:

The perps version gate allows objects whose stored version is less than or equal to the current package version. Therefore package version 3 can freeze a registry or clearing house still at version 2. The freeze stores the old version as `resume_version` and sets live version to `u64::MAX`.

The unfreeze path then requires `resume_version == constants::version!()`. Under the same version-3 package, an object frozen with `resume_version == 2` always aborts unfreeze. Move atomicity restores the removed dynamic field, leaving the object frozen.

Key code:

[File: packages/perpetuals/sources/registry.move:1217-1224](#)

```
assert!(registry.version <= constants::version!(), invalid_version!());
```

[File: packages/perpetuals/sources/clearing_house.move:5076-5085](#)

```
assert!(clearing_house.version <= constants::version!(), invalid_version!());
```

[File: packages/perpetuals/sources/registry.move:759-760](#)

```
let resume_version = df::remove<_, u64>(&mut registry.id,  
keys::frozen_version());  
assert!(resume_version == constants::version!(), invalid_resume_version!());
```

[File: packages/perpetuals/sources/clearing_house.move:1015-1016](#)

```
let resume_version = df::remove<_, u64>(&mut clearing_house.id,  
keys::frozen_version());  
assert!(resume_version == constants::version!(), invalid_resume_version!());
```

This can turn emergency freeze authority into a temporary fund lock for lagging markets during staged upgrades.

Recommendations:

Implement 1 of the code changes below:

1. Match MMV unfreeze semantics (preferred). Perps currently requires `resume_version == constants::version!()`. MMV allows `resume_version <= constants::version!()`. With `<=`, freezing a lagging object (e.g. object 2 under package 3) is still recoverable: unfreeze restores the old object version, then admin runs the normal `upgrade_version` path to catch up. This keeps emergency freeze usable during staged upgrades without a permanent lock.
2. Or reject lagging freezes. At freeze time, require `object.version == constants::version!()` (exact match). Then freeze is only allowed after objects are already on the current package version, so `resume_version` always matches and unfreeze cannot wedge. Downside: guardians cannot freeze markets that have not been bumped yet.

Or document clearly Operational procedure for safe upgrade.

Customer Response:

Fixed in commit [6705163](#). Both `registry::unfreeze_package` and `clearing_house::unfreeze_clearing_house` now assert `resume_version <= constants::version!()`, so a lagging-version freeze can be restored and then upgraded normally.

Fix Review:

Fix confirmed.

M-13: Liquidation bad-debt socialization can under-recover or over-charge collateral in inverse markets

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/market.move packages/position/sources/position.move	Status: Acknowledged	

Description:

Aftermath supports inverse markets, where the contract is denominated in USD but margin is settled in a volatile collateral asset such as BTC or ETH.

When a liquidation creates bad debt and the insurance fund is not enough to cover it, the protocol socializes the remaining loss to the opposite side of the market. The uncovered shortfall is first converted from collateral tokens to USD using the collateral price at the liquidation session time.

Later, the charged traders pay this socialized loss through funding settlement. At that time, the same USD amount is converted back into collateral tokens using each payer's current collateral price.

If the collateral price changes between these two moments, the number of collateral tokens collected will not equal the original token shortfall. This can leave the market vault under-recovered or charge traders more tokens than the original loss.

This is especially relevant for inverse markets because collateral price can move materially.

The protocol mixes two units:

- the liquidation shortfall is a **collateral-token** shortfall;
- the socialized loss is stored as a **USD funding-rate** adjustment;
- later settlement converts that USD amount back into collateral tokens at a potentially different price.

The accounting is therefore not token-conservative when $\text{price}(T1) \neq \text{price}(T2)$.

Vulnerable Flow

1. Liquidation records bad debt at the liquidation session price

During liquidation, if the liquidated account produces bad debt, `execute_liquidation` calls `handle_bad_debt` with `hot_potato.collateral_price`.

[File: packages/perpetuals/sources/clearing_house.move:3790-3803](#)

```
if (hot_potato.session_summary.bad_debt != 0) {
  handle_bad_debt<T>(
    &mut hot_potato.clearing_house,
    hot_potato.session_summary.bad_debt,
    hot_potato.mark_price,
    hot_potato.collateral_price,
    hot_potato.session_summary.is_liqee_long,
    ch_id,
    option::none(),
  );
};
```

2. The uncovered token shortfall is converted to USD at

T1

If the insurance fund cannot fully cover the bad debt, the remaining token shortfall is converted to USD using the liquidation-time collateral price.

[File: packages/perpetuals/sources/clearing_house.move:4181-4213](#)

```
let amount_to_socialize = ifixed::mul(
  ifixed::sub(bad_debt, insurance_fund_value),
  collateral_price
);
```

Here:

- `bad_debt - insurance_fund_value` is denominated in collateral tokens;
- `collateral_price` is the liquidation session collateral price;

- `amount_to_socialize` is denominated in USD.

3. The USD loss is added to the funding rate

The USD amount is divided by open interest and applied as a one-sided funding-rate change to the opposite side of the liquidated position.

[File: packages/perpetuals/sources/clearing_house.move:4243-4259](#)

```
let social_fundings = ifixed::div(amount_to_socialize, open_interest);

market::add_bad_debt_to_market(
  market_state,
  ch_id,
  !is_liqee_long,
  amount_to_socialize,
  social_fundings
);
```

[File: packages/perpetuals/sources/market.move:960-987](#)

```
if (add_to_long) {
  market_state.cum_funding_rate_long = ifixed::add(
    market_state.cum_funding_rate_long,
    delta
  );
} else {
  market_state.cum_funding_rate_short = ifixed::sub(
    market_state.cum_funding_rate_short,
    delta
  );
};
```

4. Payers settle later using their current collateral price

Funding settlement is public and reads the current collateral oracle price at the time of settlement.

[File: packages/perpetuals/sources/clearing_house.move:1736-1768](#)

```
let collateral_price = clearing_house
    .market_params
    .collateral_oracle_price(
        oracle,
        clock,
    );
```

The position module then converts the USD funding amount into collateral tokens using that settlement-time price.

[File: packages/position/sources/position.move:982-1019](#)

```
let numerator = abs_rate_delta * abs_base;

// User pays funding: ceil in collateral units.
let collateral_magnitude = (numerator + collateral_price - 1) / collateral_price;
position.collateral = collateral_after;
```

This means the payer does not necessarily repay the same number of collateral tokens that the vault originally lost.

Consider an example as follows:

Assume an inverse market collateralized by BTC.

At liquidation time:

```
Uncovered bad debt: 1 BTC
BTC price at liquidation: $40,000
Socialized loss recorded: $40,000
```

The protocol has lost 1 BTC from the vault accounting perspective. It expects the opposite side to absorb this loss.

Later, before the charged traders settle funding, BTC increases to \$50,000.

```
USD loss to collect: $40,000
BTC price at settlement: $50,000
Tokens collected: $40,000 / $50,000 = 0.8 BTC
```

The vault needed to recover 1 BTC, but only 0.8 BTC is collected.

Residual under-recovery: $1 \text{ BTC} - 0.8 \text{ BTC} = 0.2 \text{ BTC}$

If BTC instead decreases to \$30,000, then the charged traders pay:

Tokens collected: $\$40,000 / \$30,000 = 1.3333 \text{ BTC}$

In that case, the charged side pays more collateral tokens than the original shortfall.

So the general Formula

Let:

- D = uncovered bad debt in collateral tokens;
- $p1$ = collateral price during liquidation;
- $p2$ = collateral price when payers settle funding.

The protocol records:

```
amount_to_socialize = D * p1
```

Later it collects:

```
tokens_collected = amount_to_socialize / p2  
                  = D * p1 / p2
```

The token residual is:

```
residual = D - tokens_collected  
          = D * (1 - p1 / p2)
```

The residual is non-zero whenever $p1 \neq p2$.

In volatile-collateral markets, this can cause real accounting damage:

- the market vault can under-recover collateral tokens after liquidation bad debt;
- remaining LPs/traders may indirectly bear the shortfall;

- charged traders may pay more collateral tokens than the original bad debt if the collateral price moves the other way;
- valid withdrawals or deallocations can later fail if accounting claims exceed the real vault collateral balance.

The insolvency sink is the vault withdrawal check:

[File: packages/perpetuals/sources/clearing_house.move:3680-3769](#)

```
assert!(balance::value(&vault.collateral_balance) >= amount,  
insufficient_vault_collateral!());
```

If position accounting says users can withdraw more collateral than the vault actually has, later withdrawals can revert.

This is related to the broader routine funding price-timing conservation issue, but it is not exactly the same entry point.

The shared root cause is:

```
USD-denominated funding-like accounting is settled into collateral tokens at  
different collateral prices.
```

The difference is:

- routine funding creates funding deltas through normal periodic funding updates;
- this issue creates funding deltas through liquidation bad-debt socialization.

Recommendations:

Make liquidation bad-debt socialization price-consistent for inverse markets.

The important invariant is:

```
Total collateral tokens debited from charged positions must equal the  
collateral-token shortfall being socialized
```

The project should also add an invariant test for volatile collateral:

1. Create a liquidation that leaves D collateral tokens of uncovered bad debt.
2. Socialize the loss at collateral price p_1 .
3. Move collateral price to p_2 .
4. Settle the charged side's funding.
5. Assert that the collected collateral tokens equal D , or that any residual is explicitly accounted for.

Customer Response:

Acknowledged. The project acknowledged and decided not to use any different collateral other than USDC (stablecoins).

M-14: Maker Solvency Rejections Can Gas-Lock the Order Book

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/position/sources/position.move packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/orderbook.move	Status: Fixed	

Description:

An attacker can place many maker orders that pass the margin check when posted but cannot be filled later. When a taker crosses these orders, the protocol scans and rejects them one by one. If the transaction runs out of gas before cleanup finishes, all removals are rolled back and the bad orders remain.

The protocol records only the base quantity of a pending maker order. It does not record the order's limit price:

[position.move:453-463](#)

The order's margin requirement is therefore calculated using the mark price:

$$\text{Required margin} = \text{size} \times \text{mark price} \times \text{margin ratio}$$

However, when the order is filled, the maker trades at the actual limit price.

For a bid where **limit price** > **mark price**, the missing execution loss is:

$$\text{Missing loss} = \text{size} \times (\text{limit price} - \text{mark price})$$

This creates a mismatch:

Order admission: values the order at the mark price

Order fill: settles the order at the higher limit price

If the maker no longer has enough collateral at fill time, the maker solvency check rejects the fill:

[position.move:766-779](#)

`process_fill_maker` then sets the matched size to zero and marks the maker order for cancellation:

[clearing_house.move:3502-3508](#)

The taker's remaining size does not change, so matching continues with the next maker:

```
Taker crosses toxic order
    ↓
Maker solvency check rejects it
    ↓
Matched size is zero
    ↓
Matching scans the next maker
```

Rejected orders are removed only after the scan completes. With enough toxic orders, the scan runs out of gas before reaching cleanup. The transaction aborts, all removals roll back, and the same toxic orders remain for the next transaction.

Existing stale-order cleanup does not help because it only handles expired and invalid reduce-only orders, not maker solvency failures.

The easiest scenario for this bug to trigger is a thin market with a large order-book gap. An attacker can place orders far from the mark without crossing the opposite side.

On a liquid market, exploitation is less likely because another trader may fill the order before it becomes unsafe.

Impact of this bug: One side of the order book can become too expensive to cross. Traders may be unable to close positions normally, and vault withdrawals that unwind through market orders may also be blocked.

Liquidation does not use the order book and remains available.

Recommendations:

Add a maximum number of maker entries that one matching transaction may scan. When the limit is reached, stop matching successfully and commit the rejected-order cleanup already collected.

This prevents an out-of-gas rollback and applies to every zero-fill reason, including expiry, invalid reduce-only orders, and maker solvency rejection.

Customer Response:

Fixed in [67c846f](#). `execute_limit_order` now stops matching after 200 consecutive makers that fill nothing (`perpetuals::constants::max_zero_fill_matches`). It stops rather than aborts, so `batch_drop` still removes every order inspected: each crossing limit order clears up to 200 unfillable orders and the next one resumes where it stopped. A maker that fills resets the count.

Market orders are not bounded; `place_market_order` keeps fill-or-abort. Recovery is an IMMEDIATE_OR_CANCEL order across the affected price, which clears up to 200 per transaction and discards its own remainder, so nothing is posted.

The taker's limit price cannot bound the scan. Order ids are `price << 64 | counter` and the matching loop breaks on the price component only, so a group of orders at a single price is scanned in full regardless of how the taker prices its order.

`try_cancel_stale_orders` was retained. `start_session` requires the market not to be paused, so no crossing order is possible while a market is paused or cancel-only.

Fix Review:

Fix confirmed.

M-15: Reduce-Only Order Churn Can Repeatedly Block Liquidation

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: packages/perpetuals/sources/clearing_house.move	Status: Fixed	

Description:

A user below the Maintenance Margin Ratio (MMR) can repeatedly cancel and replace a reduce-only order.

Each replacement receives a new order ID. If a liquidator submits the previous ID, the liquidation transaction aborts.

This does not guarantee permanent protection from liquidation, because a liquidation that is executed before the next replacement will succeed. However, the user has a clear financial incentive to keep racing liquidators, and successful races can delay liquidation while the position becomes more insolvent.

Liquidation requires the caller to provide the liquidatee's exact current order IDs.

The protocol strictly removes every supplied ID. If an ID has already been canceled, `map::remove` aborts the whole transaction:

[clearing_house.move:4093-4127](#)

After cancellation, liquidation also requires all pending order state to be exactly zero:

[clearing_house.move:3861-3874](#)

Therefore, liquidation aborts in both cases:

Old order ID supplied	→ strict removal of missing ID aborts
New order ID omitted	→ pending-order-zero check aborts

The user can still post a reduce-only order while below MMR, as long as the position is not yet in bad debt.

Because the order does not increase exposure or reduce margin, `ensure_margin_requirements` treats it as a non-worsening action and allows it:

[position.move:840-886](#)

The user can combine cancellation and replacement in one atomic Programmable Transaction Block:

```
Current order: O1
    ↓
Cancel O1 and post O2 atomically
    ↓
Liquidator submits liquidate([O1])
    ↓
O1 no longer exists
    ↓
Liquidation aborts
```

The replacement can be placed far from the market, so it is unlikely to fill. The user can then repeat the process with $O_2 \rightarrow O_3 \rightarrow O_4$.

By continuing the above process, repeated order churn can cause failed liquidation transactions, waste liquidator gas, and delay liquidation.

If the position continues losing value during the delay, the insurance fund or healthy traders may eventually absorb additional bad debt. Or if the market condition improved, the attackers can avoid liquidation.

However, the attack is a transaction-ordering race. If the liquidation transaction is executed before the user's next churn transaction, liquidation succeeds. The issue therefore enables repeated temporary liquidation griefing, not guaranteed permanent liquidation censorship.

Recommendations:

While a position is below MMR, allow only reduce-only orders that cannot leave a pending remainder. Treat these orders as Immediate-Or-Cancel: keep any executed risk-reducing fill and automatically discard the unfilled remainder. Continue allowing collateral deposits and order cancellations.

Customer Response:

Fix in commit [67c846f](#).

1. `force_cancel_orders` now uses `try_cancel_limit_order` and returns the number of orders actually cancelled. Liquidation and ADL use that count for the position's bookkeeping instead of the length of the supplied vector, since `update_pending_orders` is a plain subtraction. Ids already gone from the book are skipped. The pending-order-zero assertion continues to guarantee that no order was left resting.

Without this, cancellation alone still breaks liquidation: every current id must be supplied for the pending-order-zero check to pass, so one cancellation invalidates the whole list. A position holding orders placed while healthy – up to `max_pending_orders`, 100 by default – can fail that many liquidations without reposting anything.

2. A position below MMR can no longer leave an order resting. Checked once per session in `end_session_`, reusing the margin already computed for `ensure_margin_requirements`, and reachable only below IMR with no margin allocation. New error code 57.

This is a rejection rather than the suggested conversion to `IMMEDIATE_OR_CANCEL`. Conversion must happen before matching, where the margin figure does not yet exist, and post-only would need a separate rejection because converting it inverts its contract.

Fix Review:

Fix confirmed.

M-16: Rolling blend masks a stale leg by returning the maximum leg timestamp, letting stale prices bypass the aggregator's staleness guard

Severity: Medium	Impact: Medium	Likelihood: High
Files: packages/integrations/pyth-lazer-rolling/sources/ price_feed_storage.move	Status: Fixed	

Description:

During a contract roll, `blended_price_and_timestamp_ms` produces a price by linearly weighting the front-month and next-month feeds, and returns it paired with a single timestamp. Within the blend window it returns the maximum of the two legs' timestamps.

The core aggregator uses that returned timestamp as the feed's freshness marker and enforces its `staleness_threshold` against it. Each leg's timestamp is derived per-feed (`Feed.timestamp_ms` → the feed's own `feed_update_timestamp`, which advances only when that feed's price is actually re-sourced, not on every Lazer message). Because the blend returns the maximum, the blended feed is treated as fresh as long as at least one leg is fresh:

- If the next-month (or near-expiry front-month) leg halts, lags, or is withheld while the market is open, its `feed_update_timestamp` stops advancing.
- Its stale price is still blended in at the full schedule weight, and as the roll progresses the weight on the stale next-month leg increases toward 100%.
- The fresh leg's timestamp masks the stale leg, so the aggregator sees a fresh timestamp and accepts a blended price that is heavily — eventually entirely — driven by a stale input.

This is not an edge case: next-month contracts are routinely thinner and less liquid than the front month (and the front month itself thins out near expiry), so one leg lagging during the roll is an expected operating condition. A missing leg fails closed (`EFeedNotFound`); the exposure is specifically the present-but-stale leg, whose staleness is silently discarded.

Downstream perpetual markets consume the blended price as ground truth, so a stale-weighted blend can misprice positions, trigger unwarranted or prevent warranted liquidations, and enable

mispriced entries — precisely during the roll window when weight is shifting onto the least-liquid leg.

Recommendations:

Bound the validity of a derived price by its most stale component. In the blend window, return `min(ts_current, ts_next)` instead of the maximum, so the blended feed is considered fresh only when both legs are fresh; if either leg lags, the aggregator's `staleness_threshold` correctly rejects (freezes) the update — the intended safety response.

For defense in depth, consider additionally gating each leg's own `feed_update_timestamp` against a maximum-lag bound inside the blend window, so a stale leg is caught even if the shared message timestamp (`update.timestamp`) is being used as a fallback (closed market session, or absent `feed_update_timestamp`).

Note: the previously reported post-roll "timestamp regression / oracle freeze" behavior is intentionally out of scope here — when a leg is genuinely stale, the aggregator rejecting the update is the desired outcome. Switching to `min` yields exactly that behavior without masking staleness.

Customer Response:

Fixed in commit [d96b128](#). We've switched from max -> min.

Fix Review:

Fix confirmed.

M-17: Pyth integration completely ignores the price confidence interval, exposing downstream protocols to highly uncertain prices

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: packages/integrations/pyth/sources/ price_feed_storage.move	Status: Acknowledged	

Description:

The Pyth integration extracts the `price` magnitude and `timestamp_ms` from the `PythPriceInfo` object, but it completely discards the `conf` (confidence interval) field.

Pyth's official documentation explicitly warns that during periods of high market volatility, illiquidity, or market outages, the price feed can become highly uncertain, causing the confidence interval to widen significantly. Pyth strongly recommends that protocols verify the confidence interval is within an acceptable percentage of the price (e.g., $\text{conf} / \text{price} < 1\%$) before trusting the price feed to ensure that they are not using a severely dislocated price.

Furthermore, the `oracle_aggregator` core module's `PriceFeed` struct and `PriceFeedStorage` do not contain any fields to store or expose oracle confidence intervals. This systemic design flaw forces all uncertainty-aware oracle integrations (like Pyth and Switchboard) to silently drop uncertainty data. As a result, it exposes all downstream consumers (such as the `iperps` and `market_making_vault` packages) to highly uncertain or dislocated prices. During a flash crash or liquidity crisis, the reported price may be far from the true market price, but downstream protocols will accept it as perfect truth, which can lead to unfair liquidations, bad debt, or arbitrage exploits.

Recommendations:

1. Redesign the `oracle_aggregator`'s `PriceFeed` struct to store the confidence interval (`conf`) alongside the price.
2. Update the `PriceFeedStorage` getters to expose the confidence interval, allowing downstream protocols to enforce their own uncertainty thresholds when calculating margins or processing trades.

3. Update the Pyth integration's `price` and `price_and_timestamp_ms` macros to extract the `conf` field from the Pyth object, scale it to 10181018 similarly to the price, and pass it to the `PriceFeedStorage` during feed updates.

Customer Response:

Acknowledged with no changes. Our v1 move-oracle-aggregator contained a `confidence` variable within `PriceFeed` and all `price.move` getters had variants that accepted the confidence threshold. As all oracles (optionally) implement confidence in their own way, it was impossible for us to normalize this across all oracles. In the change from v1 -> v2, we've decided to fully remove confidence from core OAgg.

As for Pyth core, our integration does not factor in confidence. If this is a property required by another user of our OAgg, a separate Pyth integration can be created that defines this concept (e.g., the integration defines a `FeedInfoObject` with a confidence threshold field). As we have phased out Pyth core from our exchange, and have seen a push for Pyth lazer over Pyth core, we have left this for future work if at all.

M-18: Rolling-feed maintenance operations lack protocol-level parameter and state-machine validation

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/integrations/pyth-lazer-rolling/sources/ feed_info_object.move price_feed_storage.move	Status: Fixed	

Description:

`migrate` and `set_rolling_window` mutate a live `FeedInfoObject` — the underlying `next_feed_id` and the roll window [`roll_start_timestamp_ms`, `roll_end_timestamp_ms`] — from caller-supplied values, and enforce essentially no protocol-level validation on those inputs. The only invariant checked is `roll_start_timestamp_ms < roll_end_timestamp_ms`. Specifically, the functions:

- do not receive or consult a `Clock`, so they cannot enforce any temporal correctness — a window entirely in the past is accepted, and a new roll can be made to "start" before the current time;
- do not require the current roll to have completed before a new one is installed (no state-machine progression; the intended one-step next → front promotion can be skipped);
- place no bound on how far the window may move, no minimum notice period, and no rate limit / one-migration-per-epoch guard;
- accept an arbitrary `next_feed_id: u32` with no relationship check tying it to the symbol/asset the `FeedInfoObject` represents.

Because these operations take effect immediately and silently, the consequence of a bad value is outsized. A past-dated window causes `blended_price_and_timestamp_ms` to select the new leg at 100% weight instantly (`roll_end_timestamp_ms <= current_timestamp_ms`), and a wrong `next_feed_id` rebinds a live symbol to an unrelated feed. The `FeedInfoObject`'s object ID is unchanged, so the core aggregator's `PriceFeed.from` check does not detect the semantic rebind, and downstream markets can consume the discontinuous price in the same programmable transaction — with potential for incorrect liquidations or bad debt.

The **MAINTENANCE** role is a trusted role (as are **ADMIN** and **ASSISTANT**), so this is not an untrusted-attacker path and should be rated accordingly. However, trust in the role does not remove the need for these validations. A long-lived operational maintenance key is a realistic compromise target, and even an honest operator can supply an incorrect value (a fat-fingered feed ID or a mistaken timestamp). In both cases the absence of contract-level guardrails means a single wrong input immediately and silently redirects a live oracle with no circuit breaker to bound the impact. Enforcing sane limits at the protocol level is defense-in-depth: it constrains the damage from a compromised key or operator error rather than relying on the role being simultaneously honest and mistake-free.

Recommendations:

Enforce the rolling state machine and input bounds at the contract level, even for trusted callers:

- Pass a trusted **Clock** and require `roll_end_timestamp_ms > now`, with the new window beginning at/after now and a minimum notice period before it takes effect.
- Permit **migrate** only after the previous `roll_end_timestamp_ms`, and prevent multiple migrations within the same roll epoch.
- Bound parameter deltas (maximum window shift, sane minimum/maximum roll durations).
- Constrain `next_feed_id` — e.g., require **ADMIN** to preapprove/register the permissible feed IDs and let **MAINTENANCE** only activate a preapproved transition, and/or use a propose/commit delay for feed-ID changes.
- Retain an **ADMIN**-only emergency override to react to unexpected delistings.

This trades a small amount of operational flexibility for a bounded impact against both key compromise and accidental misconfiguration of an otherwise-trusted role.

Customer Response:

Fixed in commit [d96b128](#). We have added a plethora of checks to `pyth_lazer_rolling_integration::feed_info_object::{new, migrate, set_rolling_window}`.

Here is a breakdown of checks we've added:

new

1. `roll_end_timestamp_ms <= expiry_timestamp_ms`
2. `next_feed_id != front_feed_id`

migrate

1. `feed_info_object.roll_end_timestamp_ms <= current_timestamp_ms`
2. `current_timestamp_ms < roll_start_timestamp_ms`
3. `roll_end_timestamp_ms <= expiry_timestamp_ms`
4. `next_feed_id != feed_info_object.next_feed_id`

set_rolling_window

1. `clock.timestamp_ms() < roll_end_timestamp_ms`
2. `roll_end_timestamp_ms <= feed_info_object.expiry_timestamp_ms`

Fix Review:

Fix confirmed

M-19: Immediate collateral-haircut increases can manufacture liquidation eligibility for solvent positions

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/market.move packages/position/sources/position.move	Status: Acknowledged	

Description:

`collateral_haircut` is set through `set_core_params` and applied IMMEDIATELY — no proposal delay, no activation timestamp, no grace period:

[File: packages/perpetuals/sources/market.move:454-456](#)

```
params.core_params.lot_size = lot_size;
params.core_params.tick_size = tick_size;
params.core_params.collateral_haircut = collateral_haircut;
```

This is the opposite of how margin-ratio changes are handled, which are the protocol's baseline for parameters that tighten solvency requirements: they go through a 24-72h delayed proposal (`create_margin_ratios_proposal` sets `maturity = now + delay_ms`, and `commit_margin_ratios_proposal` may only be applied after maturity — [clearing_house.move:1515](#), [clearing_house.move:1599](#)).

Positive collateral is valued $\text{collateral} \cdot \text{price} \cdot (1 - \text{haircut})$ while the maintenance requirement is unchanged, so raising the haircut toward 100% can push a position with positive true equity below MMR purely from the parameter update:

[File: packages/position/sources/position.move:808-816](#)

```
if (collateral_haircut != 0 && !ifixed::is_neg(position.collateral)) {
    ifixed::mul(
        collateral_usd,
        ifixed::sub(constants::one_fixed!(), collateral_haircut)
    )
}
```

```
)  
} else collateral_usd
```

Liquidation reads the new haircut immediately for both eligibility and sizing ([clearing_house.move:3821](#), [clearing_house.move:4306-4319](#)). A vendor can therefore update the haircut, start a liquidation session, and liquidate the affected accounts in the SAME PTB — collecting the liquidation fee from positions that were healthy an instant earlier. This bypasses the delayed margin-ratio mechanism while producing an economically-equivalent-or-stronger increase in collateral requirements, with no window for users to add collateral or reduce exposure.

Recommendations:

Route haircut INCREASES through the same delayed proposal mechanism used for margin-ratio changes (snapshot value + activation timestamp + grace period); haircut decreases can stay immediate. For emergencies, allow an immediate pause of new risk-taking / matching, but require delayed activation before existing solvent positions can be liquidated under a harsher haircut.

Customer Response:

Acknowledged. The project does not expect vendors in this version of the contracts so we preferred to leave the haircut as a vendor parameter to save on package size.

M-20: Resting Perpetuals orders remain executable during a force-withdraw pause and can censor exits

Severity: Medium	Impact: Low	Likelihood: High
Files: packages/market-making-vault/sources/vault.move packages/market-making-vault/sources/perpetuals_api.move	Status: Fixed	

Description:

Pausing a vault for force withdrawal only writes the local `vault.paused` field. This field is checked when the vault initiates new Perpetuals operations, but the pause is not propagated to the vault's Perpetuals `Account`, its positions, or its resting orders.

Consequently, limit orders posted before the pause remain executable as maker orders. Any external taker—including an account controlled by a malicious vault operator—can fully fill one of these orders after the pause without invoking a vault entry point or checking `vault.paused`.

This conflicts with force-withdraw processing, which calls the strict `ch::cancel_orders` function for every supplied order ID. If even one supplied order was filled and removed after the user obtained the order list, cancellation aborts. Conversely, omitting an order that still exists causes the subsequent zero-pending-orders assertion to abort.

Recommendations:

Make force-withdraw cancellation idempotent: tolerate already-filled or missing IDs—such as through `try_cancel_orders`—and then enforce that the position has no remaining orders. Since the vault pause already blocks posting new orders through vault entry points, this permits the withdrawal to proceed against the live post-fill position.

Alternatively, cancel every tracked order atomically when entering the paused state. This increases pause transaction gas and cross-contract contention, but provides the state-consistency guarantee the pause currently promises.

Customer Response:

Fixed in commit [c7cfae8](#). Force-withdraw cancellation now calls `ch::try_cancel_orders` instead of `ch::cancel_orders`, so ids that are no longer on the book are skipped rather than aborting the

withdrawal. The `pending-order-zero` assertion that follows is unchanged and remains the guarantee that nothing was left resting: omitting an id for an order that is still on the book still aborts with `wrong_order_ids_in_force_withdraw`.

The copy of the id vector that the strict entry point required has been removed, since `try_cancel_orders` takes a reference.

Not changed: pausing the vault still does not propagate to the vault's Perpetuals account, so orders posted before the pause remain executable by an external taker. The withdrawal now tolerates that rather than preventing it.

Fix Review:

Fix confirmed.

M-21: Permissionless CoinRegistry::migrate_legacy_metadata can be front-run to permanently block vault creation

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: packages/market-making-vault/sources/vault.move packages/coins/sources/af-lp/sources/af_lp.move packages/coins/otw-template/sources/template.move	Status: Fixed	

Description:

The deployment flow intentionally leaves a transaction gap between publishing the LP type and creating its vault. During initialization, the canonical `CoinMetadata<L>` is frozen and becomes publicly usable. Vault creation later calls `coin_registry::migrate_legacy_metadata` unconditionally. That framework operation is permissionless and claims the unique `Currency<L>` registration. A subsequent migration attempt aborts once the registration has already been claimed. An attacker can observe the LP package publication and migrate its frozen metadata before the owner submits `create_vault`. Every subsequent vault-creation attempt will reach the unconditional migration and abort atomically, persistently grieving vault creation.

Recommendations:

Make vault creation tolerant to existing metadata registrations: check whether a `Currency<L>` already exists and skip migration (or accept a pre-existing `Currency<L>` as an alternative input) instead of aborting. Alternatively, gate the migration to the coin publisher.

Customer Response:

Fixed in commit [c7cfae8](#). `create_vault` now skips the migration when the registration already exists, instead of calling it unconditionally:

```
if (!sui::coin_registry::exists<L>(coin_registry)) {  
    coin_registry.migrate_legacy_metadata(lp_coin_metadata, ctx);  
};
```

Skipping loses nothing. Whoever calls `migrate_legacy_metadata` derives `Currency<L>` from the same frozen `CoinMetadata<L>`, so the resulting object is identical regardless of who created it,

and `create_vault` does not consume the `Currency` afterwards – it reads decimals directly from the passed `CoinMetadata<L>`.

We did not gate the migration to the coin publisher, as that is framework behaviour rather than ours.

Fix Review:

Fix confirmed

M-22: Liquidation sizing subtracts an insurance-fund fee that settlement waives, over-liquidating solvent (negative-raw-collateral / positive-equity) positions

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: packages/perpetuals/sources/clearing_house.move	Status: Fixed	

Description:

A partial liquidation should close only as much of the position as needed to bring it back to the initial-margin requirement (IMR), so the liquidated trader keeps as much residual value as possible. The protocol picks a close fraction `alpha` by solving for the point where post-liquidation margin equals the post-liquidation IMR requirement. When it does that math, the formula's denominator subtracts fees — and it subtracts both the liquidation fee and the insurance-fund fee:

[File:packages/perpetuals/sources/clearing_house.move](#) (`compute_liquidation_size_no_haircut`)

```
let fee_notional = ifixed::mul(
  ifixed::add(liquidation_fee, insurance_fund_fee),
  ifixed::mul(mark_price, abs_b),
);
...
let denom = ifixed::sub(mi_pm_abs_b, fee_notional); // mi_pm_abs_b =
IMR*mark*|b|
let alpha = ifixed::div_up(num, denom);
```

The same fee math is used when haircuts are enabled. In `compute_liquidation_size_with_haircut`, “candidate 1” covers the case where collateral after the close is still ≤ 0 — and that candidate also sets `denom = IMR_notional - (liq_fee + ins_fee)`.

But `reduce_liquidated_position` — the function that actually settles the close — waives the insurance-fund fee whenever the resulting raw collateral is negative:

[File: packages/perpetuals/sources/clearing_house.move:4047-4054](#)

```
// Unpaid insurance-fund fees are waived in bad debt: only PnL and
// liquidation fees can create debt that IF or socialization must cover.
let insurance_fund_fee_capacity = if (
  !full_close_bad_debt &&
  ifixed::greater_than(equity_after_liquidation_fees, 0) &&
  !ifixed::is_neg(collateral_after_liquidation_fees) // requires raw
collateral >= 0
) ifixed::min(collateral_usd_after_liquidation_fees,
equity_after_liquidation_fees)
else 0; // capacity 0 when
collateral < 0
```

That waiver matters in a real, tested state: raw collateral is negative, but true equity is still positive (for example after funding losses, or after an earlier partial liquidation). In that state settlement charges no insurance-fund fee, yet sizing still subtracts that fee in the denominator. A smaller denominator makes **alpha** larger than it should be, so the protocol over-closes by roughly this fraction of the base that actually needed to be liquidated:

```
excess = insurance_fund_fee / (IMR - liquidation_fee - insurance_fund_fee)
```

Consider following example (**collateral_price** = 1):

Input	Value
Long base b	100
Quote q	500
Mark	10
Raw collateral c	-460
IMR / MMR	10% / 5%
liq_fee / ins_fee	0.5% / 0.5%

```
True equity = collateral + uPnL = -460 + 500 = 40 (> 0 → still solvent)
MMR requirement = 5% × 1000 = 50
margin = 40 < 50 → liquidatable
```

What sizing does today (assumes both fees are charged):

```
num    = 100 - 40 = 60
denom  = 100 - 10 = 90          // subtracts liq+ins = 1% of notional
alpha  = 60/90 ≈ 0.667 → closes 66.7 base
```

What settlement actually does (waives ins_fee because collateral stays < 0):

```
denom should be 100 - 5 = 95    // only liq_fee
alpha = 60/95 ≈ 0.632 → closes 63.2 base (lands at IMR)
```

Over-close $\approx 66.7 - 63.2 = 3.5$ base (~5.6% too much vs the minimal close).

With more aggressive but valid fees (ins=1%, IMR=5%, liq=1%) the excess can reach ~33%.

The liquidated trader still has positive equity, but the protocol closes more of their position than the minimum needed to restore IMR. They also pay liquidation fee on that extra size, and the extra fee goes to the permissionless liquidator ([hot_potato.liquidator_fees](#)).

This bug does not need a special attacker setup: anyone can call `liquidate<T>`, and the same oversize can repeat if the account later re-enters the negative-collateral / positive-equity region.

Recommendations:

Make the sizing consistent with the settlement fee logic: wherever post-liquidation raw collateral stays non-positive, the denominator must subtract only the liquidation fee that is actually charged. `compute_liquidation_size_with_haircut` already splits on it (candidate 1 vs candidate 2), but `compute_liquidation_size_no_haircut` does not, so it needs the same split rather than a blanket change.

In `compute_liquidation_size_no_haircut` ([clearing_house.move:4362-4386](#)), add the waived-fee candidate and keep the existing equation as the positive-collateral case. `num` is fee-independent and is shared by both:

```
    let upnl = ifixed::sub(ifixed::mul(mark_price, b), q);
    let collateral_usd = ifixed::mul(collateral_price, c);
+   let notional = ifixed::mul(mark_price, abs_b);
+   // Settlement waives the insurance-fund fee while post-close raw collateral
+   // stays negative (`reduce_liquidated_position`), so size that region
against
+   // the liquidation fee alone.
+   let liq_fee_notional = ifixed::mul(liquidation_fee, notional);
    let fee_notional = ifixed::mul(
        ifixed::add(liquidation_fee, insurance_fund_fee),
-       ifixed::mul(mark_price, abs_b),
+       notional,
    );
    let rounding_cushion =
liquidation_collateral_rounding_cushion(collateral_price);
    let num = ifixed::sub(
        ifixed::add(mi_pm_abs_b, rounding_cushion),
        ifixed::add(upnl, collateral_usd),
    );
-   let denom = ifixed::sub(
-       mi_pm_abs_b,
-       fee_notional,
-   );
+
+   // Candidate 1: post-close collateral stays <= 0, insurance fee waived.
+   let denom_waived = ifixed::sub(mi_pm_abs_b, liq_fee_notional);
+   if (ifixed::greater_than(denom_waived, 0)) {
+       let alpha_waived = ifixed::div_up(num, denom_waived);
+       let collateral_after = ifixed::add(
+           collateral_usd,
+           ifixed::mul(alpha_waived, ifixed::sub(upnl, liq_fee_notional)),
+       );
+       if (
+           valid_liquidation_alpha(alpha_waived) &&
+           !ifixed::greater_than(collateral_after, 0)
+       ) {
+           return (ifixed::mul(alpha_waived, abs_b), false)
+       };
+   };
+   };
```

```
+
+ // Candidate 2: post-close collateral is positive, both fees charged.
+ let denom = ifixed::sub(
+     mi_pm_abs_b,
+     fee_notional,
+ );
+
+ // Only a fraction in [0, 1] represents a valid partial liquidation.
+ // Anything outside that range falls back to full liquidation.
+ let alpha = ifixed::div_up(num, denom);
```

In `compute_liquidation_size_with_haircut` (`clearing_house.move:4440-4485`), candidate 1 is already the non-positive-collateral region, so it only needs the fee swapped. Its acceptance test projects `collateral_after` from `collateral_delta`, which must use the same waived fee or the test will judge the candidate against a charge settlement never makes:

```
    let liq_fee_notional = ifixed::mul(liquidation_fee, ifixed::mul(mark_price,
abs_b));
    let collateral_delta = ifixed::sub(upnl, fee_notional);
+   let collateral_delta_waived = ifixed::sub(upnl, liq_fee_notional);
    let mut alpha = 0;
    let mut has_alpha = false;

    // Candidate 1: final collateral remains non-positive.
-   let no_haircut_denom = ifixed::sub(mi_pm_abs_b, fee_notional);
+   let no_haircut_denom = ifixed::sub(mi_pm_abs_b, liq_fee_notional);
    if (ifixed::greater_than(no_haircut_denom, 0)) {
        ...
        let collateral_after = ifixed::add(
            collateral_usd,
-            ifixed::mul(no_haircut_alpha, collateral_delta)
+            ifixed::mul(no_haircut_alpha, collateral_delta_waived)
        );
```

Candidate 2 of the haircut branch needs no change: it is accepted only when `collateral_after >= 0`, where settlement does charge both fees, so `fee_notional` and `collateral_delta` stay as they are.

Customer Response:

Fixed in commit [67c846f](#) . Confirmed in both sizing functions. The region where post-close raw collateral stays non-positive is now solved by a shared waived_fee_liquidation_alpha, which subtracts only the liquidation fee, projects the resulting collateral, and reports whether that projection is indeed non-positive. `compute_liquidation_size_no_haircut` calls it before its existing equation and returns directly when the region matches; `compute_liquidation_size_with_haircut` uses it in place of its candidate 1. The positive-collateral candidate is unchanged in both, because settlement does charge both fees there.

The projection matches settlement: `reduce_liquidated_position` computes `collateral_after_liquidation_fees` as `position.collateral() + (pnl_to_realize - liquidation_fees) / collateral_price`, which in USD terms is `collateral_usd + alpha * (upnl - liq_fee_notional)`.

One mismatch remains unaddressed. In the positive-collateral region, settlement caps the insurance-fund fee at `min(collateral_usd_after_liquidation_fees, equity_after_liquidation_fees)` rather than charging it in full, so where that cap binds, candidate 2 still assumes a larger fee than is charged.

Fix Review:

Fix confirmed.

M-23: Tightening the book-spread limit leaves the liquidation mark at the obsolete wider bound, enabling permissionless liquidation of healthy positions

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/market.move	Status: Acknowledged	

Description:

`set_risk_limit_params` overwrites `max_book_index_spread` in place. It validates the new bound and stores it, but never checkpoints or rebases the running `spread_twap` / `premium_twap` that were accumulated under the old, wider bound.

[File: packages/perpetuals/sources/market.move:684-689](#)

```
params.limits_params.min_order_usd_value = min_order_usd_value;
params.limits_params.max_pending_orders = max_pending_orders;
params.limits_params.max_open_interest = max_open_interest;
params.limits_params.max_open_interest_threshold = max_open_interest_threshold;
params.limits_params.max_open_interest_position_percent =
max_open_interest_position_percent;
params.limits_params.max_book_index_spread = max_book_index_spread;
```

New observations are clipped to the new limit before entering the TWAP, but `update_twap` blends each clipped sample with the obsolete stored value, so until a full TWAP period elapses the stored TWAP stays outside the newly-declared band:

[File: packages/perpetuals/sources/market.move:996-1008](#)

```
let max_spread_abs = ifixed::mul(index,
params.limits_params.max_book_index_spread);
let max_book = ifixed::add(index, max_spread_abs);
let min_book = ifixed::sub(index, max_spread_abs);
// ... clips the *new* sample only; the historical spread_twap is unchanged
```

The session mark is a median of `index_twap` + `spread_twap`, the funding price (from `premium_twap`), and the raw, unclipped order-book midpoint:

[File: packages/perpetuals/sources/market.move:1494-1502](#)

```
let twap_price = ifixed::add(index_twap_price, spread_twap(market_state));
ifixed::max(
  ifixed::min(twap_price, funding_price),
  ifixed::min(ifixed::max(twap_price, funding_price), book_price) // raw book,
  not clipped
)
```

Being a median is normally the defence: a single leg outside the band is outvoted by the other two. The tightening removes that. After a 5% → 1% change, the stale `spread_twap` leg already sits at ~+5%, so an attacker only has to bring one further leg up to that level rather than two. Moving the raw book mid to ~+5% and starting a liquidation session makes those two the matching median legs, and the mark holds ~5% above index instead of ≤1%. A short that is healthy under the new 1% limit fails MMR and is liquidated permissionlessly; the attacker receives the liquidation position and the liquidation fee, which `execute_liquidation` assigns to the caller through `hot_potato.liquidator_fees`.

Moving the book mid is the part that costs something, and it is what keeps likelihood low. `book_price` is $(\text{best_ask} + \text{best_bid}) / 2$ and returns `none` when either side is empty, in which case the index is used, so the attacker has to own both sides of the top of book. An ask at +5% rests without crossing, but a bid at +5% sits above the best ask on any liquid two-sided book and will sweep every ask below it instead of resting. The cheap version of this attack therefore needs a thin or one-sided book around the obsolete bound; on a liquid book the attacker first has to buy out the ask side, which costs real capital and leaves them long.

Where the book can be moved, the rest is atomic and close to riskless. `liquidate` must be the first operation in its session, but a single PTB can hold several sessions and `start_session_` recomputes the mark from the live book on each one, so posting the orders, liquidating, and cancelling can run as three sessions in one transaction with no window for anyone to fill the attacker's quotes.

On some configurations no book move is needed at all, because the funding leg carries a stale value too. `calculate_funding_price` (`market.move:940`) returns $\text{index} + \text{premium_twap} * \text{alpha}$, where `alpha` is the fraction of the funding period still to run, and `assert_funding_parameters`

(`market.move:1603`) requires `funding_period_ms` to be an exact multiple of `funding_frequency_ms` greater than one, so `alpha` is at most $\text{funding_frequency_ms} / \text{funding_period_ms}$ and never exceeds one half. With the ratio at its minimum of two and a stale premium of ~5%, the funding leg sits at ~+2.5%, which is itself outside a newly-declared 1% band. Legs one and two are then both out of band on their own and the median holds at ~+2.5% with no attacker participation, leaving only the `liquidate` call to make. The wider the funding period is relative to its frequency, the more this leg is damped: at the common eight-to-one ratio it contributes ~+0.6% and the attack does need the book.

Recommendations:

Fix the issue or consider giving a tightening of ``max_book_index_spread`` the same proposal-and-commit delay that margin-ratio changes already have. The parameter takes effect immediately today, so the window opens the instant the operator acts and no one has a chance to reduce exposure first.

Customer Response:

Acknowledged. The project does not expect vendors in this version of the contracts so we preferred to leave the haircut as a vendor parameter to save on package size.

M-24: Dust-sized base oracle prices let attackers realize inflated short PnL and drain vault collateral

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/oracle-aggregator/sources/price_feed.move packages/perpetuals/sources/market.move packages/position/sources/position.move packages/perpetuals/sources/clearing_house.move	Status: Acknowledged	

Description:

The oracle feed validates only that prices are not signed-negative; it does not enforce a positive lower bound. `price_feed::set_price` and `price_feed::new` both call `assert_is_positive!`, whose implementation rejects only `is_neg!(price)` (`price_feed.move:154, :63, :270-274`). A value of 0 passes that check, and any positive dust value also passes. The public relay `price_feed_storage::update_price_feed` forwards the source-supplied price after checking only package version, feed existence, the whitelisted source object, and timestamp monotonicity (`price_feed_storage.move:523-551`).

The perps base-oracle helper then reads the raw spot and TWAP with only a freshness check:

```
// market.move:1526-1534
let feed = price_feed_storage::price_feed($oracle, $source_id);
let (price, timestamp_ms) = price_feed::price_and_timestamp_ms(feed);
let twap_price = price_feed::twap_price(feed);

let now_ms = clock::timestamp_ms($clock);

abort_if!(now_ms - now_ms.min($oracle_tolerance) > timestamp_ms,
bad_index_price!());

(price as u256, twap_price as u256)
```

The exact-zero case is not the exploitable withdrawal path: `deallocate_collateral_internal` calls `try_update_fundings_and_twaps` before computing free collateral, and that calls `clip_max_book_index_spread`, which aborts on `index == 0` (`clearing_house.move:4489-4512`,

`market.move:870-879`, `market.move:989-995`). However, a strictly positive dust price is not blocked by this guard.

If both base spot and base TWAP are fresh dust values, the divergence check can pass because the absolute delta is within `index_twap_price * max_index_twap_divergence` (`market.move:1011-1027`). The mark used for withdrawal then collapses toward zero: `calculate_mark_price` derives the funding and TWAP legs from `index_twap_price`, and an empty or thin book can fall back to or be clipped around the dust index (`market.move:1478-1501`, `orderbook.move:162-167`, `market.move:989-1009`).

At a dust mark `p`, a short's margin requirement also becomes dust because it is multiplication-only:

```
// position.move:899-908
let abs_net_base = abs_net_base(position);
let abs_net_base_value = ifixed::mul(abs_net_base, mark_price);

ifixed::mul(abs_net_base_value, margin_ratio)
```

For a short with base $-B$, entry notional $B * P_{\text{entry}}$, collateral value C , and dust mark $p > 0$, `unrealized_pnl =  $B * (P_{\text{entry}} - p)$` is positive. `compute_free_collateral` deliberately excludes positive uPnL from withdrawable margin by setting `capped_margin = collateral_usd`, but the minimum margin is only $B * p * \text{margin_ratio}$; therefore `free_collateral =  $C - B * p * \text{margin\_ratio}$` , which approaches the entire posted collateral as p approaches zero (`position.move:564-605`). `deallocate_free_collateral` then pays that amount from the shared vault to the account (`clearing_house.move:1961-1977`, `:3505-3583`).

The short keeps the position open with almost no collateral remaining. When the base price recovers to a normal value, the same short has large negative PnL with insufficient backing, creating bad debt for the insurance fund or socialization. The exploit is permissionless at the perps layer once the feed reads a dust base price; practical likelihood depends on whether the whitelisted upstream oracle source/integration can publish a fresh dust spot and TWAP.

The impact is not limited to deferred bad debt: a dust mark also enables direct extraction of vault funds. A dust mark inflates a short's unrealized PnL to roughly `size *  $P_{\text{entry}}$` , and `compute_free_collateral` deliberately excludes positive uPnL from withdrawable margin (`position.move:588-592`), so a direct withdrawal only recovers the attacker's own collateral. But an attacker using two of its own accounts can crystallize that inflated uPnL into realized,

withdrawable PnL. The self-trade guard is keyed only on `account_id` (`clearing_house.move:3246-3248`), so account B can post an ask at the dust price and account A (short from `P_entry`) buys it to close; realized PnL is computed from the fill quote, not the mark (`position.move:307-323`), so A realizes about `size * (P_entry - p)` and, with `base == 0`, withdraws it from the commingled vault (`position.move:571-575`). Account B's offsetting dust-entry short passes the maker check because that check reserves only liquidation fees (`abs_base * mark_price * liquidation_fee`, `position.move:766-779`), which is dust at a dust mark — it does not enforce initial or maintenance margin. When the base price recovers, B is deeply undercollateralized and its loss becomes bad debt. Net, the attacker extracts other users' collateral/insurance fund now (bounded by vault liquidity and by `min_order_usd_value = size * index_price`, `clearing_house.move:5017-5025`, which forces a larger `size` at a dust index), leaving the protocol with the matching loss. This chains the missing price floor with the maker-side margin gap and the cross-account self-trade path.

Recommendations:

Reject zero and dust-magnitude prices at the oracle layer. `price_feed::new` and `price_feed::set_price` should require `price` to fall inside a configured per-asset lower and upper bound and abort the update when it does not.

Reject rather than clamp. Pinning an out-of-range price to the bound and carrying on is the Chainlink `minAnswer/maxAnswer` failure mode that kept protocols valuing assets at a stale bound through the LUNA crash and taking on bad debt.

An absolute bound is the primitive that is missing, and no existing guard substitutes for it. `assert_index_twap_divergence_within_limit` is purely relative, `max_allowed_delta = index_twap_price * max_index_twap_divergence`, so its accepted range shrinks with the price and vanishes in the dust regime. That is precisely why this bug passes it.

Customer Response:

Acknowledged with no changes. Oracle aggregator enforces an index twap bound. If the price drops from realistic to dust then the twap bound would fire, preventing the market from using that price. It will only use dust price if the price dropped slowly enough, likewise on the rebound. And the liquidation/adl system is in place to intervene in that case, preventing the loss from compounding.

M-25: Vault reads the collateral-oracle price with no strictly-positive guard, so a zero collateral price freezes every LP entry and exit

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/market-making-vault/sources/vault.move	Status: Fixed	

Description:

The vault helper `get_price` performs ONLY a staleness assert and returns the raw feed value, with no `price > 0` check:

[File: packages/market-making-vault/sources/vault.move:2778-2791](#)

```
assert!(now_ms - min(now_ms, oracle_tolerance) <= timestamp_ms,
errors::bad_oracle_price());
```

```
price as u256
```

Zero is a reachable input: the oracle aggregator's positivity checks are written with `is_neg!` / `!get_is_negative`, so `price == 0` passes ingestion (the same root as confirmed known finding M-07, but this is the vault's own separate read of the collateral feed).

The zero then flows straight into divisors, and Move aborts on divide-by-zero. Normal withdraw:

[File: packages/market-making-vault/sources/vault.move:2924-2926](#)

```
ifixed::div(value_to_withdraw, collateral_price),
```

Force-withdraw:

[File: packages/market-making-vault/sources/vault.move:2563-2565](#)

```
ifixed::div(amount_to_withdraw, withdraw_session.collateral_price),
```

Deposit, where a zero price collapses the value and trips the non-zero-mint assert:

[File: packages/market-making-vault/sources/vault.move:1638-1639](#)

```
assert!(lp_to_mint_b9 != 0, errors::user_lp_calculation_zero());
```

a single zero collateral print aborts EVERY vault entry and exit simultaneously — including the trustless force-withdraw exit, which is the only keeper/admin-independent path for an LP to recover collateral tied up as clearing-house margin.

Recommendations:

Assert strict positivity at the vault sink: `assert!(price_u256 != 0, errors::bad_oracle_price())` inside `get_price`. Ideally also value collateral at `min(spot, twap)` with a divergence bound, mirroring the base-oracle path, so a single bad print cannot drive the collateral value to zero.

Customer Response:

Fixed in oracle-aggregator [d96b128](#) and commit [496b8ca](#). `price_feed::new` aborts on `price == 0` (error `EInvalidPrice`); `maybe_set_price` / `update_price_feed` drop zero updates as no-ops and keep the last non-zero quote. Vault `get_price` is unchanged (still no local positivity assert) — the ingest path is hardened so a stored collateral price cannot be zeroed by a bad print.

Fix Review:

Fix confirmed.

M-26: Funding/TWAP parameter changes are applied retroactively to un-accrued funding and to the liquidation mark

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/market.move	Status: Fixed	

Description:

`set_twap_params` overwrites `funding_frequency_ms`, `funding_period_ms`, and the premium/spread TWAP periods, with no prior `try_update_fundings` or TWAP checkpoint. Because the funding grid is not settled first, the new values apply retroactively to the un-accrued window since `funding_last_upd_ms`:

[File: packages/perpetuals/sources/market.move:574-579](#)

```
params.twap_params.funding_frequency_ms = funding_frequency_ms;
params.twap_params.funding_period_ms = funding_period_ms;
params.twap_params.premium_twap_frequency_ms = premium_twap_frequency_ms;
params.twap_params.premium_twap_period_ms = premium_twap_period_ms;
params.twap_params.spread_twap_frequency_ms = spread_twap_frequency_ms;
params.twap_params.spread_twap_period_ms = spread_twap_period_ms;
```

1. **Grid reinterpretation.** Intervals are counted as $(\text{now}/\text{freq}) - (\text{last}/\text{freq})$, so a timestamp recorded under the old frequency but divided by the new frequency can skip or manufacture a due interval:

[File: packages/perpetuals/sources/market.move:1247](#)

```
let intervals = (now / funding_frequency_ms) - (funding_last_upd_ms /
funding_frequency_ms);
```

1. **Unsettled window is re-priced.** The next funding charge is $\text{premium_twap} \times (\text{intervals} \times \text{freq}/\text{period})$. Shortening `period` multiplies that next charge. History already written into `cum_funding_rate_*` is left alone; only the open window that has not yet been settled is re-priced under the new terms:

[File: packages/perpetuals/sources/market.move:1043](#)

```
let funding_rate = ifixed::mul(state.premium_twap, period_adjustment);
```

1. **Immediate liquidation-mark shift.** `calculate_funding_price` uses `alpha = time_until_next_funding / funding_period_ms` over `premium_twap`, and its output is a median leg of the mark ([market.move:1482-1505](#)):

[File: packages/perpetuals/sources/market.move:944-949](#)

```
let time_until_next_funding = max(now, next_settlement_time) - now;
let alpha = ifixed::from_u64fraction(time_until_next_funding,
market_params.twap_params.funding_period_ms);
ifixed::add(index_price, ifixed::mul(funding_premium, alpha))
```

`set_twap_params` only overwrites the parameter fields. It never advances or resets the last-update clocks (`funding_last_upd_ms`, `premium_twap_last_upd_ms`, `spread_twap_last_upd_ms`), so the unfinished funding/TWAP window still measures from the pre-change timestamp. Those three timestamps are updated only when funding actually accrues, or a TWAP sample is taken — never in the setter itself ([market.move:1052;1079;1107](#)).

The following example shows the size of the defect. Funding adjustment is $(\text{intervals} \times \text{freq}) / \text{period}$. Values below still pass `assert_funding_parameters` under registry defaults (min frequency 1 min, min period 6 h, max period 10 days — [registry.move:297-299](#)).

Setup. Frequency is 1 hour. One hour has already passed since the last funding update, so the next settlement will count exactly one interval. Premium is at the 5% clip. Vendor shortens `funding_period_ms` from **10 days** → **6 hours**, without settling first. Someone then calls permissionless `update_funding`.

Effect on funding. That same unsettled hour was going to charge $1\text{h} / 10\text{d} = 1/240$ of the premium, i.e. about **0.02% of notional**. Under the new period, it charges $1\text{h} / 6\text{h} = 1/6$ of the premium — **40× more**, about **0.83% of notional** — even though market time and premium did not change. Longs and shorts reprice each other on the already-open window as if the new terms had always applied.

Effect on mark (same change, no funding call needed). `funding_price` uses `alpha = time_until_next_funding / period`, so the same cut also moves the funding mark leg from roughly

`index + 0.01%` to `index + 0.42%` immediately. The session mark is a median of three legs, so the final mark only moves as far as the TWAP and book legs allow.

The setter needs no clock or oracle, and a subsequent funding update is permissionless ([clearing_house.move:1696](#)), so the param change and the next funding update can run in the same PTB and permanently apply the new rate to the still-open funding window before users can react. Note also that `set_twap_params` has no pause or closed-market guard, unlike its sibling setters.

Recommendations:

Before mutating any funding/TWAP param, checkpoint funding and both TWAPs under the old params and set the last-update timestamps to `now` (mirror `close_market` at [clearing_house.move:1183-1187](#)). Note the wrapper currently takes no `Clock`, so this changes a public signature — the same breaking change `close_market` itself took. Alternatively, schedule the change for a future funding boundary with an effective timestamp.

Add `assert_market_is_not_paused` for consistency with the other setters, and a test asserting that changing `funding_frequency_ms` / `funding_period_ms` neither skips nor duplicates an interval for an already-open position.

Customer Response:

Fixed in commit [dba11e4](#) . Updated the function to update fundings with current parameters before updating to the new ones.

Fix Review:

Fix confirmed.

M-27: Completing any force-withdraw clears the vault's force-withdraw pause unconditionally, letting the owner repeatedly censor the trustless exit

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: packages/market-making-vault/sources/vault.move	Status: Fixed	

Description:

`process_clearing_house_for_force_withdraw` aborts with `wrong_order_ids_in_force_withdraw` unless the caller-supplied `order_ids` covers every pending order the vault position holds at execution time:

[File: packages/market-making-vault/sources/vault.move:2442-2452](#)

```
// Check that order_ids contained all the order ids for this position.
let position = clearing_house.position(account_id);
let (base_amount, _quote_amount) = position.base_and_quote_amounts();
let (pending_asks, pending_bids) = position.pending_base_amounts_by_side();
let pending_orders = position.pending_order_count();
assert!(
    pending_asks == 0 &&
    pending_bids == 0 &&
    pending_orders == 0,
    errors::wrong_order_ids_in_force_withdraw()
);
```

An LP building a force-withdraw transaction must therefore snapshot the vault's live order set off-chain, and the only thing that makes that snapshot usable is `pause_vault_for_force_withdraw`, which sets `vault.paused` for `config.force_withdraw_pause_ms` (default 10 s) and gates vault trading entry points via `assert_vault_is_not_paused`. That pause is the anti-front-run mechanism for the keeper-independent exit.

Two properties combine into a censorship primitive.

First, the pause is not bound to whoever obtained it. `end_withdraw_session_` clears it for **any** completed force-withdraw:

[File: packages/market-making-vault/sources/vault.move:2982-2993](#)

```
if (can_force_process) {
  events::emit_user_force_withdraw(...);

  // Unlock the `Vault` now that the Force withdraw has been processed.
  vault.paused = option::none();
}
```

This ignores which withdraw request caused the pause and whether the 10-second window has elapsed. Completing a force-withdraw clear the pause early, and that path is unguarded. Force-withdraw itself checks only the admin pause — never the temporary one — so the owner can run a force-withdraw during an LP's live pause:

[File: packages/market-making-vault/sources/vault.move:2270-2272](#)

```
withdraw_session.vault.assert_package_version();
withdraw_session.vault.assert_vault_is_not_admin_paused();
assert!(withdraw_session.can_force_process, errors::cannot_force_withdraw());
```

Second, the cooldown is per-vault. Default `min_pause_vault_for_force_withdraw_frequency_ms` = 1800000 (30 minutes); `pause_vault_for_force_withdraw` checks and writes a single `vault.last_paused_timestamp_ms`:

```
assert!(
  vault.last_paused_timestamp_ms
    + vault.vault_params.min_pause_vault_for_force_withdraw_frequency_ms
    <= current_timestamp_ms,
  errors::pause_vault_for_force_withdraw_too_frequent()
);
vault.last_paused_timestamp_ms = current_timestamp_ms;
```

Consider the scenario as follow. The owner pre-stages a dust deposit at a throwaway address, waits out `lock_period`, files a withdraw request, and waits out `force_withdraw_delay`. When LP A calls `pause_vault_for_force_withdraw`, burning the vault-global 30-minute budget, the owner immediately runs `start_force_withdraw_session` → `process_*` → `end_withdraw_session` for the dust request, which sets `paused = none` seconds later, and re-replaces orders in the same transaction. A's transaction — built against the pre-pause order set — now aborts on `wrong_order_ids_in_force_withdraw`, and A cannot re-pause until 30 minutes later. The owner

repeats with a fresh address each round. Two honest LPs also cannot obtain independent windows at all, since there is one 10-second window per vault per 30 minutes.

Recommendations:

Preserve each temporary force-withdraw pause until its configured deadline. Completing a force-withdraw must not clear the pause early. While the pause is active, vault trading remains disabled, but any mature force-withdraw request may be processed. Existing timestamp checks should automatically restore normal vault operation after the deadline, while the vault-global cooldown remains in place to limit aggregate trading downtime.

Customer Response:

Fixed in commit [e5fa9f9](#) . Removed the vault unpause from force withdrawal as suggested.

Fix Review:

Fix confirmed.

M-28: A fill resets the consecutive zero-fill cap, leaving total limit-order traversal dependent on taker size

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/utils/constants.move	Status: Fixed	

Description:

`execute_limit_order` limits only a consecutive run of makers that fill nothing. Any positive maker fill resets the counter:

```
if (taker_size_matched == 0) {
    zero_fill_matches = zero_fill_matches + 1;
    if (zero_fill_matches == constants::max_zero_fill_matches!()) {
        zero_fill_cap_reached = true;
        break
    };
} else {
    zero_fill_matches = 0;
};
```

The bound is 200 ([constants.move:108](#)), but there is no independent bound on total makers inspected by one limit order ([clearing_house.move:3075](#), [clearing_house.move:3155](#)).

An attacker can place repeating same-price FIFO blocks of 199 soon-to-expire or otherwise zero-fill makers followed by one solvent one-lot maker. A taker of `S` lots then inspects `200S` entries: each one-lot fill resets the counter before the next 199-order run reaches the cap. Every maker inspection borrows and settles the maker position, and live makers also run the solvency preview ([clearing_house.move:3421](#), [clearing_house.move:3452](#), [clearing_house.move:3516](#)).

If traversal exceeds the transaction computation budget, Sui aborts atomically before the later `batch_drop` cleanup at [clearing_house.move:3196](#). The fills and inspected-order removals therefore roll back, and retrying the same large atomic order makes no progress.

The existing [a_filling_maker_restarts_the_zero_fill_run](#) test directly confirms the reset behavior: a two-lot IOC order traverses 100 expired makers, fills one live maker, then traverses another 101 expired makers, removing more than 200 zero-fill makers overall ([test_orderbook_limit_order.move:571](#)). The companion negative test confirms that 201 consecutive zero fills stop at 200 and commit the removal of that prefix ([test_orderbook_limit_order.move:490](#)). Both tests pass on commit [dba11e4](#).

The attack blocks sufficiently large crossing limit orders, not every order above one lot. Account creation is permissionless, but each position is limited to 100 pending orders, resting orders must meet lot and minimum-value checks, and reset makers incur real fills, fees, inventory, and price risk. Users can also make committed progress by splitting execution into smaller IOC limit orders; each successful transaction consumes the inspected prefix. The result is a repeated-cost availability attack and forced transaction chunking, not theft, protocol insolvency, or a permanent fund freeze.

This finding is a residual of the M-14 / expired-order cleanup DoS remediation: the remediation bounds consecutive zero fills but not total transaction work. It is distinct from the separate uncapped market-order path.

Recommendations:

Add a transaction-wide monotonic counter for all inspected maker entries and stop successfully when either the total-inspection cap or the consecutive-zero-fill cap is reached. Reaching the total cap should follow the existing safe limit-order behavior: discard the taker remainder as necessary and execute [batch_drop](#) so inspected stale makers are committed rather than rolled back.

Choose the total cap from measured worst-case per-maker computation with headroom for maker/taker settlement and event processing. Add regression tests for repeated [199 zero-fill + 1 one-lot fill](#) blocks, asserting that total inspected makers never exceed the new bound and that a subsequent IOC order resumes from the shortened book.

Customer Response:

Fixed in commit [31c584d](#).

Fix Review:

Fix confirmed.

M-29: Changing the funding frequency before the old boundary can charge intervals that exist only on the new grid

Severity: Medium	Impact: Medium	Likelihood: Low
Files: perpetuals/sources/clearing_house.move	Status: Acknowledged	

Description:

`set_twap_params` is meant to settle funding with the old parameters and rebase `funding_last_upd_ms` to the current time before it stores new parameters. The code comment states this rule, and the function calls `try_update_fundings` before changing the parameters ([clearing_house.move](#)).

```
clearing_house.market_params.try_update_fundings(  
    &mut clearing_house.market_state,  
    clock.timestamp_ms(),  
    &ch_id,  
);
```

However, `try_update_fundings` returns before it writes `funding_last_upd_ms` when the next boundary under the old frequency has not been reached ([market.move:1027](#)). The parameter setter then stores the new frequency while the timestamp is still based on the old grid ([market.move:525](#)).

```
if (!is_time_to_update(  
    now, state.funding_last_upd_ms, params.twap_params.funding_frequency_ms  
)) return;
```

The next funding update divides both timestamps by the new frequency. It limits the result to three intervals, but it does not check whether those intervals existed under the old frequency ([market.move:1241](#)).

```
let intervals = (now / funding_frequency_ms)
```

```
- (funding_last_upd_ms / funding_frequency_ms);  
let intervals = if (intervals > constants::max_funding_accrual_intervals!()) {  
    constants::max_funding_accrual_intervals!()  
} else {  
    intervals  
};
```

For example, assume the old frequency is one hour, `funding_last_upd_ms = T` is on that grid, and the setter runs at `T + 10 minutes`. Nothing is due under the old frequency, so `try_update_fundings` returns and leaves the timestamp unchanged. If the frequency is changed to one minute, the next update counts ten intervals on the new grid and charges the maximum of three.

The extra rate is added to the cumulative funding indices ([market.move:1037](#)). Position settlement multiplies the index change by the position's signed base size, so one side loses collateral and the other side receives it ([position.move:1040](#)). The wrong debit can also reduce a position's margin.

The call requires a valid vendor `ADMIN`, authorized `ASSISTANT`, or authorized `MAINTENANCE` capability ([clearing_house.move:1320](#)). This makes the likelihood Low. The impact is still a contract accounting error because both the old and new frequencies can be valid, but the transition creates funding that was not due under the old configuration. This is a remaining case from the M-26 fix.

Recommendations:

Review the remediation for M-26 together with this residual finding and implement one coherent parameter-transition policy. The project may choose the implementation, but the resulting code should satisfy all of the following requirements:

- A funding or TWAP parameter change must have one explicit, atomic effective time. Validate the complete proposed parameter set before changing funding indices, timestamps, accumulators, or stored parameters. If the transaction aborts, none of those values should change.
- Funding attributable to time before the effective time must be determined only by the old funding frequency, old funding period, and the premium state applicable to that window. Funding attributable to time after the effective time must be determined only by the new

parameters. A timestamp recorded under one funding grid must never be reinterpreted as if it had been recorded under another grid.

- The transition must neither skip, duplicate, manufacture, nor retroactively reprice funding time. This must hold for a period-only change, both shorter and longer frequencies, non-aligned old and new grids, and changes immediately before or after either grid's boundary.
- Preserve the intended funding-accrual cap. A parameter change must not bypass, multiply, or reset the cap, and the cap must not cause the same elapsed time to be counted again under the new parameters.
- Keep the cumulative funding indices, `funding_last_upd_ms`, the next funding boundary, and the funding-price leg of the liquidation mark mutually consistent. The mark should not price an old portion of the window under new terms, and the first funding settlement after the transition should agree with the time weighting represented by the mark.
- Define an explicit transition for the premium and spread TWAP windows covered by M-26. For example, the implementation may finalize the old windows before activating the new parameters or schedule the change at a safe future boundary. It should not silently combine an accumulator and elapsed window created under the old TWAP parameters with incompatible new weighting. Do not discard accumulated observations merely by resetting timestamps; preserve or settle their economic effect according to the selected policy.
- Preserve the safety property of ordinary permissionless funding updates: calls made before a normal funding boundary must not erase a partial funding window. A fix for the administrative transition must not introduce a public way to suppress funding through repeated early calls.
- Apply the existing market-lifecycle policy consistently. In particular, funding/TWAP parameters must not be mutable while the market is paused unless the protocol deliberately defines and tests safe paused-market transition semantics.
- Treat omitted values and values equal to the current configuration as no funding-parameter change. A TWAP-only update should not unintentionally settle, reset, or rebase funding state.

The implementation can use checkpointing, a scheduled parameter change, a dedicated transition path, restrictions on when live parameters may change, or another design that proves

these properties. The audit recommendation does not require a specific helper, formula, or refactor.

Add regression tests that exercise period-only changes, frequency shortening and lengthening, non-aligned grids, changes on both sides of old and new boundaries, the accrual cap, the first post-change funding update, funding-mark consistency, TWAP-window transitions, no-op and TWAP-only updates, paused markets, and repeated permissionless updates before a boundary. Tests should assert cumulative funding values and all relevant timestamps, rather than checking only whether the setter succeeds.

Customer Response:

Acknowledged with no changes. In case we want to change funding parameters we will plan to do that immediately after a funding update, so that the impact is completely negligible. We already did that during v1 and will continue to try to minimize the impact.

M-30: Passive maker fills bypass post-fill margin and direction controls

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/position/sources/position.move packages/perpetuals/sources/clearing_house.move	Status: Acknowledged	

Description:

When a resting maker order is filled, `apply_maker_fill_or_restore_if_bad_debt` applies the exact fill and fees. It then checks only three conditions: equity is not negative, the position can pay the liquidation fee, and the maker concentration cap is not exceeded. It does not check IMR, MMR, or whether the fill changes the position from long to short or from short to long ([position.move:723](#)).

[Code location](#)

```
let margin_after = ifixed::add(collateral_usd_after, pnl_after);

if (macros::is_neg!(margin_after)) return 'solvency;
if (abs_base * mark_price / ifixed_constants::one!()
    * liquidation_fee / ifixed_constants::one!()
    > margin_after
) return 'solvency;
```

This differs from the taker path. At the end of a taker session, the protocol computes the IMR requirement and calls `ensure_margin_requirements` ([clearing_house.move:2874](#)). If the account remains below IMR, that function allows only a non-flipping action that does not increase exposure or worsen the margin ratio ([position.move:841](#)). This end-session check runs for the taker, not for a passive maker filled during another account's session.

The missing check has two effects with the same root cause.

First, a maker can open exposure below MMR. For example, take a flat maker with \$10 collateral, mark \$100, IMR 10%, MMR 5%, liquidation fee 1%, and a resting bid for one base at \$109. After the fill, the exact \$109 execution price produces -\$9 unrealized PnL at the \$100 mark, leaving \$1 margin. The maker guard accepts equality with the \$1 liquidation-fee reserve, although maintenance margin is \$5. The position commits in an immediately liquidatable state. This unsafe

interval exists for every valid market configuration because the protocol requires `liquidation_fee + insurance_fund_fee < MMR` ([market.move:1706](#)). If the liquidation fee is zero, the same path can accept a non-zero position at exactly zero equity.

Second, a maker already below IMR can change direction without returning to IMR. Take a long 1 maker with \$10 collateral and a \$100 entry. After the mark falls to \$96, its margin is \$6: above the \$4.80 MMR requirement but below the \$9.60 IMR requirement. The maker can post an ask for 2. Pending-risk accounting still reports an absolute exposure of 1 because $\max(|1|, |1 - 2|) = 1$ ([position.move:911](#)). At posting time, the settled base is still long 1, so the taker-style `did_not_flip` check sees no direction change. A later passive fill at \$96 changes the maker to short 1 with the same \$6 margin. The maker remains above MMR but starts the new short exposure below IMR. The exact fill is accepted because the maker guard does not run `did_not_flip` ([clearing_house.move:3570](#)).

The matched pending amount is removed only after the maker preview succeeds ([clearing_house.move:3670](#)). Therefore, a post-fill requirement computed inside the current preview without adjusting the pending amount would use the wrong final exposure. This detail is important for a fix that must cover both cases.

Existing tests confirm the guard boundaries. `rejects_bad_debt_and_allows_solvent_bid` rejects negative margin but accepts zero margin when the liquidation fee is zero, while `rejects_fill_below_liquidation_fee_reserve` rejects margin below the reserve and accepts equality ([test_maker_margin_check.move:69](#), [test_maker_margin_check.move:128](#)). The direction-flip example also uses the tested state in which a maker can post a risk-reducing order while at a 6.25% margin ratio ([test_single_limit_order_session.move:1118](#)).

The fill does not create bad debt immediately at the same mark because negative equity is rejected. Unaffordable insurance-fund fees are also waived during liquidation ([clearing_house.move:4137](#)). However, the maker starts with less protection than the protocol requires. A small adverse mark or funding change, together with liquidation delay, can create bad debt that reaches the insurance or loss-sharing process. This makes the issue Medium: the potential loss is high, but protocol loss requires another adverse condition.

The execution price is accounted for correctly. `process_fill_maker` reads the maker order price, and `fill_base_and_quote_deltas` includes it in `quote_asset_delta` ([clearing_house.move:3534](#), [clearing_house.move:4774](#)). The bug is the missing post-fill risk policy, not execution-price accounting.

Recommendations:

Apply the same post-action margin transition policy to passive makers that is used for takers. Evaluate the maker's settled state before the fill and its final state after both the fill and removal of the matched pending amount.

A fill that opens, increases, or flips exposure should leave the maker at or above its effective IMR. If the maker was already below IMR, allow only a risk-reducing fill that does not flip the base sign and does not worsen its margin ratio. Compare the settled `base_before` and `base_after`; `abs_net_base` alone is not enough because equal absolute exposure can hide a direction change. Keep the non-negative-equity, liquidation-fee-reserve, and concentration checks as additional conditions.

Do not replace mark-price valuation with the maker's execution price. The exact execution price is already accounted for in quote/PnL, while the mark is needed to measure the resulting position's current equity.

Add regression tests for an opening maker fill that lands between the liquidation-fee reserve and MMR; equality at the liquidation-fee boundary; zero liquidation fee with zero post-fill equity; a valid risk-reducing maker fill below IMR; and a long `b` maker below IMR whose ask `2b` would flip it to short `b`. The last case must be rejected even when the final absolute exposure is unchanged and the final margin remains above MMR. Also test full and partial fills so the requirement uses the correct remaining pending quantity.

Customer Response:

Acknowledged with no changes. The maker guard is solvency-only by design. We are fine with the flipping position situation in case of makers. We were actually thinking of allowing it for takers too, but we prefer to just stay as we are now and make things more consistent in our v3.

M-31: Full-risk trading during closed equity sessions can externalize reopening-gap losses

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/integrations/pyth-lazer/sources/ price_feed_storage.move packages/integrations/pyth-lazer-rolling/sources/ price_feed_storage.move packages/oracle-aggregator/sources/price_feed.move packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/market.move	Status: Acknowledged	

Description:

The project confirmed that users may place orders while the Pyth market session is **closed**, and that the affected equity markets use Pyth Lazer and Pyth Lazer Rolling. The project also confirmed that the book and mark prices may move during this period, while the oracle index stays stale and **max_book_index_spread** limits the mark's movement around that stale index.

This means the stale-price behavior is intended. The issue is that users can still open full-risk isolated positions during a known oracle blackout, but there is no matching control for the price gap when the real index starts updating again.

For non-rolling Pyth Lazer, a signed closed-session payload uses the message timestamp instead of the time when the price was generated ([price_feed_storage.move:388](#)). Pyth Lazer Rolling applies the same policy ([price_feed_storage.move:411](#)).

```
//
https://github.com/AftermathFinance/move-oracle-aggregator/blob/46d1e0a555f021829ad4a9429e702d4c38ea0f99/packages/integrations/pyth-lazer/sources/price_feed_storage.move#L393-L409
let market_session = feed.market_session();

if (market_session.is_some()) {
  // During off hours (e.g., weekends during 24/5 markets), the
```

```
`feed_update_timestamp`  
    // remains stale. We consider these "stale" prices as valid.  
    if (market_session.borrow().is_closed()) return update.timestamp().to_ms();  
};
```

The payload must still be signed by Pyth. The attacker cannot forge its price. However, anyone can submit a valid signed payload through the public update function ([price_feed_storage.move:202](#)). The core oracle accepts a newer timestamp and updates both the stored price and TWAP ([price_feed.move:267](#)). As a result, signed closed-session payloads can keep a carried price fresh for the protocol even though its real observation time is old.

The perpetuums package does not read the Pyth session state. `start_session` checks account authority, package version, and the Aftermath pause state, but not whether the underlying equity session is closed ([clearing_house.move:2356](#)). Limit orders remain available through the normal path ([clearing_house.move:2396](#)).

The 5% book-to-index limit is useful during the closed session. It clips the book input around the stale index ([market.move:981](#)). It does not limit the change in the index itself when genuine prices resume. A large reopening update may also trigger the index/TWAP divergence check. This can temporarily block both the winner's exit and liquidation. It delays the outcome; it does not remove the insolvent position. Once the TWAP converges enough for sessions to resume, transaction ordering and available liquidity decide whether the winner exits before the losing account is processed.

A trader can create two accounts with different global account IDs ([account.move:125](#)). The self-trade check rejects only equal account IDs ([clearing_house.move:3522](#)). The trader can therefore use one account to open a long and another to open an equal short at an available off-hours book price.

This removes the need for an outside counterparty when opening the two legs. It does not remove the need for outside liquidity when closing the winning leg after reopening. The winning account must close against other liquidity and withdraw before liquidation or loss socialization can charge or block it. If the winner stays open and the losing debt is socialized first, the funding debit on the opposite side may reduce the winner's gain.

Let N be the opening notional, r the initial margin ratio, g the absolute reopening price change, and F all fees. If each account starts with rN , the best-case attacker profit is:

$$\text{profit} = (g - r)N - F$$

The [24/5 Equity Markets specification](#) states that the affected single-stock markets use up to 5× leverage, so the price change must exceed about 20% plus fees. US500 uses up to 10× leverage, so it must exceed about 10% plus fees. Liquidation can start earlier: for $r = 20\%$ and $\text{MMR} = 10\%$, the thresholds are about 9.09% for a losing short and 11.11% for a losing long. Both are outside the 5% closed-session mark range, so the two single-stock legs can remain healthy throughout the full allowed closed-session mark movement. For US500, the losing-short threshold is about 4.76%, so a 5% off-hours mark movement may already liquidate that leg. The US500 variant is therefore less reliable.

If the first usable reopening price path jumps over the liquidation range, the losing account can become insolvent before a liquidator has a usable intermediate state. Full liquidation then records negative collateral as bad debt ([clearing_house.move:4148](#)). The insurance fund pays first, and any remainder enters loss socialization ([clearing_house.move:4293](#)).

The impact can be High because the winner withdraws real vault tokens while the losing account can leave debt for the insurance fund or other traders. Likelihood remains Low. Profitable exploitation still requires a gap above 10% or 20% plus fees, enough off-hours and reopening liquidity, a usable oracle/TWAP state for the winner to close, sufficient vault liquidity, and favorable ordering before liquidation or socialization. The project confirmation proves that closed-session entry and the Pyth Lazer source are reachable, but it does not prove that these remaining conditions occur often. The correct severity is therefore Medium.

Recommendations:

Keep the intended off-hours trading model, but add a risk policy that uses the oracle's session state and real price-observation time.

The safest policy is to allow cancellation and reduce-only trading while the underlying session is closed, and block new or larger exposure. If full-risk off-hours trading must remain available, use a higher off-hours IMR and a separate cap for exposure opened during the closed session. Set both values from measured closed-to-open gap data. $\text{max_book_index_spread}$ cannot cover a gap in the index itself.

At reopening, keep the market reduce-only until a genuine fresh price is available, the index/TWAP check permits sessions, and liquidators have had a defined opportunity to process

unhealthy positions. Preserve `feedUpdateTimestamp` separately from the message timestamp so the risk engine can distinguish a fresh message from a fresh price observation.

Add an end-to-end regression test covering a signed closed-session payload, two accounts opening opposite positions, the full allowed off-hours book movement, a reopening gap, TWAP convergence, winner withdrawal, and losing-account liquidation. Test single-stock and US500 parameters separately because the 5% mark range crosses the US500 short-liquidation threshold but not the single-stock thresholds.

Customer Response:

Acknowledged with no changes. This is by design: 24/7 markets can have significant price changes during the weekends (or closed hours) and the index price at reopening would just reflect it. Nothing stops traders from managing their positions during the closed hours since the trades and all the other operations are totally possible.

M-32: Deposits price a closed-market position with a live mark and can dilute existing LPs

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/market-making-vault/sources/vault.move packages/perpetuals/sources/clearing_house.move	Status: Fixed	

Description:

A vault can still hold a valuable position after a market is closed and settlement prices are enabled. The position's final value is fixed by [SettlementPrices](#), but the deposit path prices it with the live mark.

[process_clearing_house_for_deposit](#) skips the funding update for a paused market, but still calls [mark_price](#) and adds the result to the NAV used to mint LP shares:

[Code snippet location](#)

```
if (!clearing_house.is_market_paused()) {
    clearing_house.update_funding(base_oracle, clock);
};
let mark_price = clearing_house.mark_price(base_oracle, clock);
let (vault_position_margin, _) =
deposit_session.vault.get_vault_margin_in_market(
    &clearing_house,
    account_id,
    mark_price,
    deposit_session.collateral_price,
    ifixed::from_u64(1),
    0,
    true
);
deposit_session.vault_balance_value = ifixed::add(
    deposit_session.vault_balance_value,
    vault_position_margin
);
```

The live `mark_price` reads the current base oracle, index TWAP, and orderbook. It has no closed-market check and does not read `SettlementPrices` ([clearing_house.move:390](#)).

The deposit then mints `provided_balance_value * lp_supply / vault_balance_value` shares ([vault.move:1621](#)). If the live mark understates the frozen settlement value, the depositor receives too many shares.

Settlement uses the frozen base and collateral prices and realizes the position PnL at those values.

[Code location](#)

```
let settlement_prices = clearing_house.settlement_prices();
assert!(settlement_prices.enabled, settlement_prices_disabled!());
let base_price = *settlement_prices.base_price.borrow();
let collateral_price = *settlement_prices.collateral_price.borrow();
// ...
(pnl, _) = position.add_base_to_position(
    !is_position_short,
    abs_base,
    ifixed::mul(abs_base, base_price)
);
position.add_to_collateral_usd(pnl, collateral_price);
```

The MMV settlement wrapper requires no authority ([interface.move:1080](#)). A depositor can therefore mint against the lower live NAV and then settle the position at its higher frozen value.

Let `V_live` be the NAV used for minting, `V_settle` the frozen settlement NAV, and `D` the deposit. The depositor's gain after settlement is:

$$D * (V_settle - V_live) / (V_live + D)$$

This value comes from existing LPs. The impact can be High, but the likelihood is Low. The oracle must remain fresh after closure, the prices must differ in the useful direction, the position must remain unsettled, and any user can settle first and close the window. These limits support Medium severity.

Recommendations:

When a tracked market has settlement enabled, value the vault position with the frozen base and collateral settlement prices. Do not call the live `mark_price` for that position.

An alternative is to require the permissionless settlement and `ch_id` cleanup to finish before a deposit can mint shares. Add a regression test where the live oracle moves after settlement is enabled, and verify that deposit share output is unchanged or the deposit is blocked until settlement.

Customer Response:

Fixed in commit . Added a check on the market making vault contracts about the closed markets and calculated position's value based on the settlement prices for markets where they are set.

Fix Review:

Fix confirmed.

M-33: A vault owner can capture part of a deposit through LP share rounding

Severity: Medium	Impact: High	Likelihood: Low
Files: packages/market-making-vault/sources/vault.move	Status: Fixed	

Description:

`add_yield` adds assets to the vault without minting LP shares and does not apply the deposit cap ([interface.move:138](#)). A malicious vault owner can use this to raise the value of each LP unit. The owner can then withdraw most of the owner-locked LP shares while retaining a small number of raw units. The withdrawal is proportional, so it preserves the raised share price ([vault.move:2923](#)). The remaining owner balance only needs to be non-zero and worth at least `min_owner_lock_usd` ([vault.move:3017](#)).

The deposit calculation then divides by the raised vault value and rounds the LP output down to an integer. The zero check prevents a complete loss, but the slippage check only compares the rounded LP count:

```
let lp_to_mint = multiply_by_rational_ifixed(
    provided_balance_value,
    ifixed::from_balance(vault.lp_supply_value(), B9_SCALING),
    vault_balance_value
);
let lp_to_mint_b9 = ifixed::to_balance(lp_to_mint, B9_SCALING);
assert!(lp_to_mint_b9 != 0, errors::user_lp_calculation_zero());
let lp_balance = vault.lp_supply.increase_supply(lp_to_mint_b9);
let minted_lp_coin_value = lp_balance.value();
assert!(minted_lp_coin_value >= min_expected_lp_coin_out,
errors::slippage_check());
```

If the deposit mints one LP unit and `min_expected_lp_coin_out` is 0 or 1, the transaction succeeds even when a large fraction was removed by rounding. That value increases the value of the owner's existing shares. The owner can realize part of it by withdrawing shares while leaving the required final owner share in the vault.

Consider the following pseudocode code shows the value transfer. It uses three retained owner units so part of the gain can be withdrawn immediately. It is illustrative and is not exact Move code.

```
S = 3 // owner LP units retained after
add_yield and withdrawal
P = $3 // value of each unit before the user
deposit
owner_value_before = S * P = $9

D = $5.99 // user deposit
min_expected_lp_coin_out = 1
minted = floor(D / P) = 1
assert(minted >= min_expected_lp_coin_out) // passes

post_deposit_nav = S * P + D = $14.99
post_deposit_supply = S + minted = 4
post_deposit_price = $14.99 / 4 = $3.7475

user_value = minted * post_deposit_price = $3.7475
user_loss = D - user_value = $2.2425 // 37.44% of the deposit

owner_withdraws = S - 1 = 2 units
owner_receives = 2 * $3.7475 = $7.495
owner_immediate_gain = $7.495 - 2 * $3 = $1.495
owner_locked_gain = $3.7475 - $3 = $0.7475
assert(owner_immediate_gain + owner_locked_gain == user_loss)
```

This issue has Medium severity because it can transfer a meaningful part of a user's deposit to a malicious vault owner. Likelihood is Low because the owner must prepare capital, place the state change before the deposit, and the transaction can be stopped by a suitably strict user-side minimum or by refusing deposits that would mint only a few raw LP units.

Recommendations:

Use the existing `min_expected_lp_coin_out` parameter as the immediate user-side protection. The frontend or SDK should obtain the expected LP output before the user signs, apply a small allowed slippage, and require a non-zero minimum instead of defaulting to 0. In a normal

B9-scaled vault, a deposit should mint a large number of raw LP units; an output of only one or two units is itself an abnormal warning sign. For example, the \$3.99 deposit in the control test mints 3,990,000,000 raw LP units before the dilution. With 1% allowed slippage, the client should set `min_expected_lp_coin_out = 3,950,100,000`. If the owner changes the state so the same deposit would mint only one unit, the existing check aborts and rolls back the transaction ([interface.move:224](#), [vault.move:1635](#)).

The minimum cannot detect manipulation that happened before the quote. In that case, the current state may correctly quote one LP unit, and a minimum of one still accepts the rounding loss. The quote flow should therefore also show the implied share price and rounding loss, and reject a deposit that would mint only a few raw units or exceed a configured loss limit.

As a contract-level defense, prevent the owner from reducing retained supply below a safe number of raw LP units, or use virtual or permanently locked shares to preserve precision. Extend the deposit event with the pre-deposit supply, share price, selected minimum, and calculated rounding loss so unsafe deposits can be monitored.

Customer Response:

Fixed in commit [b80f07f](#). After calculating and rounding down the LP shares to mint, the vault now computes the collateral value exactly represented by those shares using the pre-deposit NAV and supply. That collateral is rounded up to the nearest collateral atom and deposited; any unrepresented remainder is refunded to the deposit session's configured sender. The accepted amount—not the original offer—is used for the deposit minimum, vault cap, cost basis, and deposit event. The contract also conservatively values the minted shares after deposit and aborts unless the depositor's rounding loss is no greater than one collateral atom, calculated dynamically from the collateral scaling factor and oracle price. Therefore, the loss is independent of TVL or LP share price and is bounded to one token atom (e.g. \$0.000001 for a \$1 six-decimal collateral).

Fix Review:

Fix confirmed.

Low Severity Issues

L-O1: Insurance-fund surplus reserve uses live long-only open interest, drainable during settlement

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move	Status: Acknowledged	

Description:

`withdraw_insurance_fund` is meant to let the treasury withdraw only the surplus of the insurance fund, keeping back a reserve sized to the market's open-interest risk so the fund stays available to absorb bad debt. The reserve threshold is computed from `market_state.open_interest()`:

```
// clearing_house.move:1819
let open_interest_usd = ifixed::mul(
    ifixed::abs(clearing_house.market_state.open_interest()), // live,
    long-only OI
    index_price,
);
...
let insurance_threshold_usd = ifixed::mul(
    registry.config().insurance_open_interest_fraction(),
    open_interest_usd,
);
let insurance_surplus_usd = ifixed::sub(insurance_fund_usd,
insurance_threshold_usd);
assert(!ifixed::is_neg(insurance_surplus_usd),
deposit_or_withdraw_amount_zero!());
```

Two facts combine into the bug:

- `open_interest()` is long-only: each position's Open Interest delta is `max(base, 0) - max(base_before, 0)` ([position.move:397](#)), so the aggregate equals the total long base.

Closing a long subtracts its base; closing a short contributes nothing ([clearing_house.move:4804](#)).

- The threshold reads this value live. The `df::exists(settlement_prices)` branch ([:1803-1817](#)) only swaps the price source to the settlement price; it still reads live `open_interest()` at line 1820. There is no closed-market / settlement guard on the function.

During market settlement, positions close individually against the settlement price rather than being matched off against each other. If the longs close before the underwater shorts settle, live OI decays toward 0 while short exposure (and its bad debt) is still open. The threshold then collapses to 0, the entire fund reads as surplus, and the treasury can withdraw all of it. When the shorts finally settle, `handle_bad_debt` finds an empty insurance fund ([:4013](#)) and the loss is socialized onto remaining solvent positions (or exceeds `max_bad_debt` and aborts into ADL).

This is a permissioned issue, not a permissionless one: `withdraw_insurance_fund` requires `AuthorityCap<VENDOR<VendorKey>, TREASURY>` ([:1786](#)), the same trusted role that drives `close_market` and settlement. The severity is Medium because the surplus reserve is a deliberate safety rail on treasury custody of the fund, and it silently evaporates during wind-down — the one window where the fund is most needed. The documented procedure ([:1115-1121](#)) tells the operator to clear all bad debt before withdrawing, but that is operator discipline, not an enforced control. A malicious or compromised treasury, or an honest one that interleaves a withdrawal mid-settlement, defeats the reserve.

Consider the following Scenario:

- `closed_market_withdraw_drains_reserve_after_long_close` — sets up alice-long / bob-short with \$1,000,000 of open interest (so the 5% reserve is a headline \$50,000, asserted in-test) plus a \$1,000 surplus, calls `close_market`, then closes alice's long at settlement to drive live OI to 0, and shows `withdraw_insurance_fund` then succeeds in taking the entire fund including the \$50,000 reserve.
- `reserve_blocks_full_withdraw_before_long_close` — `#[expected_failure(abort_code = 29)]` on the same full-fund withdrawal *without* closing the long, confirming the reserve is real and enforced and that live-OI decay (not the settlement price) is the cause.

The scenario the test reproduces:

Setup: Alice is LONG and Bob is SHORT (same size), so `open_interest` = Alice's long base.

The price has moved against Bob, so Bob's short is underwater and will create bad debt.

The insurance fund holds: `surplus + reserve`, where `reserve = fraction * open_interest * price`.

1. `treasury: close_market()`
 - > market paused for settlement; OI snapshot frozen for the bad-debt path
 - > live `open_interest()` still == full OI
2. close Alice's LONG at settlement prices
 - > `open_interest()` counts longs only, so closing the long subtracts Alice's base
 - > live `open_interest()` -> 0 (Bob's short contributes nothing)
 - > Bob's underwater short is STILL OPEN; the bad debt has not gone away
3. `treasury: withdraw_insurance_fund(entire_fund)`
 - `reserve = fraction * live_open_interest * price = fraction * 0 * price = 0`
 - `surplus = insurance_fund - reserve = entire_fund - 0 = entire_fund`
 - > "amount <= surplus" passes; the WHOLE fund is withdrawn
4. Bob's short settles -> bad debt -> `handle_bad_debt()` finds an EMPTY fund
 - > loss socialized onto remaining solvent users (or `max_bad_debt` tripped -> ADL)

Root cause: step 3 uses `live open_interest()` (driven to 0 in step 2) instead of the frozen

snapshot from step 1. The reserve, the one safety rail meant to protect the fund during

wind-down, silently drops to 0.

Recommendations:

- Enforce the documented settlement procedure ([:1115-1121](#)) in code rather than by convention: While `SettlementPrices` exists, cap `withdraw_insurance_fund` to the snapshot-based `reserve` — hold the threshold at `fraction * settlement_prices.open_interest * price` so the reserve cannot be drained below its

frozen size during wind-down, regardless of live-OI decay. Release the reserve only through an explicit admin step (e.g. a `finalized` flag on `SettlementPrices`) that the operator sets after confirming every position is closed and all bad debt is cleared. Once finalized, the reserve requirement drops to 0 and the remaining fund becomes withdrawable.

- Or document clearly the procedure for `withdraw_insurance_fund` avoid this issue during operation.

Customer Response:

The Aftermath team acknowledged this and that is the reason why they always plan to handle bad debt in positions before making them able to settle when a market is closed.

L-O2: Collateral oracle price has no spot-vs-TWAP divergence guard, unlike the base oracle

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/perpetuals/sources/market.move	Status: Fixed	

Description:

Every risk-sensitive flow reads the base index price together with its TWAP and enforces a divergence bound, so a single manipulated or anomalous oracle print cannot be used directly:

```
// clearing_house.move:4544 (start_session_; same pattern at :378, :554, :1811, :4489)
let (index_price, index_twap_price) =
market_params.base_oracle_price_and_twap_price(base_oracle, clock);
market_params.assert_index_twap_divergence_within_limit(index_price,
index_twap_price);
```

The collateral price gets no such check. `collateral_oracle_price` reads the spot feed only:

```
// market.move:1426
public fun collateral_oracle_price(market_params: &MarketParams, oracle:
&PriceFeedStorage, clock: &Clock): u256 {
    assert!(oracle.storage_id() ==
market_params.core_params.collateral_storage_id,
invalid_collateral_price_feed_storage!());
    price!(oracle, clock, market_params.core_params.collateral_source_id,
market_params.core_params.collateral_pfs_tolerance)
}
```

`price!` ([market.move:1503-1518](#)) only checks freshness; it never compares spot to TWAP, and the collateral feed's TWAP is not read at all. This unguarded spot collateral price then feeds every collateral-denominated calculation: free-collateral/withdrawal ([clearing_house.move:4485-4512](#)), session margin snapshots ([:4540-4605](#)), liquidation/settlement conversions, and insurance-fund valuation. Collateral USD value is linear in

this price — $\text{collateral} \cdot \text{collateral_price} \cdot (1 - \text{haircut})$ ([position.move:800-817](#)) — and the haircut is a fixed discount, not a manipulation bound.

An actor able to move the collateral oracle source can therefore inflate `collateral_price`, withdraw more collateral tokens than true margin supports, and leave the shared vault under-backed once the price reverts; or inflate it at `start_session` to open/add exposure beyond what fair collateral supports, which liquidates into socialized bad debt after the price reverts. (A depressed collateral price is also harmful, but via forced liquidation of victims and blocked withdrawals rather than attacker over-withdrawal.)

This matters because collateral is not restricted to a stablecoin. The protocol supports inverse markets where the collateral is a volatile asset — per the docs, "the collateral asset is the base asset itself (or a wrapped version)" (e.g. BTC/USD collateralized by BTC, SUI/USD collateralized by BTC or ETH), corroborated in code by the aggregator's `average_reciprocal_*` getters ([oracle-aggregator.price.move:1287+](#)). In such a market the same volatile asset is treated as manipulation-resistant for the index (the TWAP-divergence guard) but manipulation-exposed for the collateral (spot only). The base guard does not cover this: base and collateral are separate feeds (`base_source_id/base_storage_id` vs `collateral_source_id/collateral_storage_id`), so moving only the collateral source leaves the base feed within its divergence bound and `assert_index_twap_divergence_within_limit` still passes. The aggregator adds no protection of its own either — the perps consumer uses a single feed per role (no median), Pyth's confidence interval is dropped, and the integrations read with `get_price_unsafe` / `_unchecked` (no in-integration staleness).

Severity is Low: the harmed value is real (over-withdrawal from the commingled vault, or socialized bad debt), but the actor must be able to move the collateral oracle source, and moving a deep BTC/ETH oracle is costly — which keeps likelihood low. For a thinly-sourced inverse-market collateral the likelihood, and so the practical severity, is higher.

Consider the following scenario:

```
PRECONDITION: inverse market, collateral coin T volatile (e.g. TokenT); account
has collateral C at true price P_true.
1. attacker moves the collateral oracle source: reported spot -> P+ > P_true
(single feed, no median, no twap guard)
2. attacker (account holder): deallocate_collateral(amount = max)
    collateral_price = collateral_oracle_price(...) = P+           //
market.move:1426 (spot, no twap divergence check)
```

```
base path still passes assert_index_twap_divergence_within_limit // base
feed untouched
free collateral valued at P+ -> withdraws ~ C*(P+ - P_true)*(1-haircut)/P+
extra tokens from the shared vault
3. collateral oracle reverts to P_true -> position under-collateralized, tokens
gone, vault under-backed -> socialized loss
NEGATIVE: with spot == twap (no manipulation) the withdrawal equals the true free
collateral; nothing is over-extracted.
```

Note that crypto assets price sgenerally are volatile, so this scenario can happen in some market conditions.

Recommendations:

The base oracle is already protected by a spot-vs-TWAP divergence check ([assert_index_twap_divergence_within_limit](#), [market.move:1011-1027](#)). Apply the same guard to the collateral oracle: keep reading the collateral spot price, but also fetch the collateral feed's TWAP and assert the spot does not diverge from it beyond a configurable [max_collateral_twap_divergence](#) bound, rejecting the price (aborting) on breach. The per-source TWAP is already maintained on the collateral feed, so this is the same shape as the existing base-side check. This preserves the freshness benefit of using spot (so a genuine collateral move is still reflected promptly) while rejecting a single manipulated or anomalous print, closing the asymmetry between the base and collateral paths.

Customer Response:

Fix in commit [c262260](#) . Collateral oracle now loads spot + TWAP and aborts on spot-vs-TWAP divergence beyond the bound (same guard as the base index path).

Fix Review:

Fix confirmed

L-03: Public settlement lets early closers escape bad-debt socialization applied later, stranding the last closers

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/market.move	Status: Acknowledged	

Description:

When a market is closed, `close_market` stores settlement prices with `enabled = false` (clearing_house.move:1107–1141). The function's own docstring (clearing_house.move:1107–1113) describes the intended procedure: admins must close the bad-debt accounts first — socializing their losses through funding-rate bumps — and only then set `enabled = true` to open public settlement.

No on-chain invariant enforces that ordering. Once `enabled = true`, any user can call `close_position_at_settlement_prices`, which is gated only by that user's own account capability, and deallocate collateral at the current funding rates. Bad debt socialized afterwards is applied as a one-sided `cum_funding_rate` bump (market.move:960–987) that only charges positions still open at that moment. A profitable trader who closes early therefore escapes the charge entirely, and the burden falls on whoever is still open when the bad-debt accounts are settled — the last closers can themselves be pushed into bad debt. If all remaining open interest has already closed, the socialization step instead aborts with `no_open_interest_to_socialize`, bricking settlement.

The attacker cannot flip `enabled` themselves — this is not permissionless theft — but the protocol provides no guard preventing users from exploiting the window the admin opens, so a single ordering mistake (or any interleaving of public closes with not-yet-settled bad-debt accounts) silently breaks value conservation among settling users.

Recommendations:

Enforce the ordering on-chain: do not allow public settlement until all bad-debt accounts have been closed and their losses socialized. A settlement state machine that requires the bad-debt set to be empty before **enabled** can become true would close the window.

Customer Response:

Acknowledged. This should be solved by the bad debt and ADL procedure.

L-04: Stale dynamic-field state after AuthorityCap destruction

Severity: Low	Impact: Low	Likelihood: Low
Files: packages/authority-cap/sources/authority.move	Status:	

Description:

When an AuthorityCap is destroyed, only the cap's own object is deleted. The tracking entries written to the parent object and the registry at cap creation time are not cleaned up. Depending on cap type this leaves different categories of stale state.

Singleton caps — re-creation permanently blocked after destruction

Singleton caps (PACKAGE:ADMIN and VENDOR:ADMIN) write a creation marker to their parent object. This marker guards against creating a second cap of the same type and is never removed when the cap is destroyed. If a singleton admin cap is destroyed, any future attempt to re-issue a cap of the same type for the same context permanently aborts — even though the original cap no longer exists.

In the current protocol both singleton creation paths are already one-time by other means, so this does not affect existing flows. The risk is that a future upgrade or emergency recovery mechanism that attempts to re-issue a lost or destroyed singleton admin cap would be irrecoverably blocked by the orphaned marker.

Multiton caps — stale authorization entries in the registry

When multiton caps are created (PACKAGE:ASSISTANT, PACKAGE:ADL, PACKAGE:PAUSE_GUARDIAN, VENDOR:ASSISTANT, VENDOR:PAUSE_GUARDIAN, VENDOR:TREASURY), an authorization entry keyed by the cap's object ID is written to the registry. The intended lifecycle is to explicitly deauthorize a cap before destroying it. However, nothing enforces this order — a cap can be destroyed while its authorization entry remains. The result is a permanently orphaned registry entry for a cap object that no longer exists. Since the cap is gone it cannot be presented to pass authorization checks, so this is not exploitable, but the stale entries accumulate with no cleanup mechanism.

Account admin cap — registry entry with no unregister path

The account admin cap is registered in the registry when an account is created, but there is no destroy function for it and no corresponding unregister path. If the cap were ever destroyed directly, its registry entry would remain permanently. For comparison, the account assistant cap has a properly paired destroy function that removes its registry entry before deleting the cap object — the same pattern is absent for the admin cap.

Recommendations:

For singleton caps, removing the creation marker on destruction would allow re-issuance if needed. For multiton caps, either make deauthorization atomic with destruction via a combined entry function, or enforce the required call order in documentation and access controls. The account admin cap should have a destroy path that mirrors the assistant cap's cleanup, removing the registry entry before deleting the object.

Customer Response:

Acknowledged with no changes.

Singleton caps: We've purposefully designed the `AuthorityCap` type (singleton vs multiton) to be immutable after creation. The package that defines the `Context : Role` pairing, defines in advance if the cap pair will be singleton or multiton.

The ability to create an emergency `AuthorityCap<Context, Role>` has been explicitly avoided as this package is anticipated to become a standard and we do not want to allow the option to perform this (even gated) within the `move-authority-cap` package. This would enable us, or a compromised `AuthorityCap<authority_cap::authority::PACKAGE, ADMIN>` to create a `PACKAGE : ADMIN` cap for any other package, an action we deem too risky to be added.

Given our stance on the above, in order to allow an emergency `AuthorityCap<Context, Role>` to be created, this would require an upgrade of the package that defines `Context`. The package can already define this emergency constructor locally by using a different UID to derive the `AuthorityCap<Context, Role>`. Thus our stance on this finding is that emergency creation should be avoided but if required should be handled by the domain's package itself and not within `move-authority-cap`.

Multiton caps: We don't find this an issue. In our CLI and API we already handle the correct flow of revoking auth before destroying. If a user interacts without contracts directly to destroy a cap, then they are just leaving a reclaimable storage rebate on the table.

L-05: Vault deposits abort on any single negative-margin market position, halting all new LP capital

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/market-making-vault/sources/vault.move	Status: Acknowledged	

Description:

A vault deposit must value every market the vault trades in before it can finalize. `deposit` snapshots `vault.ch_ids()` (`vault.move:1409`) and `end_deposit_session` asserts that all of them were processed (`vault.move:1484`, `not_all_chs_processed`). Each market is valued through `get_vault_margin_in_market` with `abort_if_negative = true` (`vault.move:1586-1596`, flag at `:1593`):

```
let (vault_position_margin, _) =
deposit_session.vault.get_vault_margin_in_market(
    &clearing_house, deposit_session.account.account_id(),
    mark_price, deposit_session.collateral_price, ifixed::from_u64(1), 0,
    true,                                     // abort_if_negative
);
```

and `get_vault_margin_in_market` aborts the whole transaction when that market's margin is negative (`vault.move:606-608`):

```
if (ifixed::is_neg(position_margin)) {
    assert(!abort_if_negative, errors::negative_market_margin());
    ...
}
```

Because the deposit PTB has to drain `ch_ids` to empty, reaching even one underwater market reverts the entire deposit. This is asymmetric with the withdraw path, which deliberately floors a negative per-market margin to zero instead of aborting (`abort_if_negative = false`, e.g. `vault.move:2083`, `:2164`, `:2174`). The consequence is that a single isolated vault position drawn below zero margin —before liquidation/ADL/socialization clears it— blocks all new deposits across the entire vault while existing LPs can still withdraw.

The impact is limited to liveness: the abort triggers Move's full atomic rollback (no partial state), existing LP funds are never at risk, withdrawals stay open, and deposits resume once the negative leg is cleared. The negative-margin state is market-driven rather than a privileged or cheap attacker toggle, so this is a temporary deposit-availability gap, not fund loss — hence Low.

Customer Response:

This is intentional. If a vault's position has negative margin (bad debt), it's not safe to let users deposit in a vault where the situation about that market has not been handled yet. The bad debt should be solved in a matter of seconds if the market is healthy. Otherwise, there is something major going on and it becomes even less responsible to let users deposit in such a vault.

L-06: A matured margin-ratio proposal can become simultaneously un-committable, un-deletable, and block creating new proposal

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/market.move	Status: Fixed	

Description:

Margin-ratio (IMR/MMR) updates use a timelock: create → wait → commit. The invariant $\text{liquidation_fee} + \text{insurance_fund_fee} < \text{MMR}$ is checked on every path, but each path compares fees to a different MMR value (proposed vs currently stored). Combined with a delete rule that rejects matured proposals, a pending proposal can become un-committable, un-deletable, and blocking new proposals.

`create_margin_ratios_proposal` checks fees vs proposed MMR:

```
// clearing_house.move:1476-1480
market::assert_liquidation_fees_against_mmr(
    margin_ratio_maintenance, // proposed MMR
    liquidation_fee,
    insurance_fund_fee,
);
```

Set fees `set_fee_params` checks new fees vs current stored MMR (ignores pending proposal):

```
// market.move:507-511
assert_liquidation_fees_against_mmr(
    params.core_params.margin_ratio_maintenance, // current MMR, not proposed
    liquidation_fee,
    insurance_fund_fee,
);
```

`commit_margin_ratios_proposal` checks current fees vs proposed MMR again:

```
// market.move:472-477
assert_liquidation_fees_against_mmr(
    margin_ratio_maintenance, // proposed MMR from proposal
    params.fees_params.liquidation_fee,
    params.fees_params.insurance_fund_fee,
);
```

Shared invariant (strict $<$):

```
// market.move:1707-1712
assert!(ifixed::less_than(
    ifixed::add(liquidation_fee, insurance_fund_fee),
    margin_ratio_maintenance,
), ...);
```

`delete_margin_ratios_proposal` trap after maturity

```
// clearing_house.move:1512-1517
let MarginRatioProposal { maturity, .. } = df::remove(&mut clearing_house.id,
key);
assert!(clock.timestamp_ms() < maturity, proposal_already_matured!());
```

If `now` $\geq$ `maturity`, the assert aborts. Move rolls back the whole transaction, so `df::remove` is undone and the proposal stays. A matured proposal can never be deleted.

Create blocked while any proposal exists:

```
// clearing_house.move:1460-1463
assert!(!df::exists(&clearing_house.id, keys::margin_ratio_proposal()), ...);
```

Consider a trap scenario as following:

Example: `MMR_old` = 5%, `MMR_new` = 3%, fees changes 2% $\rightarrow$ 4% during the delay.

1. Create passes: 2% current fees $<$ 3% (proposed MMR)
2. Set fees passes: 4% $<$ 5% (current MMR; proposed 3% not checked).
3. After maturity, commit fails: 4% $<$ 3% is false.
4. Delete fails: proposal is matured.
5. Create fails: old proposal still exists.

Recovery only by lowering fees below `MMR_new` and committing the old proposal — forcing adoption of a possibly unwanted lower MMR. Admin/assistant-gated trigger, no direct fund loss; genuine liveness trap on a safety-critical config path.

Recommendations:

Allow a matured proposal to be deleted: in `delete_margin_ratios_proposal`, drop the `clock.timestamp_ms() < maturity` assertion (or invert the design so a matured, un-committable proposal can always be cancelled by the admin cap). Alternatively, have `commit_margin_ratios_proposal` clear the proposal (and emit a "rejected" event) instead of aborting when the fee-vs-proposed-MMR invariant fails, so a stuck proposal cannot block `create`. Validating the proposed MMR against the fees that exist at commit is the right check — it simply must not leave the proposal permanently wedged.

Or document the operational procedure to avoid this bug scenario to happen.

Customer Response:

Fixed in commit [c262260](#). We have fixed this by allowing the deletion for matured proposals.

Fix Review:

Fix confirmed

L-07: receive_from_account lets a non-ADMIN assistant claim received objects and skips the package-version gate

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/perpetuals/sources/account.move	Status: Fixed	

Description:

Description

The account capability model splits privileges: an `AuthorityCap<ACCOUNT, ASSISTANT>` is a hot operational key that may drive the account but must not move value out, while value extraction is `ADMIN-only(withdraw_collateral takes &AuthorityCap<ACCOUNT, ADMIN>, account.move:240-255).receive_from_account` breaks this in two ways (`account.move:339-347`):

```
public fun receive_from_account<T, ADMIN_OR_ASSISTANT, Obj: key + store>(
    account: &mut Account<T>,
    cap: &AuthorityCap<ACCOUNT, ADMIN_OR_ASSISTANT>,
    receiving: Receiving<Obj>,
): Obj {
    account.assert_authority_cap_is_valid(cap); // accepts an active
ASSISTANT cap
    sui::transfer::public_receive(&mut account.id, receiving)
}
```

First, `assert_authority_cap_is_valid` → `has_authority!` (`account.move:394-411`) accepts an active ASSISTANT cap, and the function returns the received `Obj` to the caller. So an assistant can claim any `key + store` object (e.g. a `Coin<X>`, an airdrop, mis-sent funds) transferred to the account's address — an extraction primitive that the role split reserves for ADMIN. It cannot touch the account's own `collateral` balance, which is why this is Low rather than a collateral-theft path, but it still routes value-bearing objects to a key that is not supposed to extract value.

Second, it is the only state-affecting permissioned function in `account.move` that never calls `registry.assert_package_version()` — every sibling does (`create_account:132`,

`withdraw_collateral:247`, `new_assistant_account_cap:264`, `revoke_assistant_account_cap:290`, `destroy_assistant_account_cap:308`, `deposit_collateral:326`, `add_integrator_config:357`, `remove_integrator_config:376`). It therefore remains callable after a package upgrade bumps the registry version and freezes the rest of the module, exactly the migration window the gate exists to close.

This is privilege escalation within a trust boundary the ADMIN controls (the assistant is a chosen operational role).

Recommendations:

Restrict `receive_from_account` to `AuthorityCap<ACCOUNT, ADMIN>`, matching `withdraw_collateral`, since returning a received object is an extraction primitive; if assistants legitimately need to receive operational objects, scope and document it explicitly. Additionally, add a `registry: &Registry` parameter and call `registry.assert_package_version()` at the top, matching every other mutator in the module.

Customer Response:

Fixed in commit [89fe3ea](#). We modified that and all the other Account mutators to check the version against the Registry.

Fix Review:

Fix confirmed.

L-08: Final funding after market close can be diluted by a zero-premium settlement update

Severity: Low	Impact: Low	Likelihood: Low
Files: packages/perpetuals/sources/clearing_house.move packages/perpetuals/sources/market.move	Status: Fixed	

Description:

When a market is closed, final funding should be based on the market state at close time.

However, `close_market` only pauses the market and creates empty settlement prices. It does not store a close timestamp or finalize funding.

[File: packages/perpetuals/sources/clearing_house.move:1148-1183](#)

```
clearing_house.paused = constants::pause_mode_full!();

df::add(
  &mut clearing_house.id,
  keys::settlement_prices(),
  SettlementPrices {
    base_price: option::none(),
    collateral_price: option::none(),
    enabled: false,
  },
);
```

Later, `set_settlement_prices` accrues final funding. It passes `base_settlement_price` as both the index price and the book price:

[File: packages/perpetuals/sources/clearing_house.move:1185-1223](#)

```
clearing_house.market_params.try_update_fundings_and_twaps(
  &mut clearing_house.market_state,
  clock.timestamp_ms(),
  base_settlement_price,
```

```
    base_settlement_price,  
    &clearing_house_id,  
);
```

This creates a zero-premium observation:

```
index price = settlement price  
book price  = settlement price  
premium     = 0
```

In `try_update_fundings_and_twaps`, the premium TWAP is updated before funding is accrued. Therefore, this post-close zero premium can dilute the old pre-close premium TWAP.

[File: packages/perpetuals/sources/market.move:870-911](#)

```
update_premium_twap(...);  
...  
update_fundings(state, params, now, ch_id);
```

Final funding then uses the diluted `state.premium_twap`:

[File: packages/perpetuals/sources/market.move:1029-1059](#)

```
let funding_rate = ifixed::mul(state.premium_twap, period_adjustment);  
  
state.cum_funding_rate_long = ifixed::add(  
    state.cum_funding_rate_long, funding_rate  
);  
state.cum_funding_rate_short = ifixed::add(  
    state.cum_funding_rate_short, funding_rate  
);
```

As a result, final funding can be lower or otherwise different from the amount implied by the market state before close.

Consider following example:

```
book price > index price  
premium > 0  
longs should pay shorts
```

If the operator closes the market but does not immediately set settlement prices, the later settlement update uses:

```
index price = settlement price
book price  = settlement price
premium     = 0
```

This can reduce the premium TWAP before final funding is calculated. Shorts may receive less funding than expected, and longs may pay less than expected. If the pre-close premium was negative, the opposite side is affected.

The impact is incorrect final funding redistribution between longs and shorts during market shutdown. The error depends on the pre-close premium and the delay between `close_market` and `set_settlement_prices`.

The main risk is unfair settlement accounting caused by operator timing.

Recommendations:

Finalize funding at market close time, not later when settlement prices are set.

If the protocol accepts this as operational risk, document that `set_settlement_prices` should be called in the same PTB as `close_market`, or as soon as possible after it, to minimize the post-close zero-premium effect. This is only a mitigation; the complete fix is to finalize and freeze funding at close time.

Customer Fixed the math applying haircut correctlyResponse:

Fixed in commit [6705163](#) . `close_market` now takes `Clock` and finalizes funding via `try_update_fundings` at close using the live premium TWAP; `set_settlement_prices` only writes prices and no longer calls `try_update_fundings_and_twaps` (no zero-premium dilution).

Fix Review:

Fix confirmed.

L-09: Cross-Vendor Rolling Feed Object Can Be Bound During Provisioning

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages.integrations/pyth-lazer-rolling/sources/ price_feed_storage.move	Status: Acknowledged	

Description:

When a rolling **PriceFeed** is created, the code checks that the caller's vendor cap matches the destination **PriceFeedStorage**.

However, it does **not** check that the supplied **FeedInfoObject** belongs to the same vendor:

[price_feed_storage.move:75-99](#)

The core oracle then stores the object id of that **FeedInfoObject** as the trusted source object:

[price_feed_storage.move:324-330](#)

This creates a cross-vendor binding risk.

FeedInfoObject ownership is vendor-specific. The code already enforces this rule when the object is later modified:

[feed_info_object.move:212-213](#)

[feed_info_object.move:267-268](#)

But the same check is missing when the **PriceFeed** is first created.

So Vendor A can create a price feed in Vendor A's storage using a **FeedInfoObject** that belongs to Vendor B. After that, Vendor B can still update that object through **migrate()** or **set_rolling_window()**, because the object really belongs to Vendor B.

The update path will keep working because the object id has not changed:

[price_feed_storage.move:502-503](#)

If this wrong binding happens, Vendor B may later change the rolling feed ids or roll window used by Vendor A's storage.

This could make downstream markets read an unexpected oracle price.

The practical risk is low because Vendor B cannot trigger this alone. The initial wrong binding must be created by Vendor A's admin or assistant during provisioning.

Recommendations:

Before creating the rolling **PriceFeed**, check that the **FeedInfoObject** belongs to the same vendor:

```
source.assert_belongs_to_vendor!<VendorKey>(feed_info_object);
```

This should be added before the call into the core **new_price_feed**.

This is a small and safe check. It should not block normal users because valid setups already use a **FeedInfoObject** from the same vendor.

Customer Response:

Acknowledged with no changes. This is entirely up to the Vendor if they want to do that. In our opinion, there is no need to restrict that the **FeedInfoObject** corresponds to the same Vendor as the **PriceFeedStorage** and **AuthorityCap**. This is the same assumption as just using the **PriceFeedStorage** from the other Vendor.

We can imagine a scenario where we (Aftermath) create **FeedInfoObjects** for rolling price feeds and maintain their rolling schedule and other vendors want to piggy back off of our maintenance but have disjoint **PriceFeedStorages** for whatever reason. If the Vendor wants this relationship then we allow that but in all other cases the Vendor would just create and use their own **FeedInfoObject**.

L-10: `twap_period_ms` accepts unbounded values, letting a trusted user neutralize or freeze the TWAP safeguard

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/oracle-aggregator/sources/price_feed_storage.move packages/oracle-aggregator/sources/price_feed.move packages/oracle-aggregator/sources/authority.move	Status: Acknowledged	

Description:

`set_twap_period_ms` replaces a live feed's `twap_period_ms` from a caller-supplied value, and enforces no bounds on that value beyond rejecting zero (`EInvalidTwapPeriodMs`). Any nonzero `u64` is accepted, applied immediately, and the existing accumulator is lazily reinterpreted under the new period rather than re-warmed. Specifically, the operation is missing:

- **a lower bound:** setting the period to `1` (or any value below the source's update cadence) collapses the TWAP so it effectively resets to the instantaneous spot price on the next update, neutralizing downstream spot-versus-TWAP divergence checks;
- **an upper bound:** setting the period near `u64::MAX` freezes the TWAP near its current value, so ordinary market movement trips the divergence check indefinitely and halts every dependent market;
- any relationship to the feed's expected source cadence, and any warm-up on change — the new period takes effect against the old accumulator instantly, so the transition itself is a discontinuity.

The net effect is that a single unbounded parameter write can either disable a core security invariant (the divergence guard that gates manipulation) or turn that same invariant into a permanent DoS — without ever touching raw oracle prices.

The `MAINTENANCE` role is a trusted role (as are `ADMIN` and `ASSISTANT`), so this is not an untrusted-attacker path and should be rated accordingly. But trust in the role does not remove

the need for bounds. A long-lived operational maintenance key is a realistic compromise target, and an honest operator can just as easily supply an incorrect value (a mistyped period). In either case the absence of contract-level limits means one wrong write immediately and silently neutralizes or freezes the TWAP safeguard, with no circuit breaker to bound the damage. Enforcing sane limits at the protocol level is defense-in-depth: it constrains the impact of a compromised key or operator error instead of relying on the role being simultaneously honest and mistake-free.

Recommendations:

Bound and `stage `twap_period_ms`` at the contract level, even for trusted callers:

- Enforce a protocol-level minimum and maximum `twap_period_ms`, derived from the feed's expected source update cadence (min $\geq$ several source intervals; max $\leq$ a sane divergence-window ceiling).
- On any period change, place the feed into a warm-up state where the TWAP is unavailable (or the feed is flagged not-yet-ready) until enough observations under the new period have accumulated, instead of lazily reinterpreting the existing accumulator.
- Consider having `ADMIN` register the permissible period range per feed and letting `MAINTENANCE` only choose within that preapproved range, and/or apply a propose/commit delay to period changes so a mistaken or malicious value is observable before it takes effect.

This trades a small amount of operational flexibility for a bounded impact against both key compromise and accidental misconfiguration of an otherwise-trusted role.

Customer Response:

Acknowledged with no changes. `MAINTENANCE` is a role with more heightened permissions than `ASSISTANTS`, something closer to `ADMIN`. For this reason, we keep `MAINTENANCE` caps behind a multisig and require multisig signers to sign off on all `MAINTENANCE` functions (we do the same for `ADMIN`, of course).

We have some checks like this built into our CLI, which we all use to sign and execute the multisig transaction. For now, we are fine with these gates protecting the updates of `twap_period_ms`. In the future, we may decide to add min / max consts for this field.

L-11: Transferable loss basis can be joined into profitable LP positions to eliminate performance fees

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/market-making-vault/sources/vault.move	Status: Acknowledged	

Description:

Each transferable `UserLpCoin` carries its historical `provided_value_usd`. Joining two coins adds their balances and cost bases, after which the owner fee is assessed only on the combined result: $\max(\text{combined current value} - \text{combined historical basis}, 0)$. This permits an underwater, high-basis LP object to act as a transferable performance-fee coupon. An attacker can buy an underwater coin for approximately its current claim and join it with a profitable coin. The combined embedded basis will offset the profit, eliminating the owner fee entirely, even though the attacker realizes actual profit on the acquisition.

Recommendations:

Choose and enforce one fee-accounting model: Preserve separate fee lots and crystallize each lot's fee before joining, bind the basis to a nontransferable depositor account so loss carryforwards cannot be sold, or use a vault-wide high-water-mark model that accrues fees through share dilution.

Customer Response:

Acknowledged with no changes. The mechanism is as described and will not be changed.

`UserLpCoin` has `store` (vault.move:1391) and `interface::join` is permissionless behind a version check (interface.move:37-46). `join_user_lp_coin_` sums both `lp_balance` and `provided_value_usd` (vault.move:1436, :1445), and the owner fee is charged only on `value_to_withdraw - provided_value_usd` when that is positive (:2956-2962), so a high-basis coin does offset profit on a coin it is joined into.

Three properties bound it.

The fee accrues to `Vault.owner_fees`, which is the vault operator's revenue. LP balances and the share price are untouched, so the pro-rata claim of every other LP is unchanged. This is fee revenue the operator forgoes, not a loss borne by users.

The amount is bounded by `owner_fee_rate x offset`, and `owner_fee_rate` is capped by `Config.max_owner_fee_rate`, 20% by default (config.move:163). Producing an offset of X requires someone to have actually lost X in the vault.

The join takes the later of the two `start_timestamp_ms` (vault.move:1442-1444), so the merged coin inherits the longer remaining lock. `split_user_lp_coin_` scales the basis pro rata with the balance (:1479-1482), so the basis cannot be stripped or concentrated: every piece split off a merged coin carries the blended basis.

On the three recommended models: crystallizing each lot's fee before joining requires the vault's current value at join time, which means the oracle, the perps account and every clearing house – the same inputs `end_withdraw_session_` needs – where `join` today takes only `&mut Config`. Binding the basis to a nontransferable depositor account removes `UserLpCoin` transferability. A vault-wide high-water mark changes fee accrual for every LP.

Each is a change to the product, not a correction, and we are keeping the current model.

L-12: Missing `upgrade_version` for Config prevents deprecating old package bytecode

Severity: Low	Impact: Low	Likelihood: Medium
Files: packages/market-making-vault/sources/config.move	Status: Fixed	

Description:

The `market-making-vault` package employs a version-gating mechanism (`assert_package_version`) to restrict access to outdated package versions. Upon upgrading the package, the protocol is expected to bump the `Config.version` state to match the new `version!()` constant. This causes `config.version <= version!()` in older packages to evaluate to false, effectively disabling their bytecode. However, the `market_making_vault::config` module lacks an `upgrade_version` function to mutate the `Config`'s version field. As a result, the `Config` version will permanently remain at its initial value. This breaks the protocol's upgrade and deprecation model, allowing anyone to indefinitely invoke the old package's bytecode and bypass security fixes or constraints introduced in newer versions.

Recommendations:

Implement an `upgrade_version` function in `market_making_vault::config` that allows an authorized admin or assistant to update `config.version` to `version!()`, emitting an upgrade event and ensuring older bytecode correctly fails its version check.

Customer Response:

Fixed in commit [c7cfae8](#). `Config.version` was written only at creation, so `assert_package_version` could never reject an older package and the deprecation model did not work for `Config`. Vault already had the equivalent in `upgrade_vault_version`; only `Config` was missing it.

Added `market_making_vault::config::upgrade_version`, mirroring `upgrade_vault_version`: it asserts that `version!()` is greater than the stored version, sets it, and emits a new `UpgradeConfigVersion` event. The new error is `EBadNewConfigVersion`. It is reached through `interface::upgrade_config_version`, an entry function gated on `AuthorityCap<PACKAGE`,

ADMIN|ASSISTANT> via `assert_authority_cap_is_valid`, matching the adjacent `upgrade_vault_version` entry point.

The monotonic check also rejects re-running the upgrade at the current version rather than letting it silently succeed.

No separate freeze guard was added: `assert_authority_cap_is_valid` already runs `assert_package_version`, and a frozen Config sits at `u64::MAX`, so it cannot be upgraded until unfrozen. That preserves the resume version stored by `freeze_package`.

Fix Review:

Fix confirmed

L-13: Limit orders revert on partial fills if the remaining posted size is below `min_order_usd_value`

Severity: Low	Impact: Low	Likelihood: Medium
Files: packages/perpetuals/sources/clearing_house.move	Status: Fixed	

Description:

When a user places a limit order using `place_limit_order`, the order may partially fill against the order book. Any remaining unfilled size is then posted to the book. The code checks if the remaining `size_posted` is non-zero, and if so, asserts that its USD value is greater than or equal to `min_order_usd_value` via `assert_order_value`. If the remainder happens to be smaller than the threshold (i.e., dust), the entire transaction aborts. This means that a large, valid limit order that crosses the spread will unpredictably revert if the available liquidity leaves a dust remainder. Malicious actors can grief traders by strategically placing dust orders in the book to ensure victims' limit orders leave a remainder just below the threshold, causing the victims' trades to fail entirely.

Recommendations:

Do not abort the entire transaction when the remaining posted size is below the minimum order value threshold. Instead, either evaluate the `min_order_usd_value` constraint against the initial requested size before matching, or automatically cancel the dust remainder (acting as an Immediate-Or-Cancel for the remaining portion) while allowing the successful fills to persist.

Customer Response:

Fixed in commit [67c846f](#). `place_limit_order` now applies the `min_order_usd_value` check only when the order matched nothing against resting liquidity, which `size_posted == size` identifies on the ordinary matching path.

A remainder left by a partial fill is posted as requested; an order that matches nothing is still rejected below the threshold, as before. Post-only orders never match, so they remain covered; `IOC` and `FOK` reach that point with `size_posted == 0` and skip it as they did before.

We post the remainder rather than discarding it, because discarding size the caller asked to match or post is not something we want to do silently. The single exception is the zero-fill cap under M-14: matching there stops on an internal bound rather than on the taker's limit price, so orders crossing that price can still be resting and the remainder is dropped rather than rested across the book. Reaching that path leaves `size_posted == 0`, so the threshold check is skipped and nothing is posted.

The threshold gates size entering the book, not size already on it: `reduce_order_size` is a plain subtraction with no re-check, so any resting order that is partially filled is already taken below `min_order_usd_value` and left resting. Sub-threshold orders on the book are therefore an existing, ordinary state rather than something this change introduces.

Fix Review:

Fix confirmed.

L-14: VendorMetadata setters omit version-gating, allowing users to bypass package upgrades

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/vendor/sources/metadata.move packages/vendor/sources/config.move	Status: Fixed	

Description:

Functions like `set_name`, `set_description`, `set_extra_field`, `approve_domain_registration`, and `revoke_domain_registration_approval` in the `vendor::metadata` module do not take a reference to the global `Config` object. Consequently, they do not invoke `config.assert_package_version()`. If the `vendor` package is upgraded to a newer version to enforce new constraints or deprecate outdated logic, users can completely bypass the upgrade by continuing to interact with these functions via the older, superseded package bytecode.

Recommendations:

Add a `config: &Config` parameter to all public mutating functions in `vendor::metadata` and ensure they call `config.assert_package_version()` before performing the mutation.

Customer Response:

Fixed in commit [3c85413](#). We've updated the module dependency tree of `move-vendor`. Previously `config.move` depended on `metadata.move`. We've swapped that dependency, so now `metadata.move` depends on `config.move`. When doing so, we:

1. Added versioning asserts for all metadata mutators,
2. Split the `VaultMetadata` creation from `register_vendor`. Now the vendor registration flow is one PTB that calls `vendor::config::register_vendor` + `vendor::metadata::new`.

Fix Review:

Fix confirmed



L-15: Emergency pause and freeze states prevent revocation of compromised vault capabilities

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/market-making-vault/sources/vault.move: packages/market-making-vault/sources/interface.move	Status: Fixed	

Description:

The vault's emergency states disable the operation needed to remove a compromised assistant or treasury capability:

- Freezing sets `vault.version` to `u64::MAX`, while `deauthorize_vault_authority_cap` first calls `vault.assert_package_version()`.
- An indefinite admin pause causes the internal deauthorization function to fail `assert_vault_is_not_admin_paused()`.

Only the package `ADMIN` can unfreeze a vault, and only the package `PAUSE_GUARDIAN` can clear an admin pause. However, only the vault-specific `ADMIN` can revoke the compromised vault capability. These capabilities are normally held by separate parties, so remediation cannot generally be performed atomically.

The package operator must first reactivate the vault and then wait for the vault owner's revocation transaction. During that interval, the compromised assistant can race the revocation using the shared vault object and regain its broad trading and account-management powers. A compromised treasury cap can similarly race to withdraw accumulated fees.

Recommendations:

Permit authority-reducing revocations while a vault is paused or frozen. A dedicated emergency revocation path should:

- Validate the vault admin cap and its `for` association.
- Reject revocation of the vault admin itself.

- Require the target marker to be active.
- Update `active_assistant_count` when applicable and emit the existing revocation event.

This path should deliberately omit pause and live-version checks, as it only reduces authority and cannot move vault funds.

Customer Response:

Fixed in commit [c7cfae8](#). We've allowed to call `deauthorize_vault_authority_cap` when the Vault is frozen.

Fix Review:

Fix confirmed

L-16: Curator metadata fields use `Option::fill` and become permanently write-once

Severity: Low	Impact: Low	Likelihood: Low
Files: packages/market-making-vault/sources/utis/ metadata.move	Status: Fixed	

Description:

The curator setters mutate `Option<String>` fields with `Option::fill`, which aborts (`EOPTION_IS_SET`) if the field is already `Some`. If a curator field is provided at vault creation, the corresponding setter aborts forever making it permanently immutable. If a field is left `None` at creation, the setter works exactly once, after which any further update aborts. This prevents correction of an obsolete or compromised curator URL or logo.

Recommendations:

Replace `curator_*.fill(value)` with direct assignment `metadata.curator_* = option::some(value)` (or use a replace-or-insert behavior) so the setters update rather than fail.

Customer Response:

Fixed in commit [c7cfae8](#). The three setters now assign, matching `set_name` and `set_description`:

```
metadata.curator_name = option::some(curator_name);
metadata.curator_url = option::some(curator_url);
metadata.curator_logo_url = option::some(curator_logo_url);
```

Not changed: the setters take `String`, not `Option<String>`, so a field cannot be cleared back to `None` once set.

Fix Review:

Fix confirmed

L-17: OwnerWithdraw events report the pre-fee amount instead of the user's actual proceeds

Severity: Low	Impact: Low	Likelihood: High
Files: packages/market-making-vault/sources/vault.move	Status: Fixed	

Description:

When a withdrawal is processed via `end_withdraw_session`, the user's output `Coin` amount is calculated. If the withdrawal requires a performance fee, the fee is deducted and split into `owner_fees_balance`. The remaining amount is given to the user. However, when the `OwnerWithdraw` event is emitted, it uses the variable `coin_out_amount`, which represents the pre-fee amount, rather than the actual post-fee amount received by the user (`coin_out_amount_sans_fee`). This systematically overstates user proceeds and corrupts off-chain indexers and accounting systems.

Recommendations:

Update the `emit_owner_withdraw` event payload to use `coin_out_amount_sans_fee` instead of `coin_out_amount`. Prefer also emitting the fee amount explicitly so off-chain accounting can reconcile gross vs. net.

Customer Response:

Fixed in commit [c7cfae8](#). In `end_withdraw_session_`, `coin_out_amount` (vault.move:2932) is the gross amount pulled from the perps account, the owner fee is split off `coin_out` at :2963–2970, and :2977 captures what the user receives as `coin_out_amount_sans_fee`. `emit_owner_withdraw` was passed the gross figure, overstating proceeds by exactly the fee.

`emit_owner_withdraw` (:3053) now passes `coin_out_amount_sans_fee`.

Two related points from the same function. `emit_user_force_withdraw` (:2989) already used `coin_out_amount_sans_fee` and was correct; this matters because the fee block at :2956 is gated only on `lis_owner_locked_withdraw`, so a force withdrawal can also be charged a fee.

`emit_owner_locked_liquidity_withdraw` (:3044) used `coin_out_amount`, but its branch is `is_owner_locked_withdraw`, the condition that skips the fee, so the two values were equal there; it was switched to `coin_out_amount_sans_fee` anyway so the function reports net uniformly.

`withdraw_fees` (:941) emits the amount actually split and was already correct.

We did not add a fee field to the event. The gross amount is already emitted on chain in the same transaction by `perpetuals::account::withdraw_collateral` (account.move:258), which carries exactly `coin_out_amount`, so net comes from `OwnerWithdraw`, gross from that event, and the fee is the difference.

Fix Review:

Fix confirmed

L-18: Vault metadata setters are not gated on the vault's version or pause state, allowing mutation of a frozen vault

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/market-making-vault/sources/interface.move packages/market-making-vault/sources/vault.move	Status: Fixed	

Description:

Every `VaultMetadata` setter checks the global `Config` version and validates the supplied vault capability, but none checks the vault's version or pause state. Capability validation itself only checks the cap's association and active marker.

Consequently, an assistant can continue changing a vault's name, description, curator URLs, logo URL, and extra fields while that individual vault is frozen or indefinitely admin-paused. This is particularly problematic when the emergency action was taken because that assistant was compromised. The attacker can continue controlling user-facing metadata and publish phishing URLs while the vault owner is unable to revoke the assistant through the normal halted-vault path.

Recommendations:

Require the metadata setters to verify that the associated vault is neither frozen nor indefinitely admin-paused. If metadata editing during a halt is operationally necessary, restrict such edits to the vault `ADMIN`; active assistants should not retain metadata authority during an emergency halt.

Customer Response:

Fixed in commit [c7cfae8](#). All six setters (`interface.move:1844-1915`) ran only `config.assert_package_version()`, which checks the global `Config` version rather than the vault's, and `vault.assert_vault_authority_cap_is_valid(cap)`, which checks the cap's association and active marker. Neither reaches vault state, so name, description, curator name, curator url, curator logo url and extra fields stayed writable by an active assistant while the vault was frozen or indefinitely admin-paused.

Each setter now also runs:

```
vault.assert_package_version();  
vault.assert_vault_is_not_admin_paused();
```

The first is what rejects a frozen vault: `freeze_vault` sets `vault.version` to `u64::MAX`. The second covers the indefinite admin pause. The setters take no `Clock`, so temporary pauses cannot be evaluated without widening six signatures; the indefinite pause is the state relevant to an emergency halt.

We did not restrict metadata editing to the vault `ADMIN` during a halt. Blocking the write outright is stronger and leaves the signatures unchanged.

This closes the metadata half of the compromised–assistant scenario independently of **L-15**: a phishing curator url can no longer be published once the vault is frozen or admin–paused.

Fix Review:

Fix confirmed

L-19: 10u64.pow arithmetic panics and permanently DoSes the price feed for tokens with exponents ≥ 20 or ≤ -20

Severity: Low	Impact: High	Likelihood: Low
Files: packages/integrations/pyth/sources/ price_feed_storage.move packages/integrations/pyth-lazer/sources/ price_feed_storage.move packages/integrations/pyth-lazer-rolling/sources/ price_feed_storage.move	Status: Fixed	

Description:

The adapters compute decimal scale factors using Sui's standard `10u64.pow(...)`. Since the maximum value of a `u64` is $\sim 1.84e19$, calling `10u64.pow(e)` with any exponent $e \geq 20$ will unconditionally abort due to arithmetic overflow.

While typical assets have exponents between -8 and -18 , assets with very low values (like meme coins) or extremely high precision feeds can have exponents ≤ -20 (or ≥ 20). If a Pyth feed's exponent reaches 20 or -20 , all permissionless updates will panic, permanently DoSing the price feed and breaking downstream protocols relying on it. Even if an asset currently falls within the safe bounds, an unexpected price crash could prompt Pyth governance to update the exponent to ≤ -20 , triggering an instantaneous unrecoverable halt of the oracle aggregator.

Recommendations:

Compute the power in `u256` and explicitly validate the final normalized value against `u128::MAX`. For exponents too large for `u256`, return a dedicated unsupported-exponent error or, for sufficiently negative exponents whose normalized result is necessarily zero, apply a documented zero/rounding policy.

Customer Response:

Fixed in commit [d96b128](#). We've updated this to perform the conversion in [u256](#).

If the exponent is ≤ -38 we return zero (an invalid price) and if the exponent is ≥ 20 we throw an `EUnsupportedExponent` error.

Fix Review:

Fix confirmed

L-20: Integration upgrades cannot disable vulnerable entry points from prior package versions

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/integrations/ dev-source/sources/utils/init.move pyth-lazer-rolling/sources/utils/init.move pyth-lazer/sources/utils/init.move pyth/sources/utils/init.move stork/sources/utils/init.move switchboard/sources/utils/init.move pyth/sources/source.move switchboard/sources/source.move pyth-lazer-rolling/sources/price_feed_storage.move pyth-lazer/sources/price_feed_storage.move pyth/sources/price_feed_storage.move stork/sources/price_feed_storage.move switchboard/sources/price_feed_storage.move	Status: Fixed	

Description:

The integration packages have no integration-local version state. Their initialization functions only retain a [Publisher](#), while public price-update entry points rely exclusively on the core oracle aggregator's [Config](#) version check.

Sui package upgrades do not remove old immutable package versions. Consequently, upgrading an integration and adding a security check to its new update function does not prevent an attacker from invoking the corresponding function in the old package version. Compatible [Source](#), [FeedInfoObject](#), and [PriceFeedStorage](#) objects retain their defining type identities across upgrades, so the old code can continue operating on the same live objects.

The core [Config](#) gate does not solve this when only the integration is upgraded: its version is independent of the integration package version and therefore remains valid for the old integration code. Deauthorizing the source halts both old and new integration versions;

reauthorizing it also re-enables both. Thus an ordinary integration upgrade cannot selectively retire a vulnerable implementation.

Operators can currently recover only by coordinating a core oracle package version bump that rejects the old core calls, or by deauthorizing the old source and migrating every affected feed to a new source type and source ID. Until that migration completes, fixes published solely in an integration upgrade remain bypassable.

Recommendations:

Add centrally enforced integration generations keyed by `source_id`, and require every feed creation and update to prove the currently authorized generation. The enforcement must reside in versioned core code, because a check added only to a new integration version can simply be omitted by the old version.

For the existing deployments, retire vulnerable versions by:

1. Deauthorizing the old source.
2. Publishing a new source witness type and allocating a new source ID.
3. Removing and recreating affected feeds under that source.
4. Alternatively, coordinating the integration upgrade with a core `Config` version bump that makes calls through the old core package abort.

Future integrations should include a version gate from their initial deployment so that incrementing shared state invalidates all prior immutable package versions.

Customer Response:

Fixed in commit [d96b128](#). We've added the concept of versioning to integrations / Source. Now, each Source carries a `version: u64` field and each integration package carries a `CURRENT_VERSION: u64` constant (defined within the package's `source.move`).

This version check is compared on every relevant function.

Fix Review:

Fix confirmed

L-21: FeedInfoObject constructors return an owned object while permanently claiming a deterministic identity, letting a revocable assistant brick a feed

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/integrations/ pyth-lazer/sources/feed_info_object.move pyth-lazer-rolling/sources/feed_info_object.move stork/sources/feed_info_object.move	Status: Fixed	

Description:

The `FeedInfoObject` constructors combine two design choices that together make a canonical feed identity permanently brickable by any account holding a vendor `ASSISTANT` capability.

1. They permanently claim a deterministic identity. `new` calls `derived_object::claim` on the shared Source, keyed by `FeedInfoObjectKey<VendorKey>(feed_id | symbol)`. This claim is one-time and irreversible: once made, `child_exists` returns `true` for that key forever, so every later creation attempt for the same identity aborts with `EFeedInfoObjectAlreadyCreated`. Nothing — including revoking the caller or deleting the object — clears the claim.
2. They return the object by value instead of sharing it. `new` hands the resulting `key + store` object back to the caller. Making it a usable, permissionlessly-updatable shared object is a separate, manual step the caller must perform (`transfer::public_share_object`). Until then the object is owned by whoever called `new`.

Because these constructors are callable by revocable `ASSISTANT` caps, a compromised or malicious assistant can claim an identity the vendor intends to onboard and then dispose of the returned object in any way that denies it to the vendor — transfer it to an inaccessible address, keep it as an owned object, or create it with incorrect parameters. In every case the deterministic key is now permanently consumed.

The result is an unrecoverable denial of service on that feed identity:

- The vendor admin can never (re)create the `FeedInfoObject` — `new` aborts on the existing claim.

- The object was never shared, so it is also unusable for the permissionless update path.
- Revoking the assistant recovers neither the object nor the claim.
- The admin cannot work around it: the integration's package-private `SourceCap` is only reachable through wrappers that require the actual `FeedInfoObject`, so a bare derived ID cannot be substituted for the stranded object. Adding a destructor does not help either — deleting the object does not clear the derived-object claim.

Variant-specific notes:

- `pyth-lazer` keys by `feed_id`, so a squat is limited to that exact feed and is self-limiting if the object were shared on creation.
- `pyth-lazer-rolling` keys by `symbol`; its feed ids are mutable, but `migrate` only rotates (`front = old` next, then sets a new `next`) and cannot directly reset a wrong `front_feed_id`, so a mis-created rolling identity is only partially correctable even after sharing.
- stork feed objects are immutable, so a wrong claim cannot be corrected at all.

Restoring a bricked identity requires an abnormal migration (a new vendor key, new storage, new key derivation, or a package upgrade).

Recommendations:

Remove the raw create-and-return flow so a claimed identity cannot be stranded:

- Create and share (or freeze) atomically inside `new`. Share the `pyth-lazer` and rolling objects in the same call that claims the key; share or freeze the immutable stork object. The constructor should never return an owned object that a caller can withhold.
- Restrict identity-claiming constructors to `ADMIN`, keeping revocable assistants off the path that burns a deterministic key.
- Add an admin-controlled generation or registry mechanism so a burned or mis-created identity can be replaced. A destructor alone is insufficient — deletion does not clear the derived-object claim.

Atomic sharing closes the strand-to-inaccessible-address vector; gating the constructor to `ADMIN` and providing a replacement mechanism close the remaining irreversibility of the one-time claim.

Customer Response:

Fixed in commit [d96b128](#). We've applied our SharePolicy design pattern to FeedInfoObject constructors. Now these constructors return a hot potato SharePolicy that can only be destroyed when the FeedInfoObject is shared.

Fix Review:

Fix confirmed

L-22: update_price_feed unconditionally emits UpdatedPriceFeed events for stale prices, poisoning off-chain indexers

Severity: Low	Impact: Low	Likelihood: Medium
Files: packages/oracle-aggregator/sources/price_feed_storage.move packages/oracle-aggregator/sources/price_feed.move	Status: Fixed	

Description:

In `update_price_feed`, the transaction fetches the previous state, calls `set_price`, and then unconditionally emits an `UpdatedPriceFeed` event using the caller's submitted `price` and `timestamp_ms`. However, `set_price` silently short-circuits and ignores the update if `timestamp_ms <= price_feed.timestamp_ms`. If a stale or duplicate price update is submitted, the on-chain state remains unchanged, but the `UpdatedPriceFeed` event is still emitted with the caller's stale data. This falsely advertises a state transition that never happened, which can cause off-chain indexers, oracle caches, and monitoring systems to diverge from the actual on-chain state.

Recommendations:

Have `set_price` return a boolean indicating whether the feed was modified, and only emit `UpdatedPriceFeed` if the update was actually applied. Alternatively, re-read the values from the feed after `set_price` and emit the actual stored state along with an explicit `accepted` flag.

Customer Response:

Fixed in commit [d96b128](#)

Fix Review:

Fix confirmed

L-23: A revocable package assistant can permanently capture or exhaust source registrations

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/oracle-aggregator/sources/config.move	Status: Fixed	

Description:

`Config::new_source_id` is public and accepts `SourceKey` as a generic type argument without requiring a module-private witness. An active package assistant can therefore register a key belonging to another integration before that integration is onboarded and permanently store the non-droppable `SourceCap` inside their own object. This installs a `RegisteredSource` without creating a canonical `Source`, meaning legitimate creation attempts will abort with `ESourceAlreadyRegistered`. Revocation does not reverse this, and no recovery mechanism exists. Additionally, an assistant could theoretically loop this to exhaust the `u16` limit of `next_source_id`.

Recommendations:

Make `new_source_id public(package)` and expose source registration exclusively through a canonical `Source` constructor that demands an integration-defined, module-private witness. Provide an administrative recovery mechanism that can forcefully unregister or migrate a captured source ID.

Customer Response:

Fixed in commit [02b0d1a](#). `new_source_id` is now `public(package)`.

Fix Review:

Fix confirmed.

L-24: A paused market still blocks permissionless vault force-withdrawals

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/market-making-vault/sources/vault.move	Status: Partially Fixed / Acknowledged	

Description:

Commit [4704533](#) fixed the original broad pause cascade for deposits and normal withdrawals. These paths now skip `update_funding` when a market is paused and can complete as long as the live oracle remains valid:

- `process_clearing_house_for_deposit`
- `process_clearing_house_for_withdraw`

However, the permissionless force-withdraw path remains blocked when any clearing house associated with the vault is paused and its position still holds value.

`process_clearing_house_for_force_withdraw` still calls `update_funding` unconditionally:

[File: packages/market-making-vault/sources/vault.move:2307-2310](#)

```
// A position that still holds value needs to trade and deallocate collateral,
// which a paused market forbids.
clearing_house.update_funding(base_oracle, clock);
```

`update_funding` requires the market to be active, so both full-pause and cancel-only modes abort:

[File: packages/perpetuals/sources/clearing_house.move:1696-1702](#)

```
public fun update_funding<T>(...) {
    clearing_house.assert_package_version();
    clearing_house.assert_market_is_not_paused();
    // ...
}
```

[File: packages/perpetuals/sources/clearing_house.move:5215-5216](#)

```
assert!(ch.paused == constants::pause_mode_active!(), market_is_paused!())
```

Skipping only this call would not solve the problem: position closing and collateral deallocation also require an active market through `start_session` and `deallocate_collateral_`.

Each withdrawal session must process every clearing house in its `ch_ids` work list. The finalizer rejects a non-empty list at `vault.move:2892`. Therefore, the user cannot simply skip the paused market.

The impact: After the configured force-withdraw delay, an LP is expected to exit without vault-keeper cooperation. A single paused market defeats this guarantee and can block the withdrawal until a privileged admin resumes the market. For a permanently closed market, the vendor must first configure and enable settlement. Only then can anyone settle the vault position and permissionlessly remove the now-empty clearing house. The pause trigger requires a trusted pause or vendor role and does not directly cause loss of funds. However, recovery has no on-chain deadline and remains dependent on privileged liveness.

Recommendations:

Provide a force-withdraw route for paused and closed markets that does not require active-market trading or collateral deallocation. For closed markets, use enabled settlement prices and allow permissionless settlement before completing the withdrawal. Clearly define how a value-bearing position should be handled during a temporary pause so that one market cannot indefinitely block the vault-wide exit path.

Customer Response:

Fixed in commit [e5fa9f9](#). For a paused market whose position still holds value; one that holds none is already dropped permissionlessly (`vault.move:2301-2305`). Skipping `update_funding` would not help, since closing and deallocating both need an active market either way. `interface::close_position_at_settlement_prices` no longer takes a vault cap, so on a closed market anyone can settle at the prices the vendor already published, after which the position holds no value and the force withdrawal completes without the operator. A temporarily paused market stays blocked until it resumes: paying the exiting LP without touching that market would leave their slice of the position with the remaining LPs.

Fix Review:

Partially fixed / acknowledged. Commit e5fa9f9 removed the vault-cap requirement from `close_position_at_settlement_prices`. Once the vendor has closed the market and enabled settlement, any caller can settle the vault position; the now-empty clearing house can then be removed and the permissionless force-withdrawal can complete. This correctly fixes the closed-market operator-dependency subcase.

The temporarily paused, value-bearing case remains by design. The force-withdraw processor still reaches active-market guards in `update_funding`, position closing, and collateral deallocation, so the transaction rolls back until the market is resumed (or permanently closed and prepared for settlement). The customer's accounting rationale is sound: paying the exiting LP without reducing the shared position would transfer that LP's exposure to the remaining LPs. L-24 should therefore be marked Acknowledged / Partially Fixed.

L-25: Dust-path force-withdraw overcharges the fractional withdrawer by making them absorb the full whole-position close cost

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/market-making-vault/sources/vault.move	Status: Acknowledged	

Description:

When a force-withdraw process is applied to a market whose notional position is below `min_force_withdraw_position_usd` (default $\approx$ \$5, [vault.move:250](#)), the vault closes the entire position rather than the withdrawer's fraction:

[File: packages/market-making-vault/sources/vault.move:2455-2463](#)

```
let securely_close_entire_position = ifixed::less_than_eq(
  ifixed::mul(ifixed::abs(base_amount), mark_price),
  withdraw_session.vault.vault_params.min_force_withdraw_position_usd
);
let size_to_close = if (securely_close_entire_position) {
  base_amount_int
} else {
  min(size_to_close, base_amount_int)
};
```

The slippage and fees of that close are booked into `slippage_delta` over `size_to_close` (the whole position in the dust branch) and accumulated:

[File: packages/market-making-vault/sources/vault.move:2526-2531](#)

```
let slippage_delta = ifixed::sub(expected_value_difference,
  user_fees_share_to_pay); // negative on a cost
withdraw_session.accumulated_slippage = ifixed::add(
  withdraw_session.accumulated_slippage,
  slippage_delta
);
amount_to_withdraw = ifixed::add(amount_to_withdraw, slippage_delta);
```

`end_withdraw_session_` then computes the payout as:

[File: packages/market-making-vault/sources/vault.move:2913-2926](#)

```
let total_vault_balance_value = ifixed::add(
  ifixed::add(initial_vault_balance_value, vault_balance_value),
  ifixed::neg(accumulated_slippage) // Add slippage back, thus invert sign
);
let mut value_to_withdraw = multiply_by_rational_ifixed(
  ifixed::from_balance(lp_balance.value(), B9_SCALING),
  total_vault_balance_value,
  ifixed::from_balance(vault.lp_supply_value(), B9_SCALING),
);
value_to_withdraw = ifixed::add(value_to_withdraw, accumulated_slippage);
```

if closing the position costs S (so `accumulated_slippage` = $-S$) and vault NAV before the close is B_{pre} , the payout formula becomes:

```
value_to_withdraw = withdraw_ratio ×  $B_{pre}$  -  $S$ 
```

Example: withdrawer owns 25% of LP supply (`withdraw_ratio` = 0.25) and the whole-position close costs \$0.40.

- Fair pro-rata payout after the close: $0.25 \times (B_{pre} - \$0.40)$ = their share of NAV minus only 25% of the cost.
- What the code pays: $0.25 \times B_{pre} - \$0.40$ = their share of NAV minus 100% of the cost
- Overcharge: $\$0.30$ = the other LPs' 75% share of S , which the force-withdrawer subsidizes.

For a normal partial close, this is fair — S is only the cost of the withdrawer's own slice. In the dust branch the vault closes the entire market for everyone, but still bills that full cost to the one fractional force-withdrawer.

Recommendations:

In the `securely_close_entire_position` (dust) branch, charge the triggering withdrawer only their proportional share $withdraw_ratio \times slippage_delta$ and socialize the remainder across all LPs via the vault NAV, so a fractional withdrawer does not subsidize the whole-position close for the remaining LPs.

Customer Response:

Acknowledged. The branch triggers only when the entire position is worth at most `min_force_withdraw_position_usd`, \$5 by default (:250), so the overcharge is a fraction of the cost of closing a position that small, and the withdrawer also gets that market closed and dropped from the vault's clearing-house list. Pro-rating it needs a second slippage accumulator threaded through the force-withdraw processor and a split of the payout formula.

L-26: Unbounded `max_one_execution_amount_bps` causes a u64 multiplication overflow that permanently bricks execution of a committed TWAP order

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/perpetuals/sources/orders/twap_orders.move	Status: Fixed	

Description:

On every TWAP `execute`, the per-execution cap is computed as a **native u64** product. Move aborts on overflow (no wrap), so when `size * max_one_execution_amount_bps > u64::MAX` every `execute` fails before placing an order and the committed ticket is permanently stuck.

[File: packages/perpetuals/sources/orders/twap_orders.move:163-165](#)

```
macro fun max_one_execution_amount($new_details: &TWAPOrderDetails): u64 {
  ({ $new_details }.size * { $new_details }.max_one_execution_amount_bps) /
  10_000
}
```

Called from `resolve` / L717 on the `execute` path.

`bps` is unbounded: `new_details` only checks `size/chunks`; ticket creation stores a hash only. Sibling `bps` math in the same module already uses `u128` ([L695-697](#), [L867-870](#)).

Reachable with ordinary config: `bps = 10_000` overflows once `size > ~1.84e15`; unbounded `bps` overflows even modest sizes. No fund loss — `cancel` / `user_cancel` do not call this macro.

Recommendations:

Widen the product to `u128` before `/ 10_000` (mirror sibling paths), and validate `max_one_execution_amount_bps <= 10_000` in `new_details`.

```
((d.size as u128) * (d.max_one_execution_amount_bps as u128)) / 10_000 as u64
```

Customer Response:

Fixed in commit [67ebe6b](#). `execution_amount_bps` is now capped in `max_one_execution_amount()`.

Fix Review:

Fix confirmed.

L-27: TWAP per-slice size is not aligned to the market lot size, so execution reverts and silently bricks the order

Severity: Low	Impact: Low	Likelihood: Low
Files: packages/perpetuals/sources/orders/twap_orders.move packages/perpetuals/sources/clearing_house.move	Status: Fixed	

Description:

Every order handed to the book must be a multiple of the market's lot size:

[File: packages/perpetuals/sources/clearing_house.move:2403-2409](#)

```
let lot_size = hot_potato
    .clearing_house
    .market_params
    .lot_size();

// Check size is a multiple of `lot_size` and is not 0 after accounting for
reduce_only
assert!(size % lot_size == 0, size_not_multiple_of_lot_size!());
```

A TWAP per-slice size is computed without any lot alignment, then passed into `place_limit_order`:

[File: packages/perpetuals/sources/orders/twap_orders.move:167-169](#)

```
public(package) fun target_chunk_amount(new_details: &TWAPOrderDetails): u64 {
    new_details.size / new_details.chunks_amount
}
```

[File: packages/perpetuals/sources/orders/twap_orders.move:163-165](#)

```
macro fun max_one_execution_amount($new_details: &TWAPOrderDetails): u64 {
    ({ $new_details }.size * { $new_details }.max_one_execution_amount_bps) /
10_000
}
```

`resolve_fresh_and_total_execution_amount!` uses those values; `execute` forwards `amount` unchanged:

```
hot_potato.place_limit_order(  
    new_details.side,  
    amount,  
    limit_price,  
    constants::immediate_or_cancel!(),  
    ...  
);
```

Ticket creation cannot validate lot divisibility — it has no `lot_size`:

[File: packages/perpetuals/sources/orders/twap_orders.move:118-123](#)

```
assert!(  
    size != 0 &&  
    chunks_amount != 0 &&  
    size >= chunks_amount,  
    invalid_order_details!()  
);
```

Because `lot_size` is admin-configurable per market (as coarse as `1e9`), the mismatch is realistic and bricks the order two ways:

- **From the first attempt:** with a tight `amount_uncertainty_bps`, `assert_amount_within_uncertainty` forces the executor to request essentially exactly `target_chunk_amount`. If `size / chunks_amount` is not a lot multiple — e.g. `size = 10e9`, `chunks_amount = 3`, `lot_size = 1e9` → `target = 3_333_333_333`, and `3_333_333_333 % 1e9 != 0` — every `execute` aborts with `size_not_multiple_of_lot_size`. The ticket was created and gas escrowed, but no slice can ever fill.
- **Mid-flight:** once backlog builds (IOC slices only partially fill in thin liquidity), the resolved amount is forced toward `base_max = size * bps / 10_000`, which is likewise generically non-lot-aligned; inside a tight uncertainty band no request value is a lot multiple, and every subsequent `execute` aborts. Because slices are `IMMEDIATE_OR_CANCEL`, any participant keeping opposing liquidity thin can push a ticket into this state.

No funds are lost — the owner can recover escrowed gas and any partial-fill collateral via `user_cancel_twap_order`. This is a liveness/availability defect: a core order type silently fails to make progress (painful for a market-making vault that relies on TWAP).

Recommendations:

Validate at ticket creation that `size` and `size / chunks_amount` are multiples of the target market's `lot_size`. Alternatively, if the protocol intentionally leaves TWAP parameter validation off-chain, document the lot-alignment requirement and enforce it across all official front-end, SDK, and vault order builders before ticket creation. Specifically, require both `size` and every derived execution slice to be multiples of the market's current `lot_size`. This is a defense-in-depth mitigation; on-chain validation remains preferable because custom clients can bypass it.

Customer Response:

Fixed in commit [0255d41](#) Aligned TWAP execution amounts with the market lot size. Lot-size compatibility is validated on the first and every subsequent execution, before any fill, preventing partially executed orders from later becoming un-executable.

Fix Review:

Fix confirmed.

L-28: Accepted fee settings can make rounded maker rebates larger than taker fees and abort a trading session

Severity: Low	Impact: Medium	Likelihood: Low
Files: packages/perpetuals/sources/market.move	Status: Fixed	

Description:

The fee validator accepts a negative maker fee when its absolute value is equal to the taker fee. It checks the rates, but it does not reserve any amount for rounding:

[assert_market_fees](#)

```
fun assert_market_fees(registry_config: &RegistryConfig, maker_fee: u256,
taker_fee: u256) {
    // ...
    assert!(
        !ifixed::is_neg(maker_fee) ||
        (!ifixed::is_neg(taker_fee) &&
ifixed::less_than_eq(ifixed::abs(maker_fee), taker_fee)),
        negative_maker_fee_not_covered!()
    );
    // ...
}
```

Maker fees are calculated separately for every fill at [clearing_house.move:3566](#). The calculation uses `ifixed::mul`, which rounds a negative result toward negative infinity at [ifixed.move:224](#). As a result, a fractional maker rebate is rounded away from zero for each fill.

[The taker fee is different](#). The protocol first adds all filled quote amounts and then floors the fee once:

```
let taker_filled_volume = quote_filled_ask + quote_filled_bid;
// ...
let taker_fees_mag = macros::mul_positive!(taker_fee_mag, taker_filled_volume);
```

For maker rebate rate m , taker fee rate t , fixed-point scale W , and per-fill quote amounts q_i , the session fee is $\text{floor}(t * \sum(q_i) / W) - \sum(\text{ceil}(m * q_i / W))$.

When $t = m$, any fractional remainder can make this value negative. `end_session_` then aborts at [clearing_house.move:2950](#):

```
total_fees = ifixed::add(
    total_fees,
    ifixed::add(taker_fees, taker_integrator_fees)
);
total_open_interest = ifixed::add(total_open_interest, taker_open_interest);
assert!(!ifixed::is_neg(total_fees), negative_fees_accrued!());
```

For example, with maker fee -10 bps, taker fee 10 bps, and raw fill quote $q = 500_000_001_500_000_001$, the accepted configuration produces a maker rebate of $-500_000_001_500_001$ and a taker fee of $500_000_001_500_000$. The session total is -1 , so the transaction aborts and all matching changes roll back.

The [current fee policy supplied by the project](#) uses a -0.5 bps maker fee and a taker fee of at least 2.6 bps. After the maximum 5% referral discount, the effective taker fee is still 2.47 bps. This spread covers the rounding difference for every fill whose raw B18 quote is at least $5,077$.

The remaining risk is that the contract does not enforce the operational fee policy or a minimum rounding reserve. A future fee update, a new market with very small order units, or a partial resting order below the normal minimum order value can reintroduce the abort condition. The order admission code explicitly permits such a partial remainder at [clearing_house.move:2439](#).

The direct failure is a transaction abort, but its effect can repeat at the orderbook level. Matching first reduces a partially filled maker order at [clearing_house.move:3633](#) or removes filled orders at [clearing_house.move:3212](#), and `end_session_` checks the final fee total later. If that check aborts, Move rolls back the orderbook change, so the same maker liquidity remains and can make later matching attempts abort again. With an unsafe equal-rate configuration, this can stop most matching for the market. With the current positive fee spread, a dust order affects only transactions whose total positive fee buffer is too small; a larger trade can absorb the rounding difference and consume it. An attacker would need enough dust orders to overcome that buffer, so a broad orderbook DoS under the current production policy has not been demonstrated.

The issue can therefore cause a repeatable matching DoS, not permanent orderbook corruption. The market can recover when the order owner cancels the affected orders or the operator changes the fee settings. It does not commit a negative balance, steal funds, or create bad debt.

Recommendations:

Make the fee rule safe in contract code instead of relying only on the current fee policy.

The preferred fix is to use compatible rounding for both sides. For example, round a negative maker rebate toward zero, or calculate both maker and taker fees per fill before adding them to the session total. This prevents a rounded maker rebate from exceeding the fee collected for the same fill.

If the current calculation must remain, reject fee settings that do not include enough reserve for the smallest possible fill. The check should include maker and taker rates, referral discounts, `lot_size`, `tick_size`, and partial resting orders.

Add regression tests for equal maker and taker rates, fragmented fills, and a small partial remainder. Each successful trading session should keep `total_fees` non-negative after rounding.

Customer Response:

Fixed in commit [f1d6feb](#). Changed the rounding of the multiplication to be towards 0 instead of negative infinite.

Fix Review:

Fix confirmed.

Informational Severity Issues

I-01: Price update from oracle with timestamp far in the future means this value can't be updated

Files:

packages/integrations/oracle-aggregator/sources/price_feed.move

Description:

`oracle_aggregator::price_feed::set_price` will prevent overwriting a more recent price timestamp with an older price at an earlier price stamp on this line.

```
if (timestamp_ms <= price_feed.timestamp_ms) return;
```

However, there are no checks bounding how far `timestamp_ms` could be in the future. If such a value ever poisoned the feed it could not be corrected except by removing the oracle and adding it again. Since the feeds used by Aftermath are Pyth (and its Lazer variants), Stork and Switchboard all sign their updates this can only happen if the price feed itself is compromised. Interestingly, at least Pyth Lazer explicitly allows a timestamp slightly in the future because it uses a `abs` to calculate the timestamp difference in `check_price_is_fresh`. See [pyth.move:L408](#). Nevertheless, it may make sense to bound how far `timestamp_ms` can be in the future for the unlikely scenario that an oracle feed is compromised and a poisoned timestamp value is set.

Recommendations:

Define a constant that captures the maximum time in the future that an oracle timestamp could be and add a check that it does not exceed this.

Customer Response:

Acknowledged. This is a piece we have gone back and forth on through development and now three audits and have landed on the current implementation. We have had three different implementations across: (1) restricting only past timestamps, (2) allowing future timestamps with some bound, and (3) the current implementation.

With (1), we ran into issues where there were sometimes slightly “future” timestamps and we would skip their update. With (2), we found the gas increase to not be worth the benefit. We’ve settled on (3).

Our reasoning for (2) vs (3) is that the exploit this finding is referring to requires a compromised downstream oracle. In that case we need to combat malicious timestamps and malicious prices. For both our response is to freeze the oracle and assess the situation likely manually intervene. Our expected intervention is one of: (i) flip to dev source (or another supporting oracle) while the oracle is compromised or (ii), if the oracle issue is resolved, to remove and re-add the oracle to flush the invalid price / timestamp.

I-02: Validating against empty RESTRICTED_KEYS

Files:

packages/vendor/sources/metadata.move

Description:

In `set_extra_field` the supplied field key is checked against `RESTRICTED_KEYS`, disallowing the use of restricted keys:

```
assert!(!RESTRICTED_KEYS.contains(key.as_bytes()), ERestrictedKey);
```

`RESTRICTED_KEYS` is defined in the module as

```
/// Set of keys that cannot be used as keys in `extra_fields` to ensure that the
/// integrity of
/// important fields that need to be maintained by the package itself.
const RESTRICTED_KEYS: vector<vector<u8>> = vector[];
```

Since `RESTRICTED_KEYS` is empty, the assert will never fail and is basically a no-op.

Recommendations:

Set values for `RESTRICTED_KEYS` or remove the const and the assert. In case the restricted key functionality is added for future use, document this in the comments.

Customer Response:

Fixed in commit [3c85413](#). Pulled `RESTRICTED_KEYS` into the `Config`, so modifying the set of restricted keys does not require a package publish. `config.move` gained permissioned mutators for `config.restricted_keys`.

Fix Review:

Fix confirmed.

I-03: Dead public(package) functions across multiple packages

Description:

Several `public(package)` functions have no callers within their own package. Unlike private functions, `public(package)` functions cannot be eliminated by the compiler and are included in deployed bytecode.

Findings

perpetuals

- `orderbook::change_maps_params` — no callers; B+ tree params are set once at construction and never updated
- `authority::assert_is_admin` — never called; its sibling `assert_is_not_admin` is called from `registry.move`

oracle-aggregator

- `authority::assert_is_admin` — same pattern as above

vendor

- `config::current_version` — wraps the `CURRENT_VERSION` constant; all version comparisons use the constant directly

market-making-vault

- `errors.move` — 7 error codes defined but never thrown: `invalid_vault_assistant_cap`, `package_already_created`, `invalid_collateral_pfs_tolerance`, `queue_is_not_empty`, `force_withdraw_tolerance`, `queue_is_empty`, `wrong_vault_for_session`

These suggest removed code paths whose error definitions were not cleaned up.

- `metadata::assert_vault_admin_cap_has_authority` — a cap validity assertion that is never invoked anywhere in the package; the corresponding authorization check is silently absent.

- `authority.move`

- `assert_is_admin_or_assistant` — role check accepting ADMIN or ASSISTANT; never called, all admin-gated functions in the package use the type system directly (`AuthorityCap<..., ADMIN_OR_ASSISTANT>`) rather than this runtime assertion
- `assert_is_admin` — stricter role check accepting only ADMIN; same situation, never called

- `config.move`

These functions are only used in tests and can be specified as `#[test_only]`

- `vault_record_vault_id` — returns `VaultRecord.vault_id`
- `vault_record_owner_cap_id` — returns `VaultRecord.owner_cap_id`
- `vault_record_metadata_id` — returns `VaultRecord.vault_metadata_id`
- `vault_record_created_at_ms` — returns `VaultRecord.created_at_ms`
- `vault_record` — returns `&VaultRecord` for a given vault ID
- `user_lp_coin_record_user_lp_coin_id` — returns `UserLpCoinRecord.user_lp_coin_id`
- `package_admin_cap_id` — returns the derived cap ID for the package admin

- `vault.move`

- `account_cap_mut` — returns `&mut AuthorityCap<ACCOUNT, ADMIN>`; unused but exposes mutable access to the vault's account cap if ever called.
- `last_paused_timestamp_ms` — returns when `pause_vault_for_force_withdraw` was last called
- `min_pause_vault_for_force_withdraw_frequency_ms` — returns the anti-spam cooldown for pause calls
- `min_force_withdraw_position_usd` — returns the USD threshold below which a position is fully closed during force-withdraw
- `owner_fee_rate` — returns the vault owner's profit share rate
- `owner_fees` — returns the accrued but uncollected owner fee balance

- `withdraw_request_user_lp_coin_id` — returns the LP coin ID stored in a pending withdraw request

Recommendations:

Remove any function that serves no production purpose or comment if they are for future use.

The 7 orphaned error codes in `errors.move` should be audited to confirm the code paths they guarded are intentionally gone.

Customer Response:

Fixed in commit [3c85413](#), [67c846f](#), [7007b4c](#), [d96b128](#) and [e5fa9f9](#)

Fix Review:

Unused code has been removed and test only functions are now `#[test_only]`.

The 7 unused errors have not been removed.

I-04: Redundant positivity checks in set_config parameter validation

Description:

Two `> 0` assertions in `registry::set_config` are implied by adjacent constraints in the same assertion and can never independently fire:

- `max_funding_period_ms > 0` is redundant: the same assertion already enforces `min_funding_frequency_ms > 0`, `min_funding_frequency_ms < min_funding_period_ms`, and `min_funding_period_ms < max_funding_period_ms`. Together these imply `max_funding_period_ms > min_funding_period_ms > min_funding_frequency_ms > 0`.
- `max_proposal_delay_ms > 0` is redundant: the same assertion already enforces `min_proposal_delay_ms > 0` and `min_proposal_delay_ms <= max_proposal_delay_ms`, which implies `max_proposal_delay_ms >= min_proposal_delay_ms > 0`.

These checks add visual noise and may give a false impression that they guard against a distinct condition, making the validation logic harder to read and reason about.

Recommendations:

Remove the two redundant `> 0` checks.

Customer Response:

Fixed in commit [67c846f](#)

Fix Review:

Fix confirmed. Redundant checks are removed.

I-05: Mark price can be manipulated by up to `max_book_index_spread` by trader

Files:

`packages/perpetuals/sources/market.move`

Description:

A trader can manipulate the *mark price* because the mark price is a “median of three” calculation based on the

- TWAP price (calculated from `index_twap_price` and `spread_twap`)
- funding price (calculated from `calculate_funding_price`)
- book price (the mid of the lowest ask and highest bid)

However, two of those values are correlated: book price and TWAP price. TWAP price is correlated to book price because TWAP price is calculated as the sum of `index_twap_price` and `swap_spread` which is a *TWAP of deltas between book price and index price*.

That a trader can manipulate the book price (as some cost to themselves) is obvious, but the TWAP price is also manipulatable because of the correlation. Thus, the trader can cause one of those two manipulated values to become the median in the “median of three”.

The extent to which they can manipulate the price is limited by a configuration parameter known as `max_book_index_spread` which is set to 5% by default. *But this is enough to cause liquidations to occur when they shouldn't.*

A trader could perform the manipulation by:

- buying up all the asks
- placing a very high ask

This shifts the mid between the lowest ask and the highest bid, thus shifting the book price. The `spread_twap` is continuously updated with new values. However, if `spread_twap_period_ms` elapses then maximum contribution of the `clipped_book_price` can be applied to the `spread_twap` which then moves the TWAP price overall. This is most likely to occur in markets without much activity.

Recommendations:

There is no obvious simple fix but having two of the prices be correlated is the root cause of the issue that should be fixed.

Customer Response:

Acknowledged. We are going to use the `max_book_index_spread` field to control this based on the leverage we set for the market.

I-06: Oracle freeze documentation does not match downstream behavior

Files:

packages/oracle-aggregator/sources/config.move

Description:

The documentation for `freeze_package` states that freezing the oracle halts all price reads and, consequently, pricing in dependent protocols ([config.move](#)). The implementation has narrower behavior: it changes the version stored in the oracle `Config`:

```
/// Freeze the entire package: `Config.version` is set to `u64::MAX`, failing
every version gate
/// mutations and price reads alike – in every package version, until
`unfreeze_package` is
/// called. Because all price reads are version-gated, freezing this package also
halts any
/// dependent protocol's pricing (e.g. perpetuals trading); this blast radius is
intentional.
///

public fun freeze_package(
    config: &mut Config,
    cap: &AuthorityCap<PACKAGE, FREEZE_GUARDIAN>,
) {
    /// ...
    let resume_version = config.version;
    df::add(&mut config.id, FrozenVersion(), resume_version);
    config.version = std::u64::max_value!();

    /// ...
}
```

This makes functions that call `config.assert_package_version()` abort, including new price updates and the aggregate read APIs in `price.move`. It does not modify or invalidate prices already stored in `PriceFeedStorage`.

The raw accessors used by perpetuals and the market-making vault do not receive the oracle `Config`. `price_feed_storage::price_feed` only takes `&PriceFeedStorage` and a source ID ([price_feed_storage.move:252](#)), while `price_and_timestamp_ms` and `twap_price` only take `&PriceFeed` ([price_feed.move:125](#), [price_feed.move:129](#)). A perpetuals transaction can therefore continue reading the last stored quote after the oracle package has been frozen:

Because these consumers do not observe the oracle `Config`, they continue operating until their own freshness limits or protocol-specific emergency controls stop them. Under the default perpetuals configuration, the primary stored-price window is approximately 10 seconds ([market.move:786](#)). In practice, an oracle incident should also trigger the circuit breakers of affected downstream protocols. Therefore, the main concern is that the current comment overstates the effect of `freeze_package` and could lead operators to assume that one oracle transaction automatically stops every consumer.

Recommendations:

Update the `freeze_package` comment to describe its actual scope: it stops oracle operations protected by the `Config` version check, but it does not invalidate quotes already stored in `PriceFeedStorage` or automatically freeze downstream protocols.

Update the incident-response runbook so that an oracle freeze is coordinated with the emergency controls of every affected consumer, including perpetuals and market-making vaults. The runbook should identify the relevant objects and authorized roles, define the expected transaction ordering, and require operators to verify that downstream protocols are frozen rather than assuming that the oracle freeze propagates automatically.

Customer Response:

Fix in commit [46d1e0a](#) .

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.